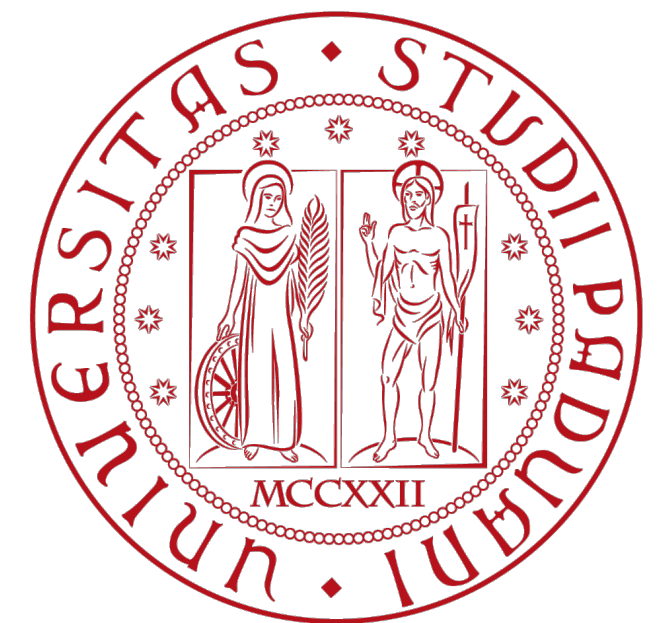
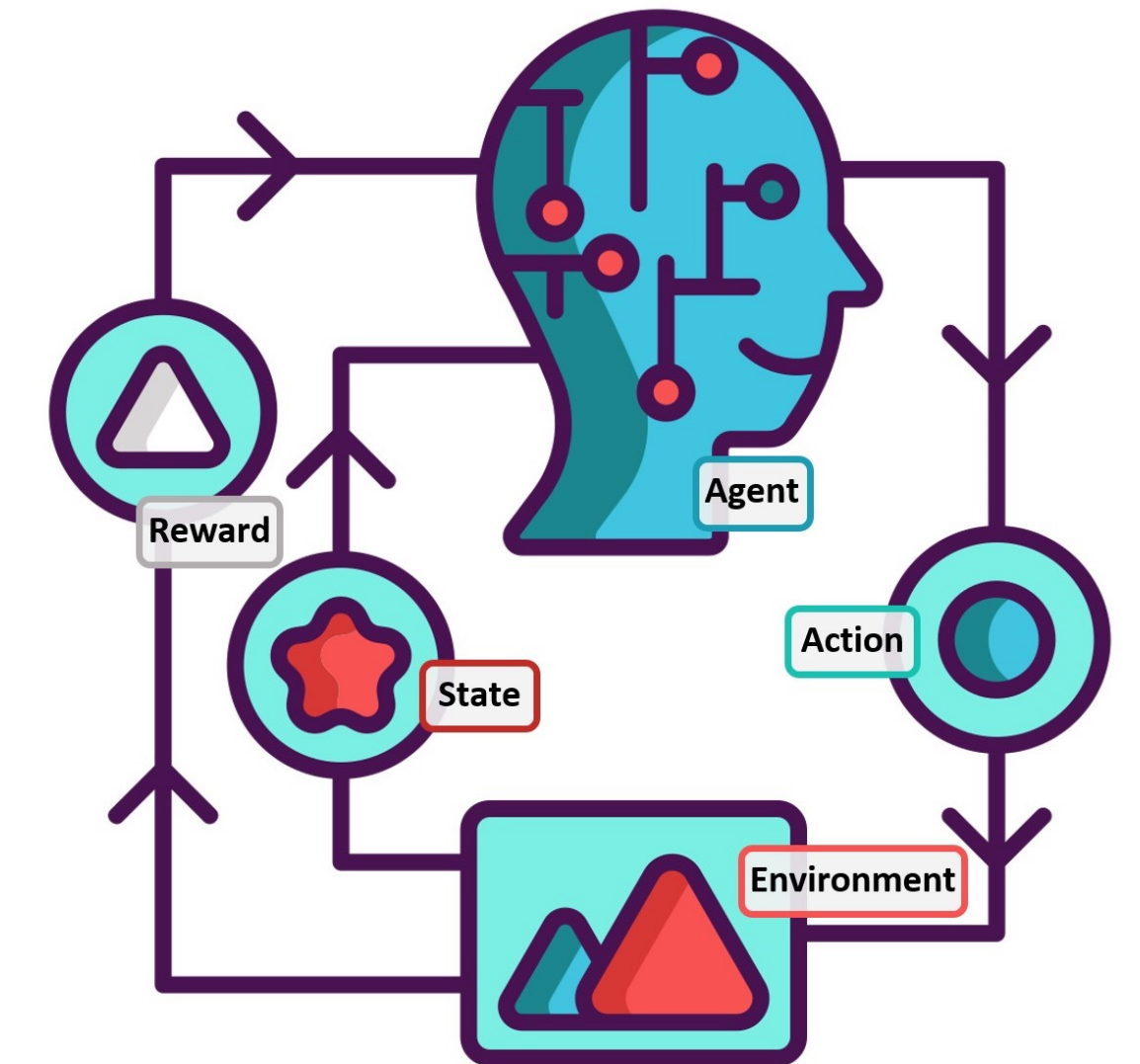


Reinforcement Learning

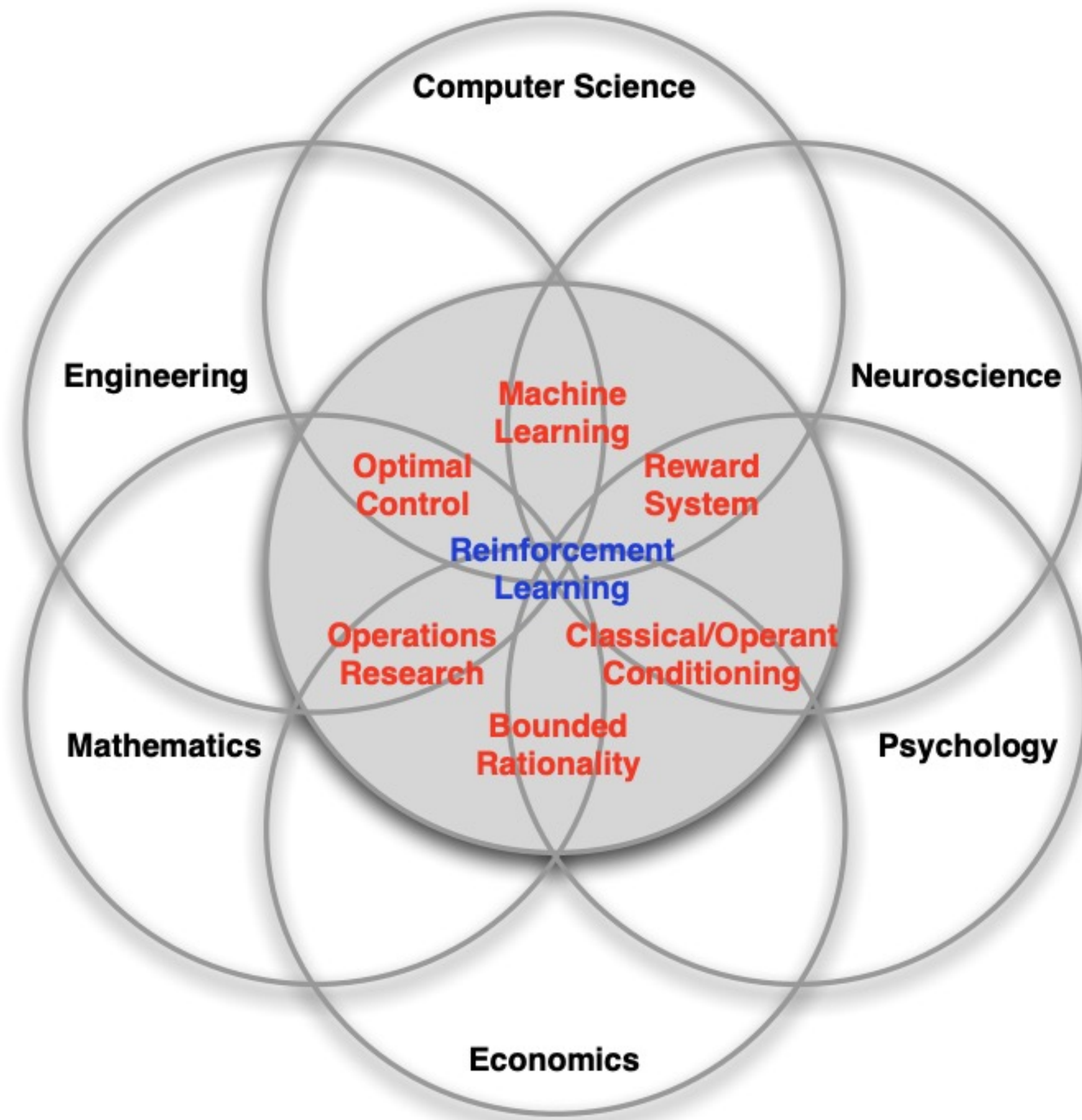
Introduction to Deep Learning #01

Gian Antonio Susto
Ruggero Carli

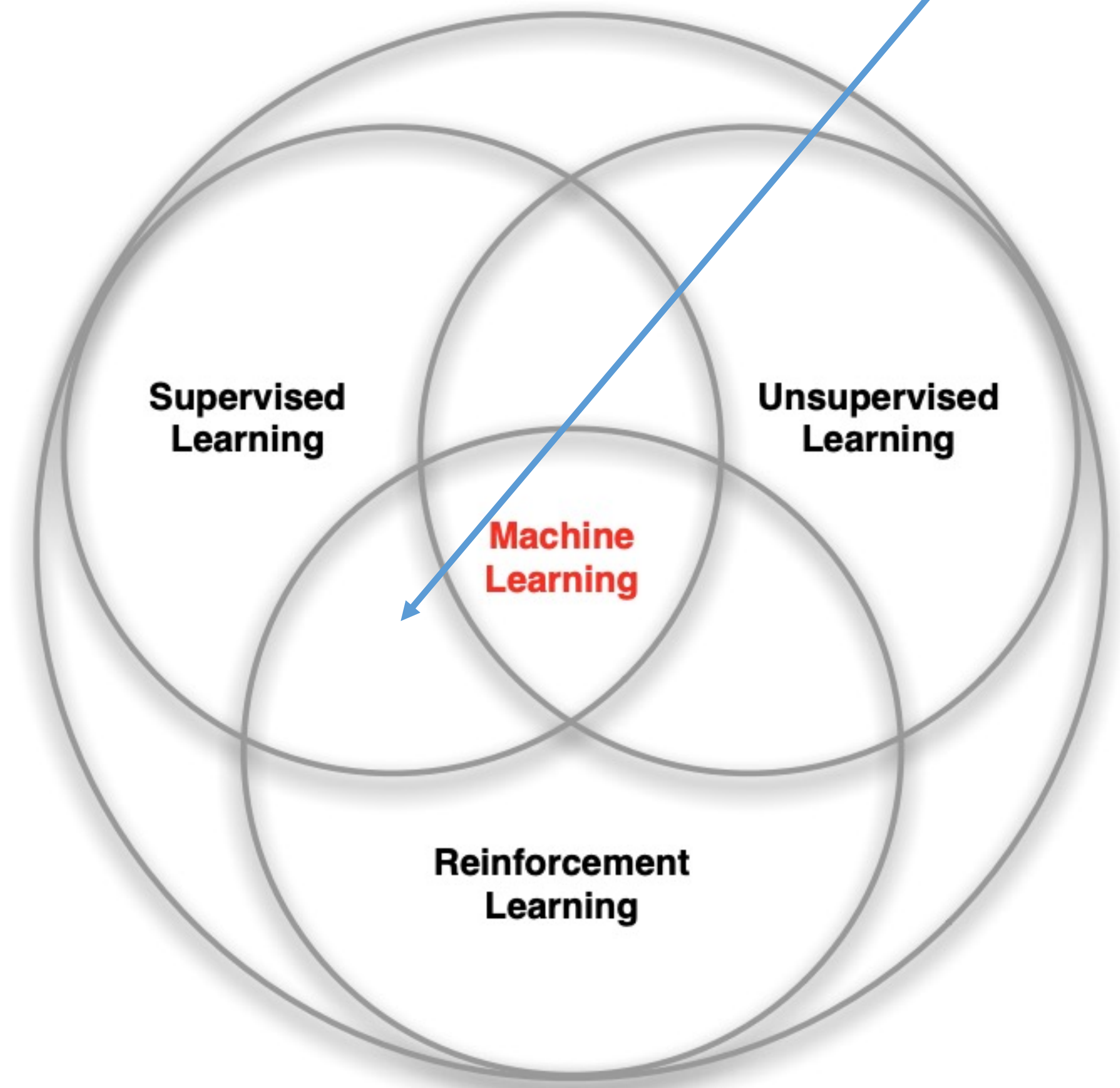


Why talking about Deep Learning in a Reinforcement Learning class?

From Lecture #01



Supervised Learning
approach are used in
solving RL problems



Approximate Solution Methods

With problems with really large state space (or state-action space), computation of 'exact' value function is unfeasible.

Instead of considering tables representations, we will consider parametric approximation of value functions (and state-value functions):

$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$

where $\mathbf{w}$ are the weights of a supervised model.

Chess



Number of states $\sim 10^{123}$
41 zeros

Go



Number of states $\sim 10^{360}$
120 zeros

Approximate Solution Methods

With problems with really large state space (or state-action space), computation of 'exact' value function is unfeasible.

Instead of considering tables rather than
we will consider parametric approximations
value functions (and state-value functions)

$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$

where $\mathbf{w}$ are the weights of a supervised model.

Deep Learning techniques are nowadays the most popular approaches for dealing with supervised learning

Chess



Number of states $\sim 10^{123}$
41 zeros

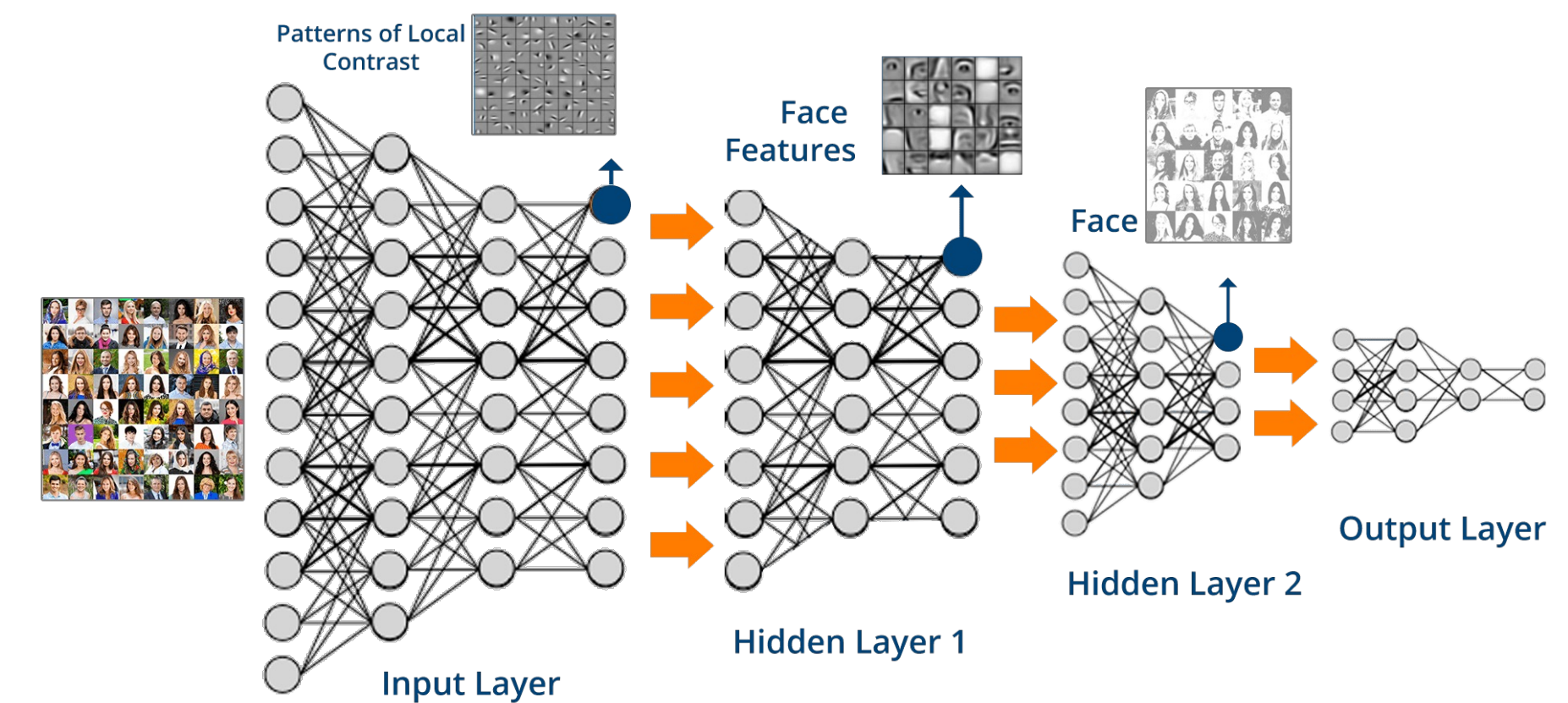
Go



Number of states $\sim 10^{360}$
120 zeros

Deep Learning

- Deep Learning (DL) techniques are nowadays the most popular approaches for dealing with supervised learning in RL:
1. Best performances over other methods (not always, we will discuss this later)
 2. DL is also used to handle non-tabular data (like images or text)
 3. DL architectures can be trained incrementally, as we do in many RL scenarios



Intro to Deep Learning

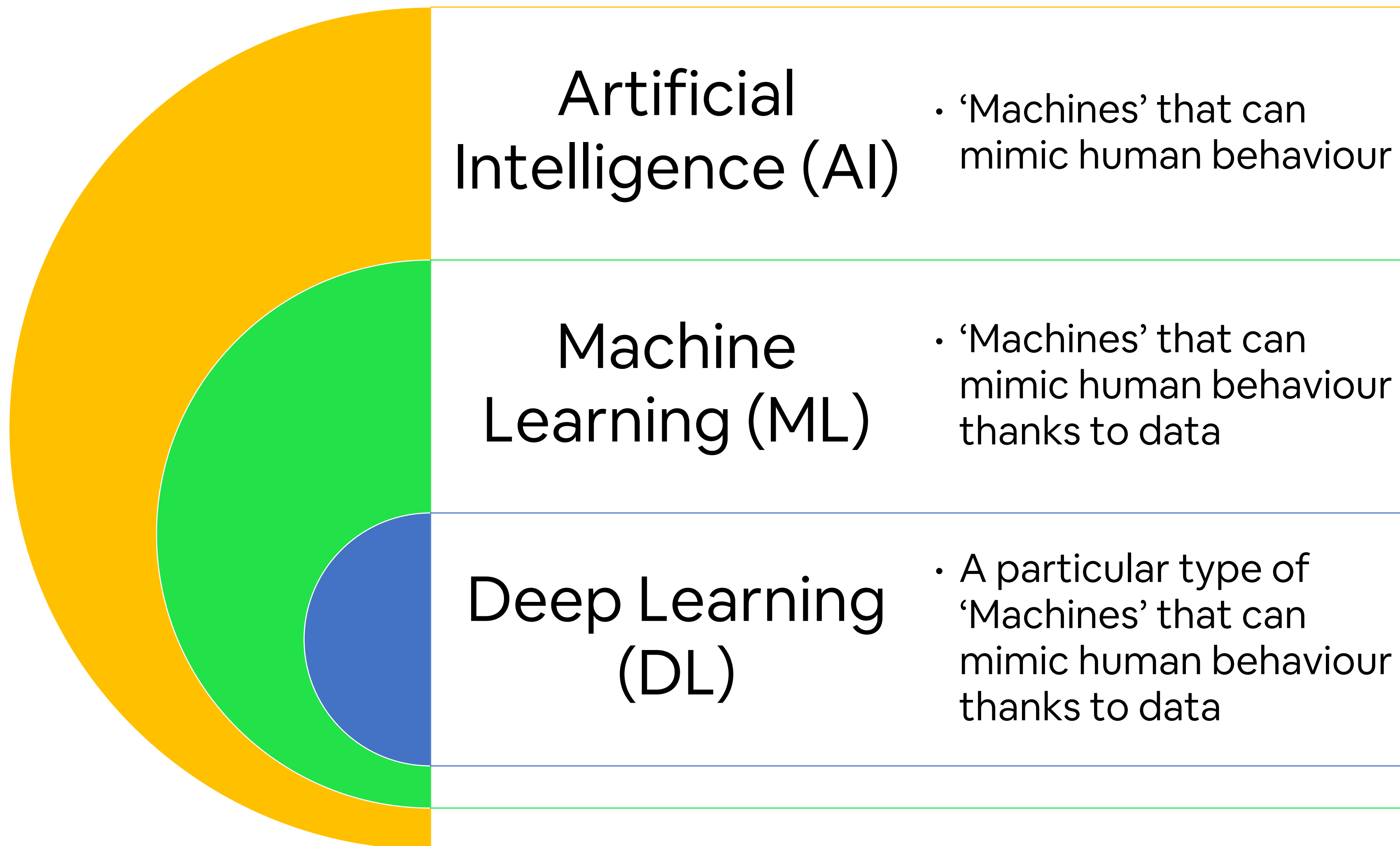
Reference Material (used for this presentation):

- MIT *Introduction to Deep Learning* <http://introtodeeplearning.com>
- L. Fridman *Deep Learning* <https://deeplearning.mit.edu/>
- I. Goodfellow, Y. Bengio, A. Courville. *Deep learning*. MIT press, 2016.
- G. Prando. *Seminars on Deep Learning*. UniPD, 2017 (Not available on-line)
- ImageNET <http://www.image-net.org/>
- C. Szegedy et al. *Going Deeper with Convolutions* CVPR2015
- B. Turovsky *Found in translation: More accurate, fluent sentences in Google Translate* <https://www.blog.google/products/translate/found-translation-more-accurate-fluent-sentences-google-translate/>
- A. Ng *Nuts and Bolts of Applying Deep Learning* <https://www.youtube.com/watch?v=F1ka6a13S9I>
- I. Goodfellow et al. *Explaining and harnessing adversarial examples* ICLR 2015
- A. Siarohin et al. *First Order Motion Model for Image Animation* NeurIPS 2019

Other references:

- Wired [Interview with Geoffrey Hinton](#) 13/05/2019
- <https://paperswithcode.com/>
- The Journal of Open Source Software <https://joss.theoj.org/>

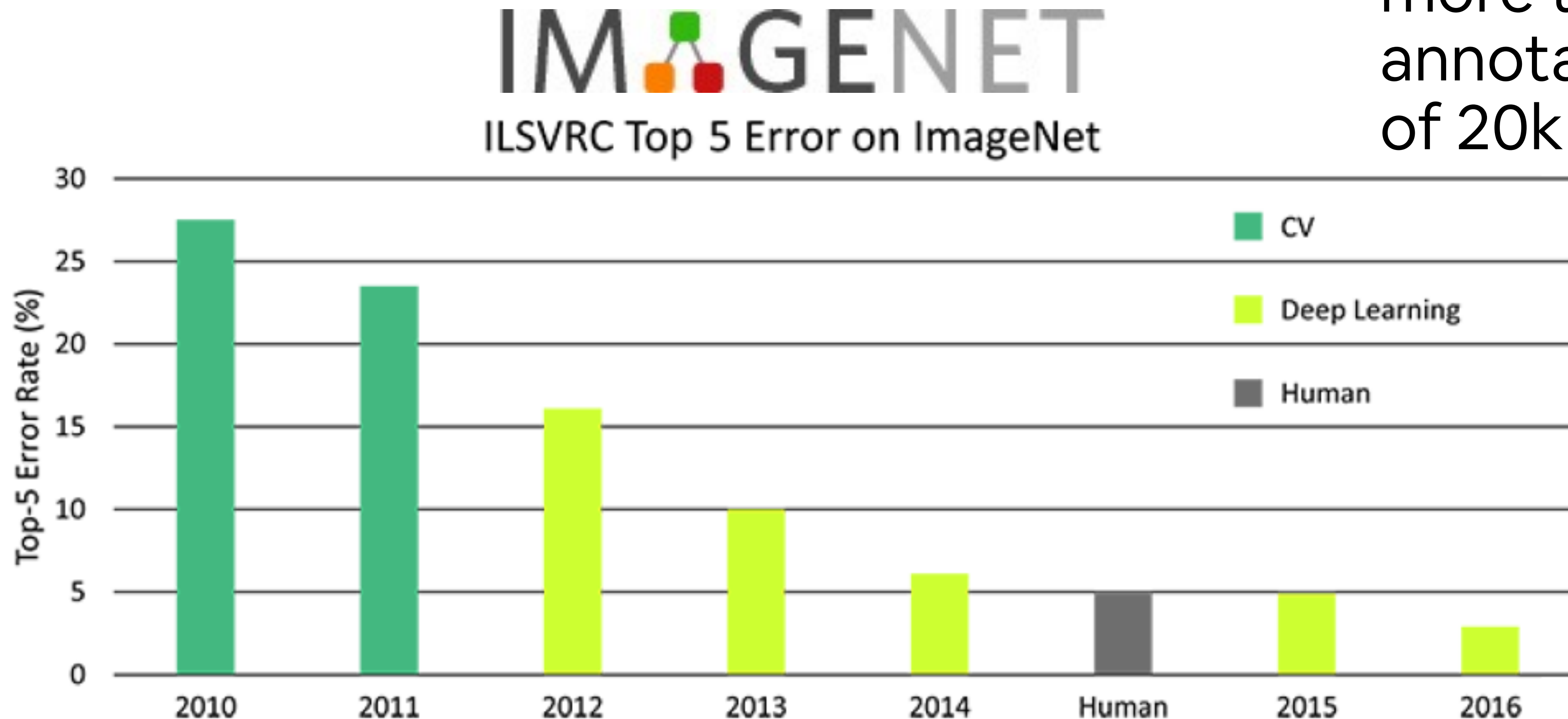
From Lecture #01



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances

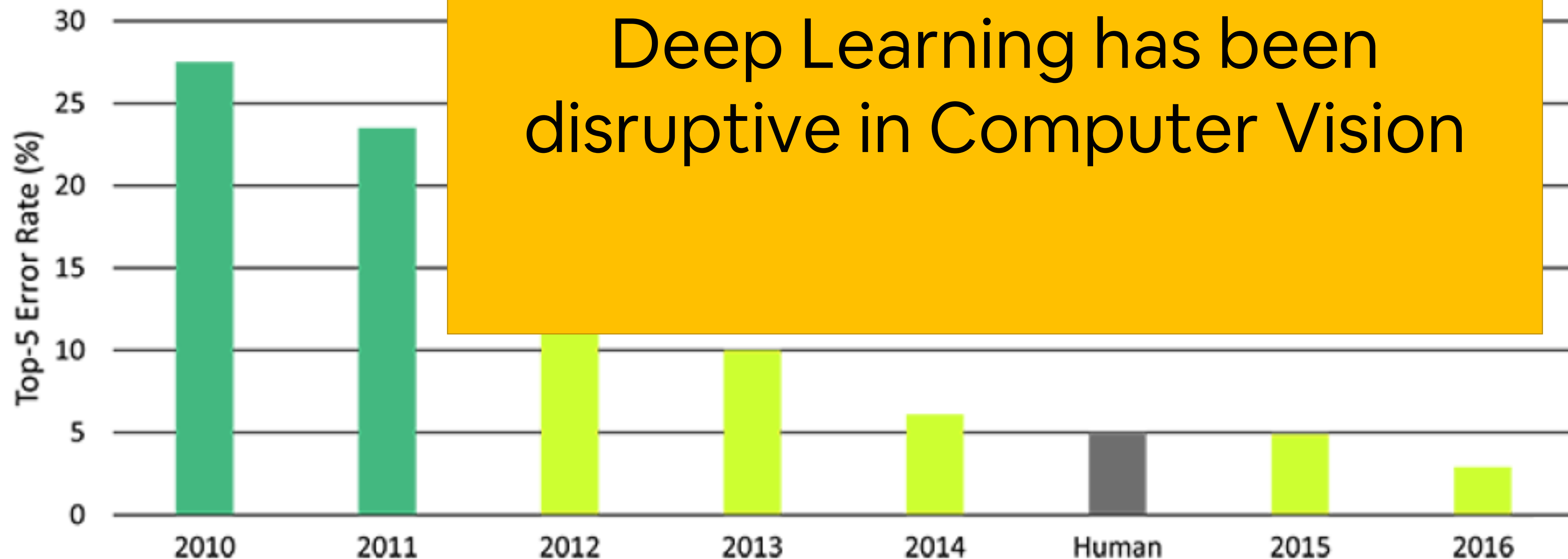
A dataset with more than 14 Million annotated images of 20k categories



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances

A dataset with more than 14 Million annotated images across categories



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances

Usage of Neural Machine Translation by Google Translate (15 November 2016)

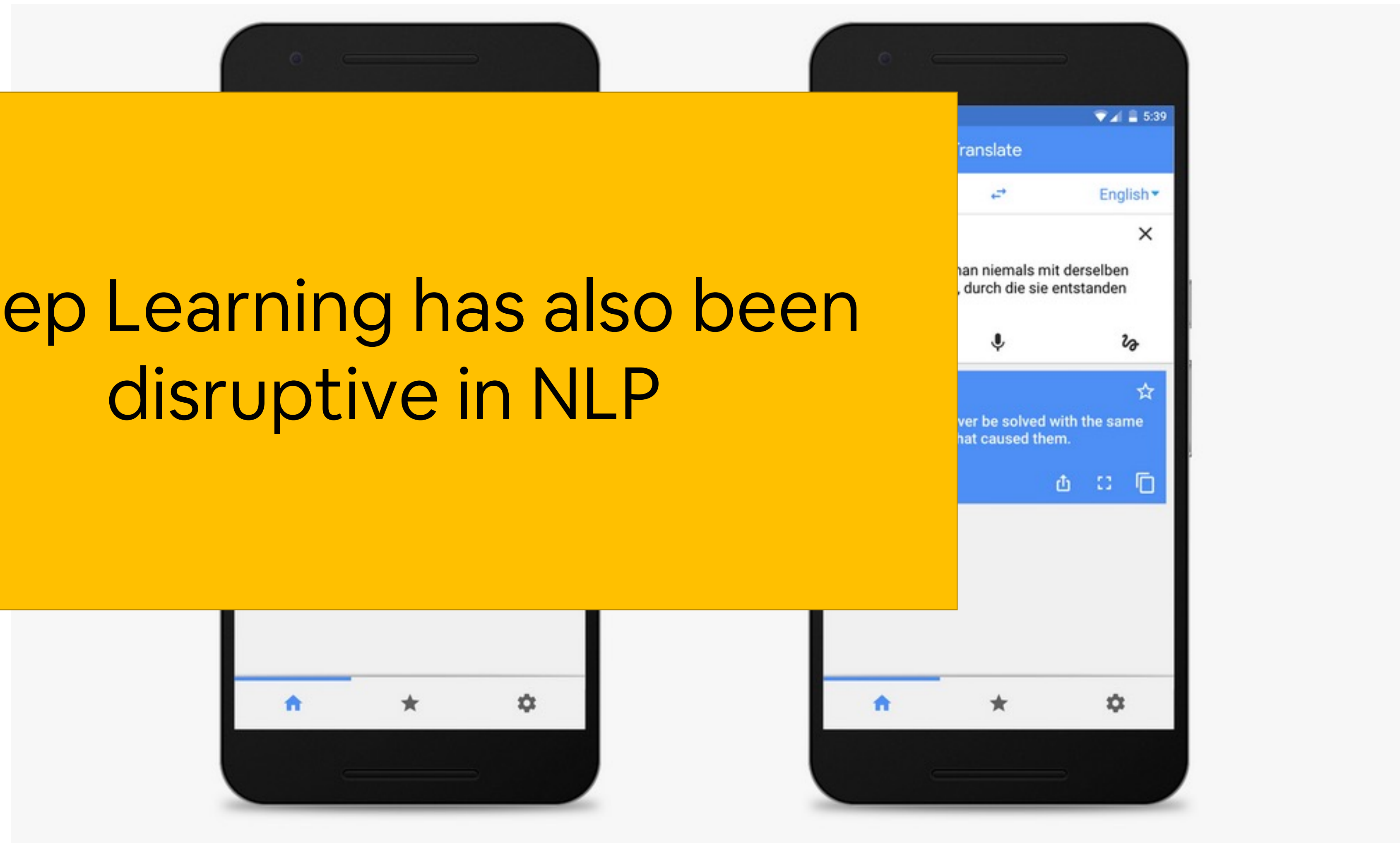


There are 3 principal reasons for choosing Deep Learning as a modeling tool

- Performances

Usage of Neural Machine Translation by Google Translate (15 November 2016)

Deep Learning has also been disruptive in NLP



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances

ChatGPT (Generative Pre-trained Transformer): a ‘super’ chatbot launched by OpenAI in November 2022.

ChatGPT is based built on top of a GPT-3 language model





can you write a python code for image preprocessing?



Sure! Here's some Python code that demonstrates some basic image preprocessing techniques:

 Copy code

```
import cv2
import numpy as np

# Reading image
image = cv2.imread("image.jpg")

# Resizing image
resized_image = cv2.resize(image, (224, 224))

# Converting image to grayscale
gray_image = cv2.cvtColor(resized_image, cv2.COLOR_BGR2GRAY)

# Normalizing image pixels
normalized_image = cv2.normalize(gray_image, None, 0, 255,
cv2.NORM_MINMAX)

# Applying Gaussian blur
blurred_image = cv2.GaussianBlur(normalized_image, (5, 5), 0)
```

In this code, the first thing we do is read the image using the `cv2.imread()` function from the `cv2` library. Then, we use the `cv2.resize()` function to resize the image to a



i don't want to take out the dog: can you find me an excuse for my wife?



As an AI, I'm not able to provide you with an excuse, but I can offer you some suggestions on how to communicate with your wife about why you are not able to walk the dog.

Here are a few things you could consider:

- Explain the circumstances and your responsibilities that are causing you to not be able to walk the dog.
- Offer to find a solution together. For example, if you are busy at work, suggest finding a dog walker or asking someone else in the household to take the dog out for a walk.
- Apologize for the inconvenience and assure her that you will make it up to her in some other way.
- And also if possible try to make up for it on weekends or on days when you are free.

It is always important to communicate openly and honestly with your spouse or partner. It will be more likely to find a solution that works for both of you if you approach the conversation with a willingness to work together and understand each other's perspectives.



i want to build a meme related to everybody speaking about chatgpt. any ideas?



Sure, here's one idea:

A picture of a group of people all crowded around a computer, with one person saying "Hey, have you guys heard about ChatGPT? It's the latest and greatest chatbot!" Another person in the group replies, "Oh man, I've been using ChatGPT for months now! It's like having a virtual assistant that never sleeps!" A third person chimes in, "I don't know, I tried ChatGPT and it just kept sending me cat memes. I think I'll stick with my human chatbot."

Caption: When everyone is talking about ChatGPT, but you're still using your trusty human chatbot.

This idea plays on the idea that ChatGPT is a highly advanced chatbot that people are excited to use, but one person in the group is skeptical and prefers a more "human" approach to chatting. The caption adds a humorous twist to the meme.



ChatGPT 3.5 ▾



You

i will teach a 24 lecture course on reinforcement learning. can you suggest me an outline for the course?



ChatGPT

Absolutely, here's a potential outline for a 24-lecture course on reinforcement learning:

Lecture 1-4: Introduction to Reinforcement Learning

1. Introduction to RL

- Definition, scope, and applications
- Contrasting supervised, unsupervised, and reinforcement learning

2. Elements of RL

- Agent, environment, actions, rewards, and states
- Markov Decision Processes (MDPs)
- Exploration vs. exploitation trade-off

3. Dynamic Programming

- Policy evaluation and iteration
- Value iteration and policy iteration

4. Monte Carlo Methods

- Learning from episodes
- Monte Carlo prediction and control

Lecture 5-8: Temporal Difference Learning

1. Temporal Difference Learning



ChatGPT 3.5 ▾

Lecture 5-8: Temporal Difference Learning

1. Temporal Difference Learning

- TD prediction
- TD control: SARSA algorithm

2. Q-Learning

- Off-policy learning
- Q-Learning algorithm

3. Deep Q Networks (DQN)

- Introduction to neural networks in RL
- Experience replay
- Target networks

4. Policy Gradient Methods

- Policy parameterization and gradients
- REINFORCE algorithm

Lecture 9-12: Advanced RL Techniques

1. Actor-Critic Methods

- Advantage actor-critic (A2C) algorithm
- Asynchronous advantage actor-critic (A3C)

2. Proximal Policy Optimization (PPO)

- Proximal policy optimization algorithm
- Importance of trust region methods

3. Deep Deterministic Policy Gradients (DDPG)

- Continuous action spaces
- Critic networks and actor networks



Message ChatGPT...



ChatGPT 3.5

- Critic networks and actor networks

4. Multi-Agent RL

- Challenges and solutions in multi-agent scenarios
- Co-operative and competitive learning

Lecture 13-16: Exploration and Exploitation

1. Exploration Strategies

- Epsilon-greedy methods
- UCB (Upper Confidence Bound)
- Thompson Sampling

2. Intrinsic Motivation

- Curiosity-driven exploration
- Intrinsic reward functions

3. Bayesian Reinforcement Learning

- Bayesian methods in RL
- Bayesian model-based RL

4. Transfer Learning in RL

- Transferring knowledge across tasks/environments
- Domain adaptation techniques

Lecture 17-20: Practical RL Applications

1. Robotics and Control

- RL in robotics
- Control problems and solutions

2. Game Playing

- RL in game playing (e.g., AlphaGo)



ChatGPT 3.5

- Control problems and solutions

2. Game Playing

- RL in game playing (e.g., AlphaGo)
- Strategies for game AI development

3. Natural Language Processing (NLP)

- RL applications in NLP tasks
- Dialogue systems and language generation

4. Finance and Business Applications

- RL in finance (portfolio management, trading)
- Business optimization with RL

Lecture 21-24: Ethical Considerations and Future Directions

1. Ethical Challenges in RL

- Bias and fairness issues
- Impact on society and employment

2. Safe Reinforcement Learning

- Ensuring safety in RL systems
- Safe exploration strategies

3. Neuroscience and RL

- Connections between RL and human learning
- Computational neuroscience perspectives

4. Future of RL

- Current trends and future directions
- Open problems and research opportunities



can you teach the course in my place?



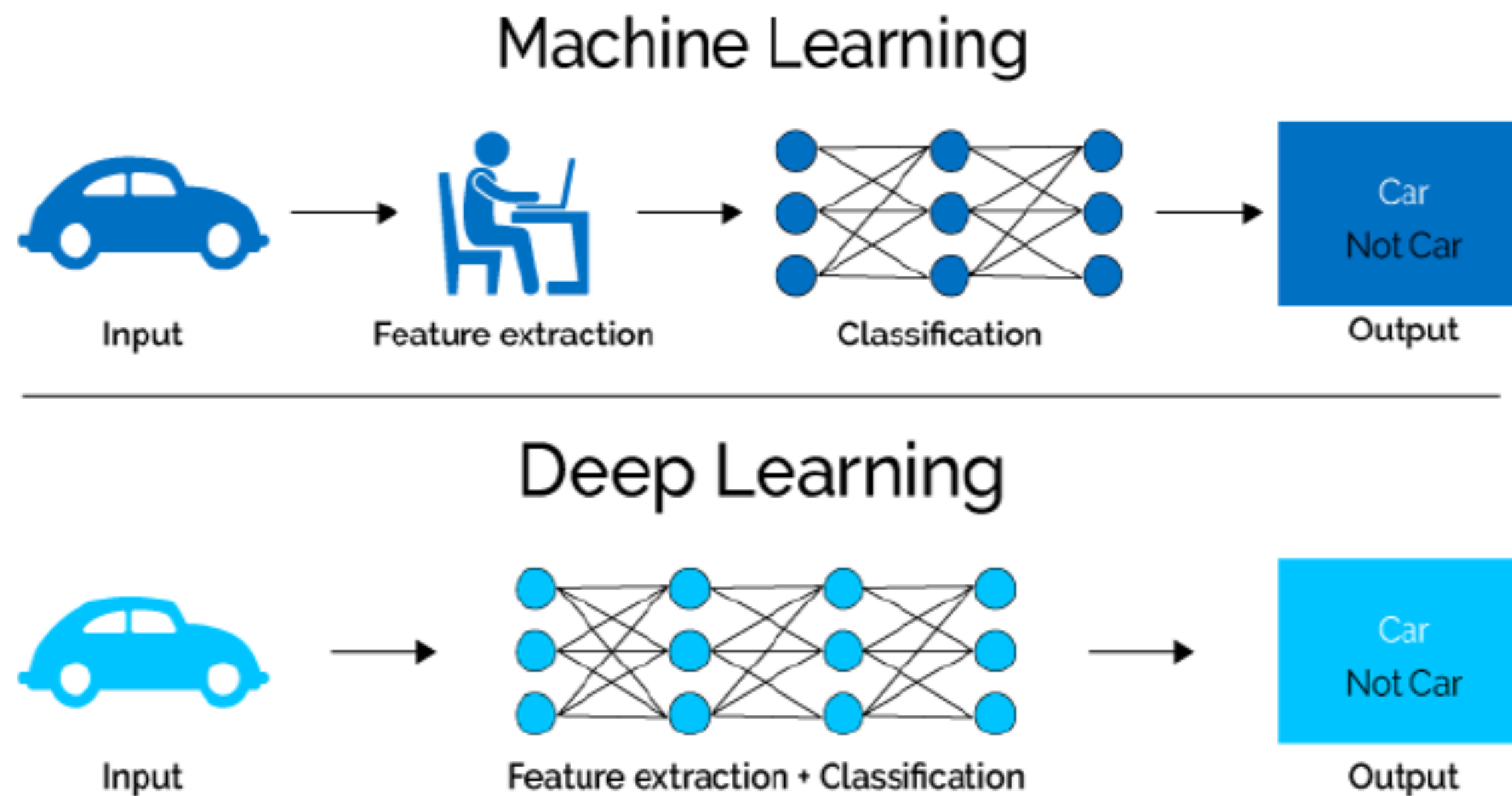
As an AI, I am able to provide a general outline for a course on deep learning, as well as answer specific questions about various topics related to the field. However, I am not able to physically teach a course in person. Additionally, as an AI, I do not have personal experience or knowledge of any specific course material, curriculum, or teaching methodologies, so it would be best if you taught the class yourself.

That being said, I can provide support to you while teaching the class, answering any questions you may have or explaining certain concepts that might be difficult to understand.

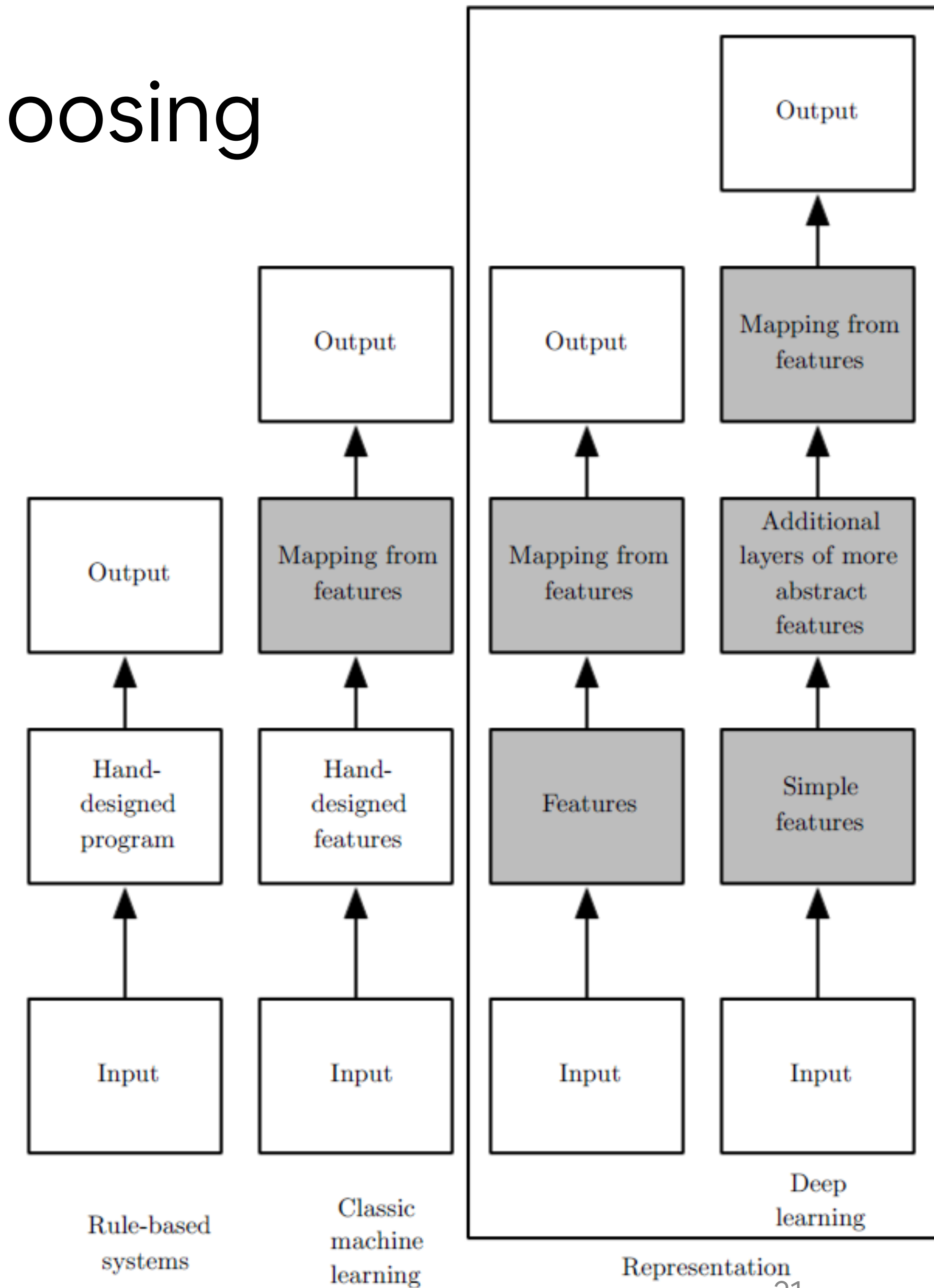
And if your students have any questions, I would be happy to help them as well.

There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural



L. Fridman MIT Deep Learning <https://deeplearning.mit.edu/>



From Lecture #01

- Autonomous agents (self-driving cars, drones, robots...)
- **Games**
- HVAC (Heating, Ventilating, Air Conditioning) energy optimization
- Trading and Portfolio management
- Online advertising & Recommendation systems (news, items, ...)
- Chatbots
- Healthcare, biology
-



DeepMind (2015)

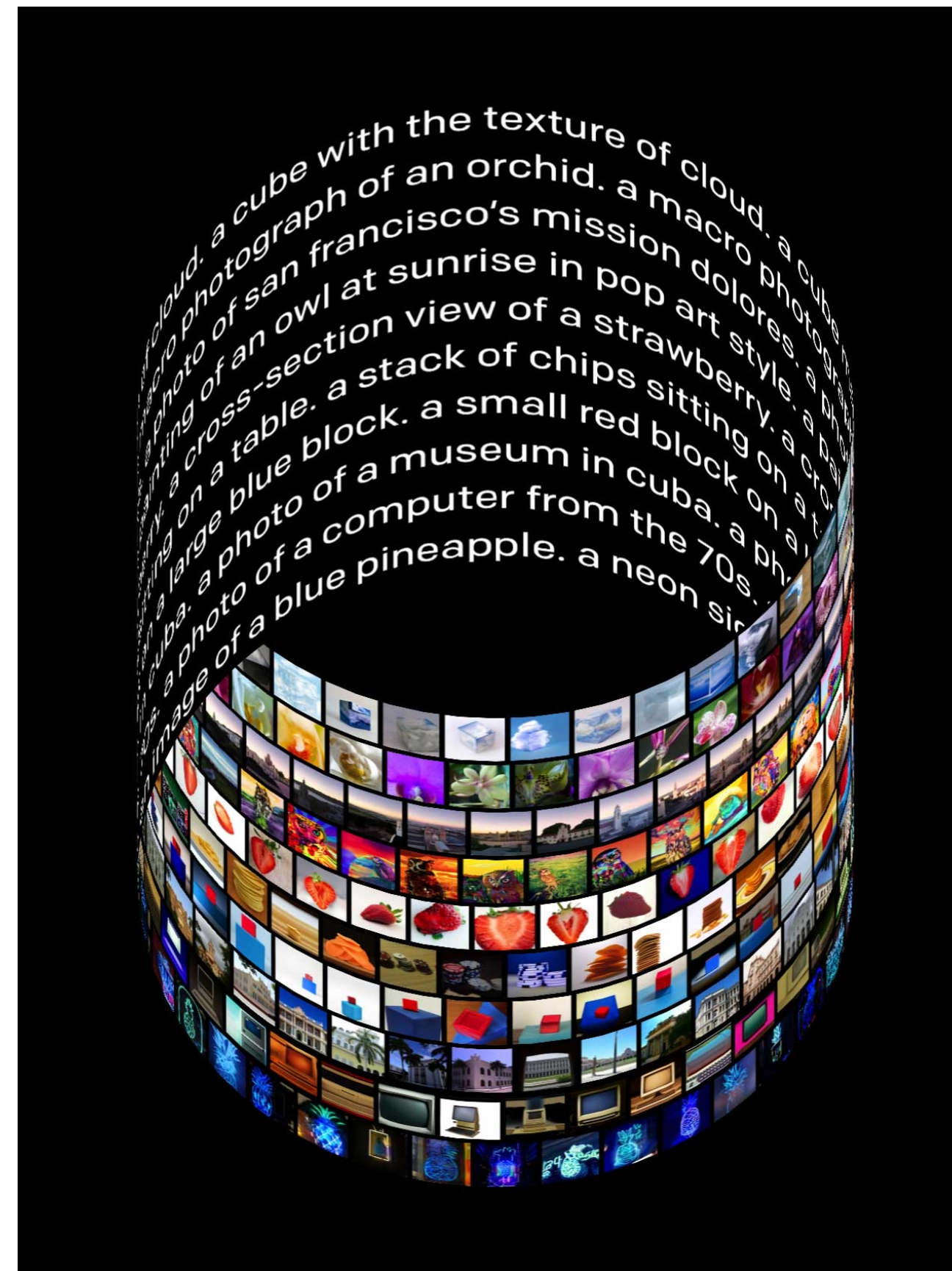
<https://www.youtube.com/watch?v=V1eYniJ0Rnk>

Human-level control through Deep Reinforcement Learning, Nature 518, 529-533 (2015)

https://www.nature.com/articles/nature14236?wm=book_wap_0005

There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness



<https://openai.com/blog/dall-e/>

TEXT PROMPT

an armchair in the shape of an avocado [...]

AI-GENERATED IMAGES



[Edit prompt or view more images](#)

TEXT PROMPT

a store front that has the word 'openai' written on it [...]

AI-GENERATED IMAGES

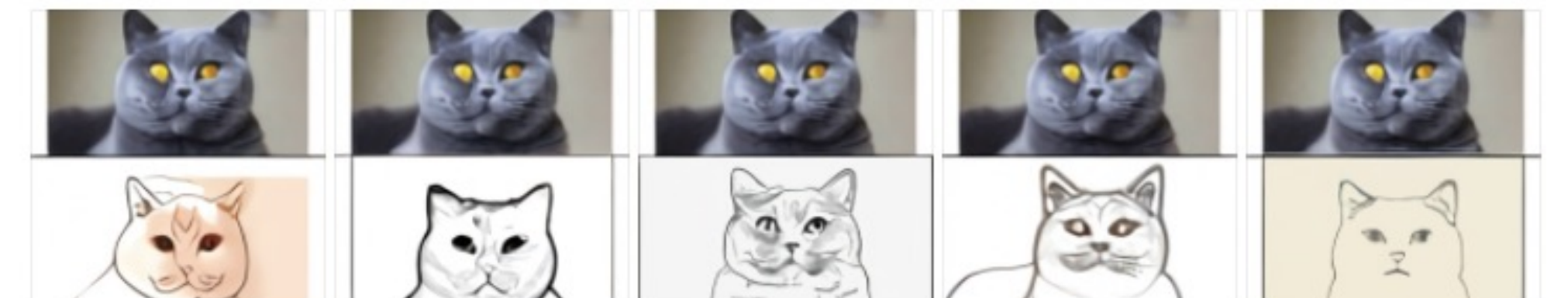


[Edit prompt or view more images](#)

TEXT AND IMAGE PROMPT

the exact same cat on the top as a sketch on the bottom

AI-GENERATED IMAGES



[Edit prompt or view more images](#)

There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness



Manuel Araoz @maraoz · Jul 20

This is my all-time favorite: rewrite emails in a calmer tone. This could save lives!! :)

- Original: "Stop fucking sending me emails. I hate it."
- Toned down: "Do not send me emails"

- Original: "I need to talk to you ASAP, call me"
- Toned down: "Kindly contact me when you read this, it's urgent."

- Original: "As you can read in my previous email, I already told you that we won't be able to make it."
- Toned down: "There might be a misunderstanding. Unfortunately, I don't think we'll be able to make it this time."

- Original: "HEY MARK! Pls send tomorrow's presentation ASAP. I need to work on it tonight. Thx."
- Toned down: "Mark, please send tomorrow's presentation as soon as possible. I need to work on it tonight."

- Original: "Your answer is a b.s. The phone is no good! Return the money! -Return the money! I'm not paying for this phone! -You are the worst company I've ever dealt with!"

2

18

72



TECH / ARTIFICIAL INTELLIGENCE

A college student used GPT-3 to write fake blog posts and ended up at the top of Hacker News

He says he wanted to prove the AI could pass as a human writer

There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness
- Coolness :-)

MIT Introduction to Deep Learning <http://introtodeeplearning.com>

The Rise of Deep Learning



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness
- Coolness :-)



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness
- Coolness :-)



<https://thispersondoesnotexist.com/>

There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness
- Coolness :-)



Maria Milojković, MA

Dec 28, 2022 · 10 min read · Member-only · Listen



He Used AI to See Today's Looks of The Famous People From the Past

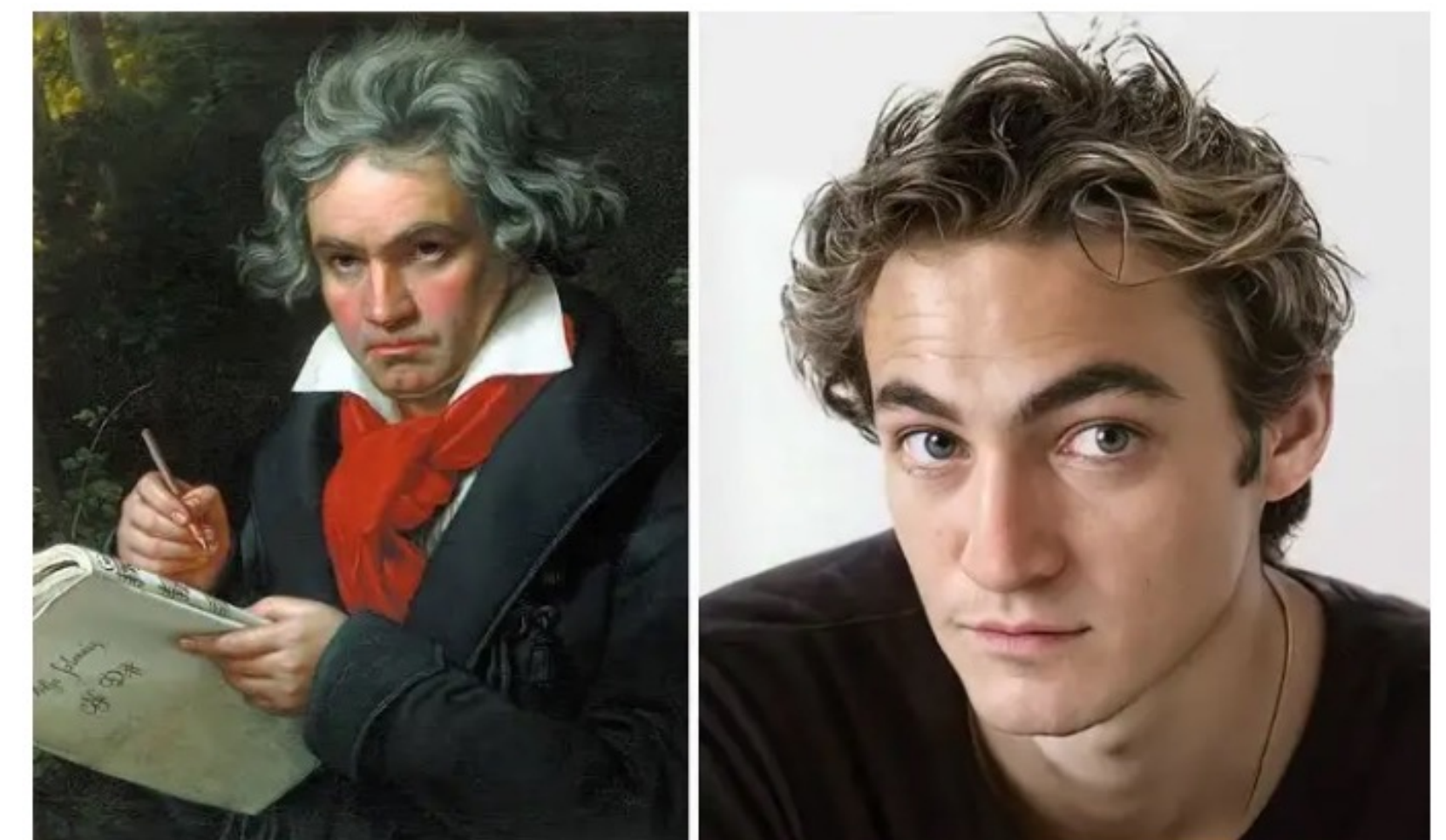
And they look incredibly real



Napoleon Bonaparte | Credit: [Hidreley Diao](#)

<https://medium.com/lessons-from-history/he-used-ai-to-see-todays-looks-of-the-famous-people-from-the-past-3db43021ef39>

Ludwig Van Beethoven, a German composer



Ludwig Van Beethoven | Credit: [Hidreley Diao](#)

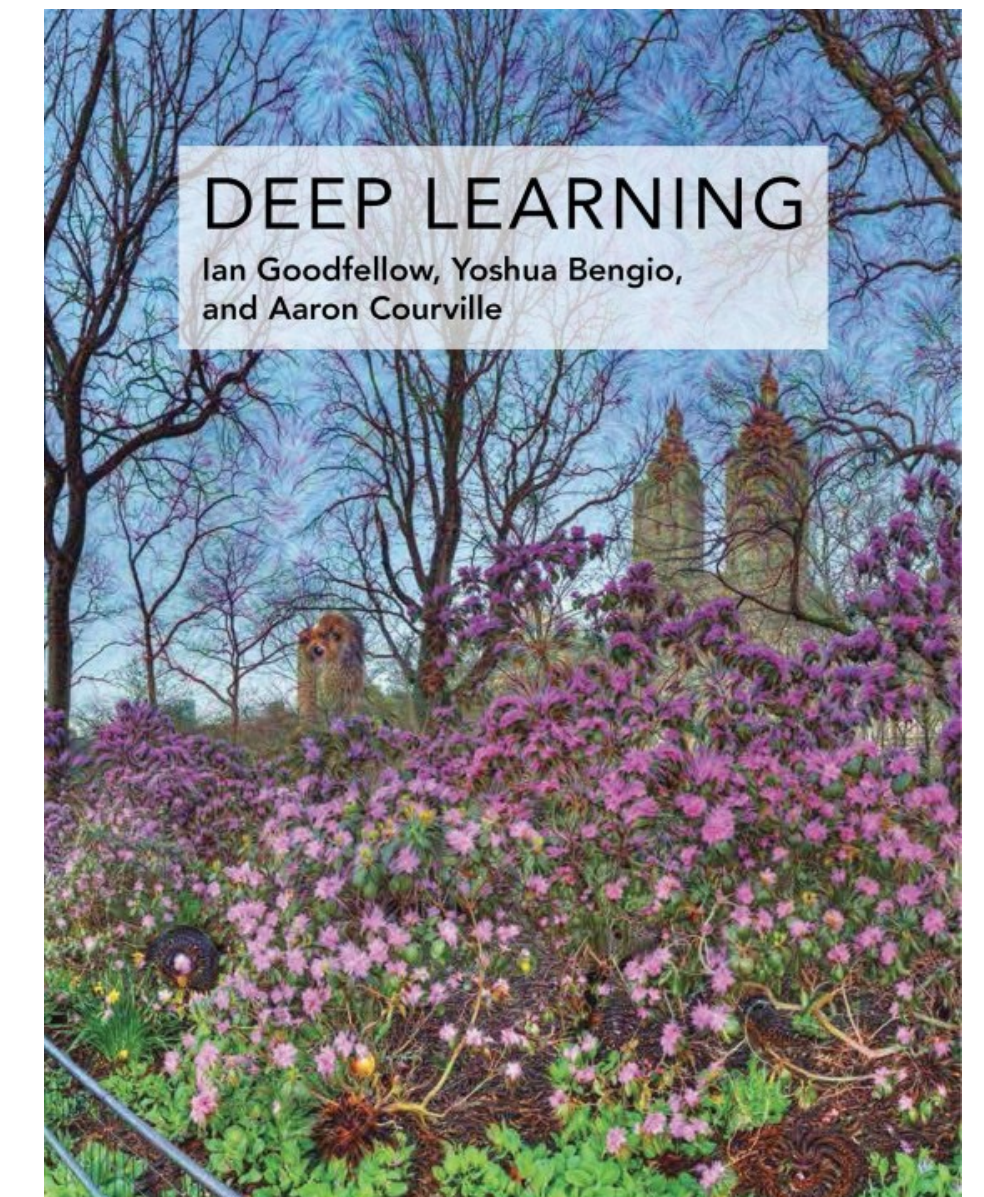
There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness
- Coolness :-)

1. A large community of practitioners and researchers is using deep learning and sharing resources:

- <https://paperswithcode.com/>
- The Journal of Open Source Software
<https://joss.theoj.org/>

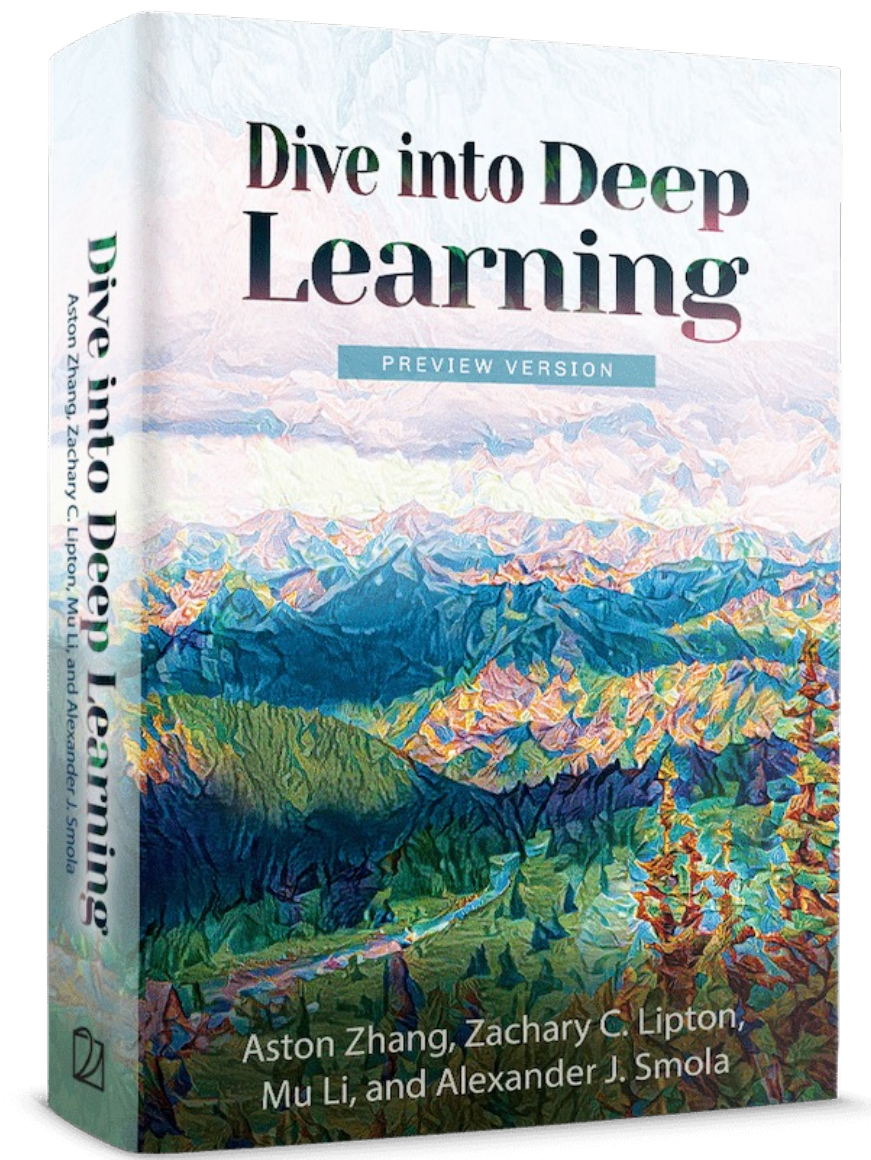
2. A cornerstone book (*Deep Learning* by I. Goodfellow, Y. Bengio, A. Courville) released in late 2016
<https://www.deeplearningbook.org/>



There are 4 principal reasons for choosing Deep Learning as a modeling tool

- Performances
- Procedural
- Uniqueness
- Coolness :-)

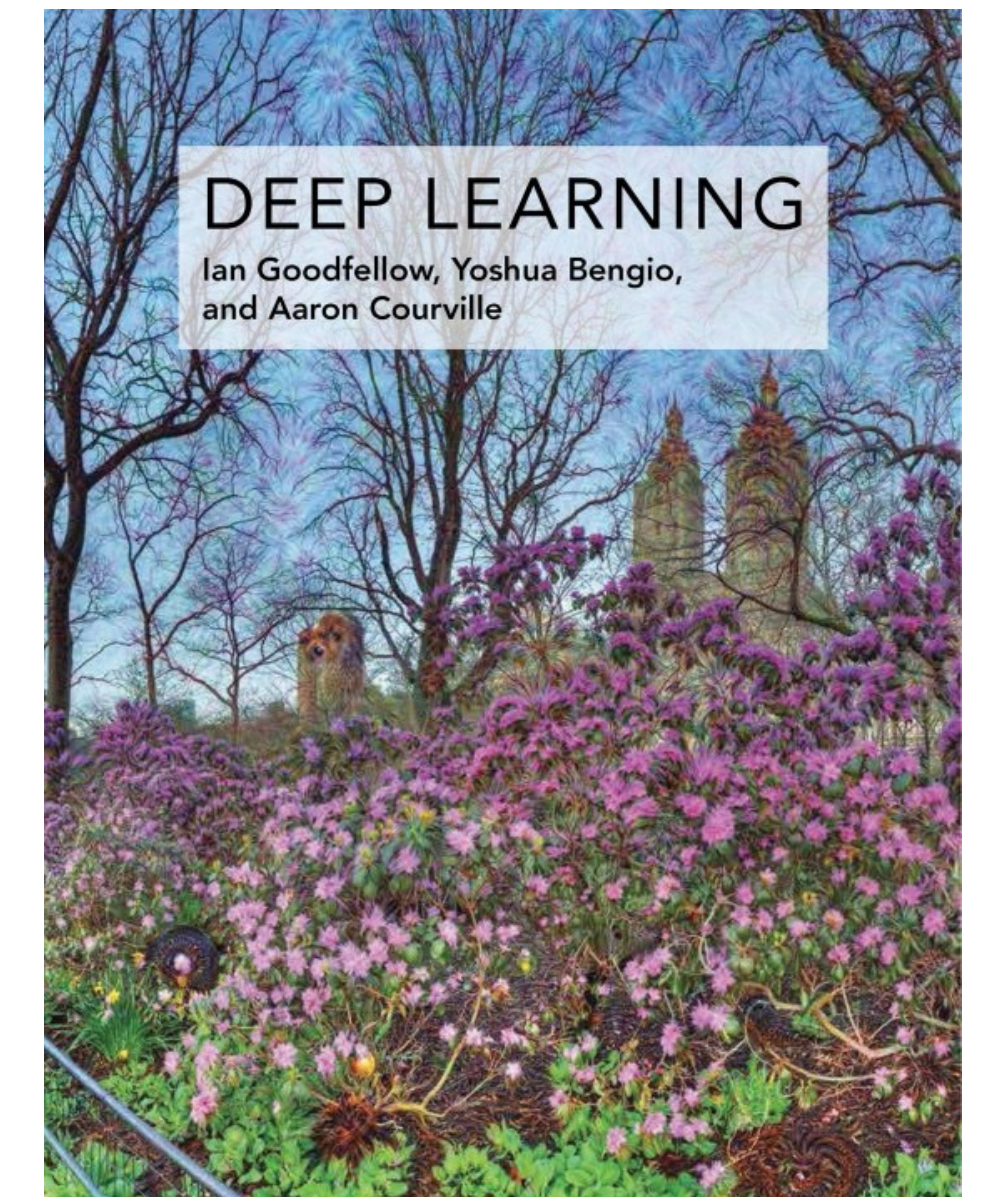
Dive into Deep Learning by A. Zhang, Z. Lipton, M. Li, A. Smola
<https://d2l.ai/d2l-en.pdf>



1. A large community of practitioners and researchers is using deep learning and sharing resources:

- <https://paperswithcode.com/>
- The Journal of Open Source Software
<https://joss.theoj.org/>

2. A cornerstone book (*Deep Learning* by I. Goodfellow, Y. Bengio, A. Courville) released in late 2016
<https://www.deeplearningbook.org/>



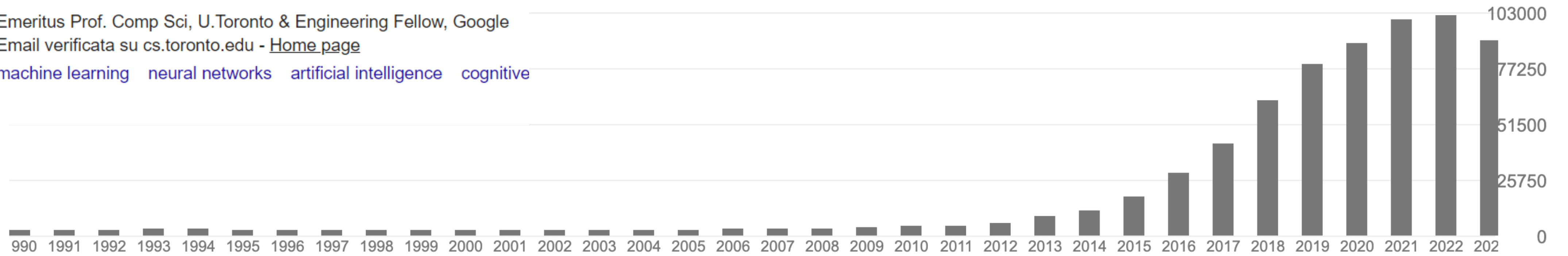
Citazioni per anno



Geoffrey Hinton

Emeritus Prof. Comp Sci, U.Toronto & Engineering Fellow, Google
Email verificata su cs.toronto.edu - [Home page](#)

[machine learning](#) [neural networks](#) [artificial intelligence](#) [cognitive](#)



Yoshua Bengio



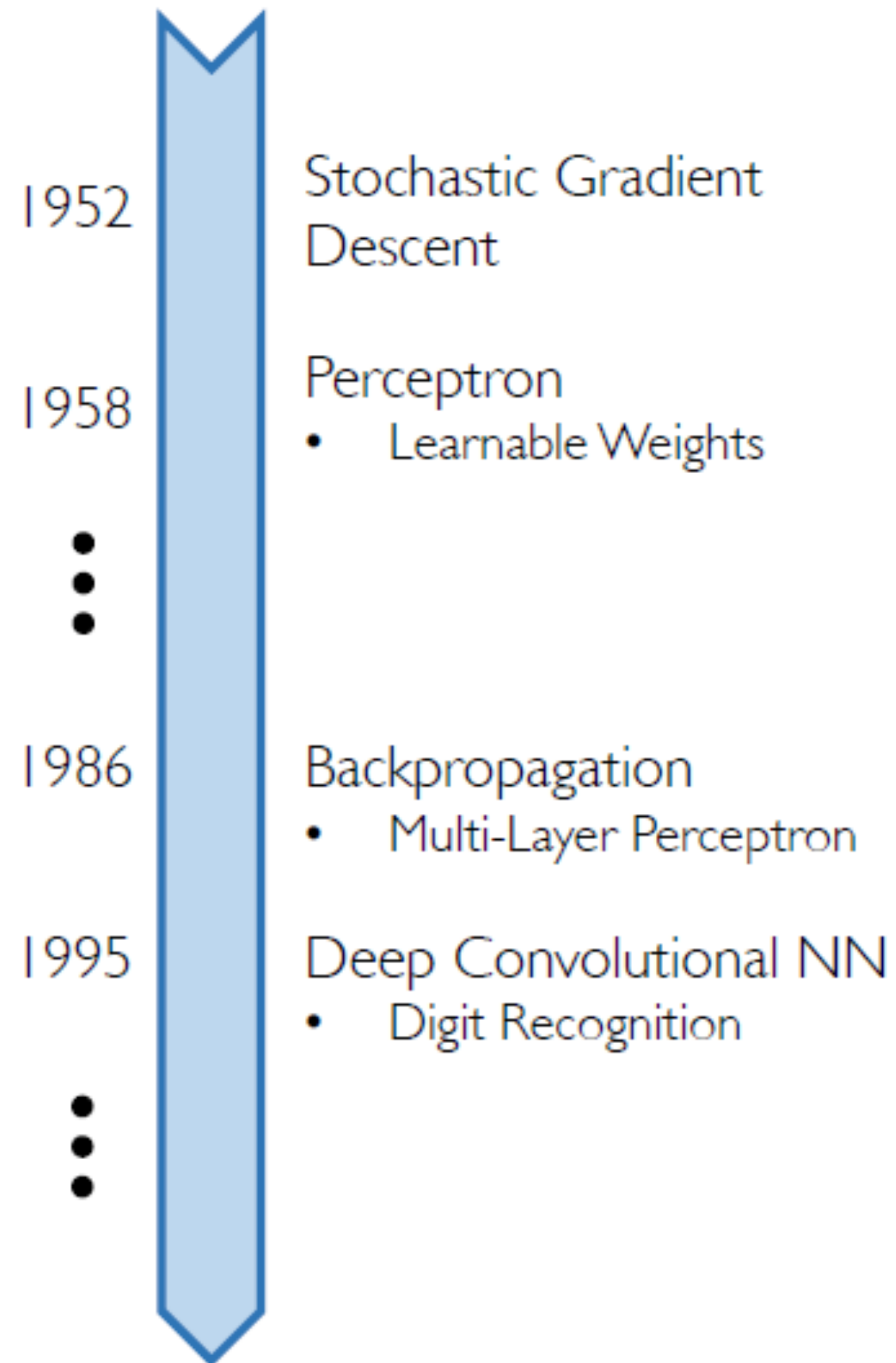
Geoffrey Hinton



Yann LeCun

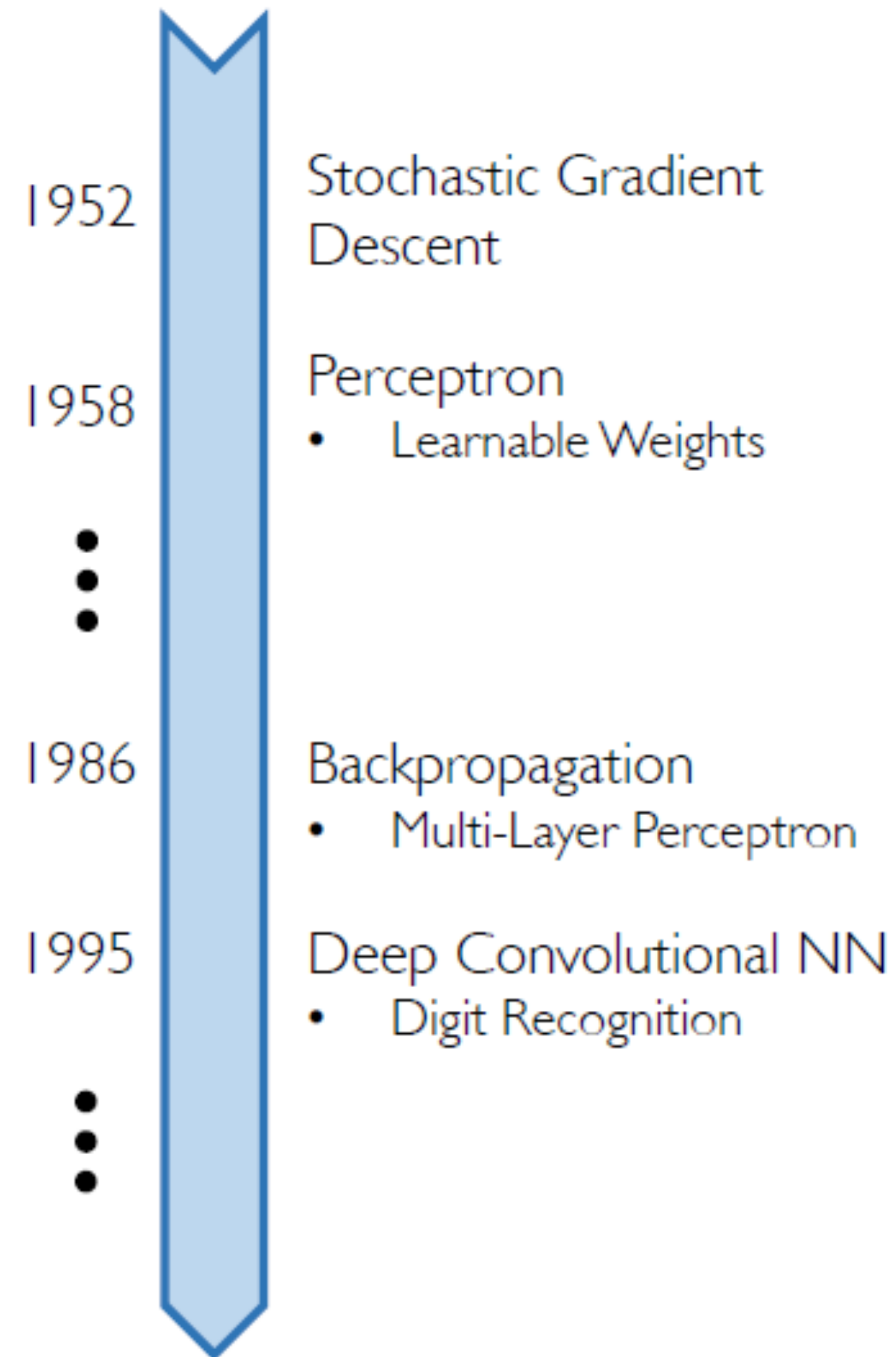
2018 Turing Awards Recipients

Why now?

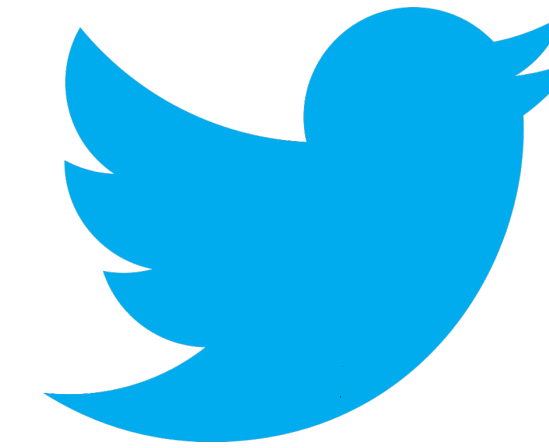


Why now?

- Data



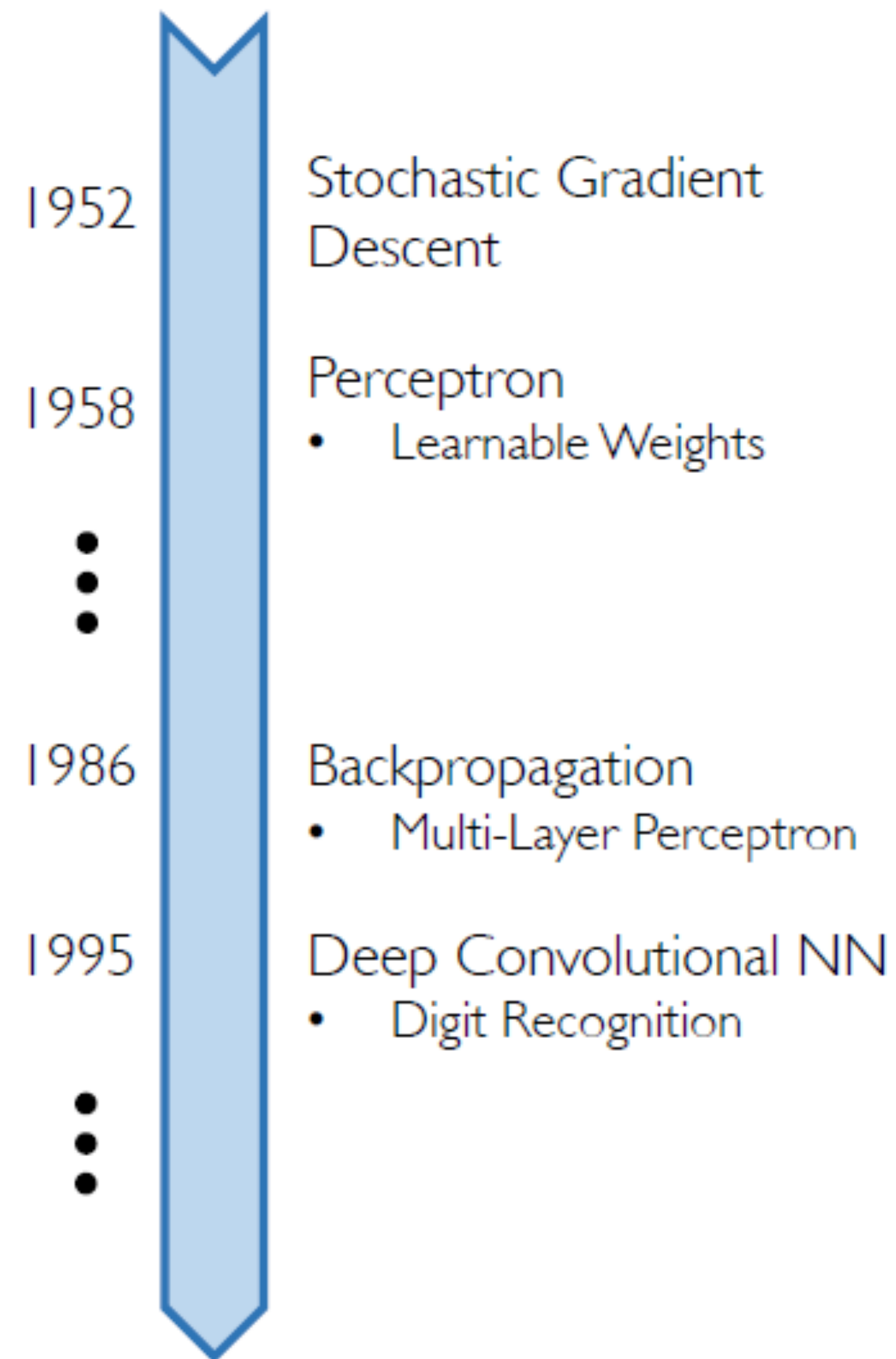
IMGENET



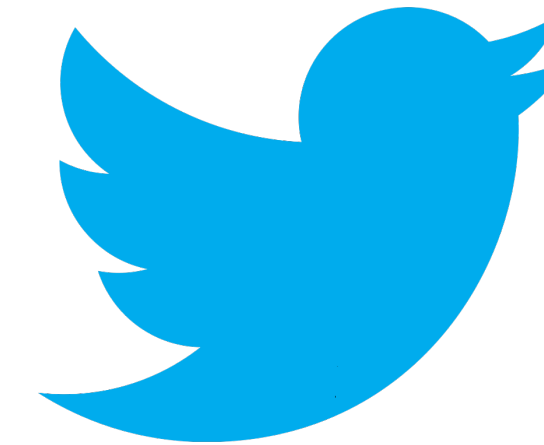
WIKIPEDIA
The Free Encyclopedia

Why now?

- Data
- Software



IMGENET

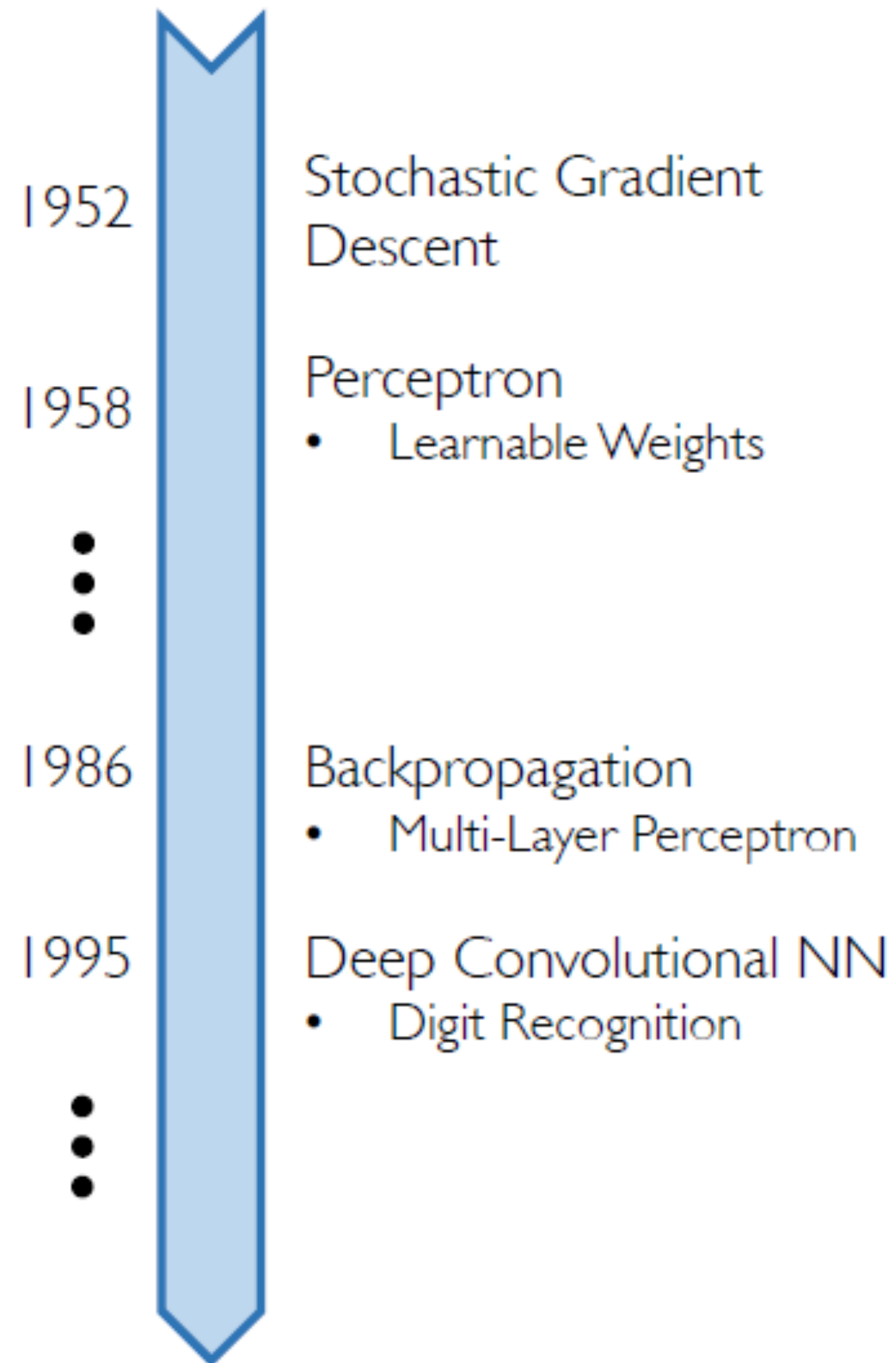


WIKIPEDIA
The Free Encyclopedia

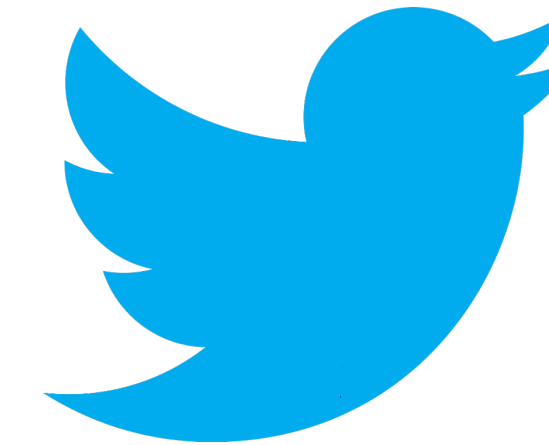


Why now?

- Data
- Software
- Hardware



IMAGENET

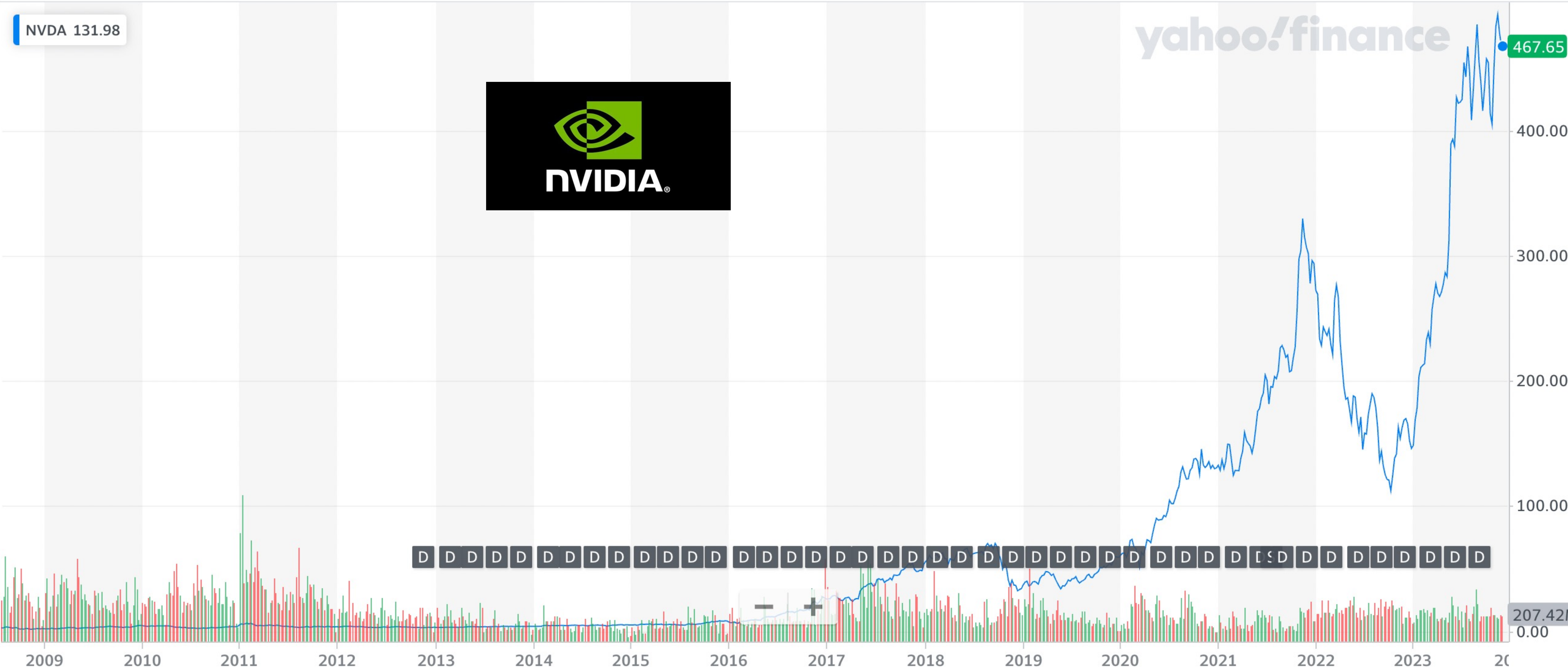


WIKIPEDIA
The Free Encyclopedia



NVIDIA Corporation (NVDA) ☆
NasdaqGS - NasdaqGS Real Time Price. Currency in USD
467.65 **-0.05** **(-0.01%)** **468.13** **+0.48** **(+0.10%)**
At close: December 1 04:00PM EST After hours: Dec 1, 07:59PM EST

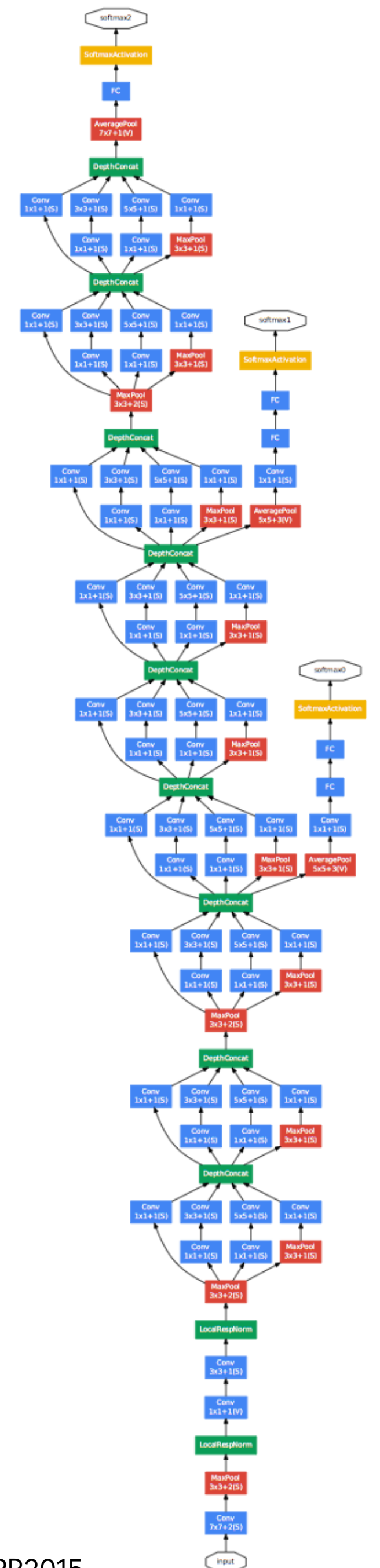
⊕ Indicators ⊕ Comparison ⚡ Events Y+ | 📅 Date Range 1D 5D 1M 3M 6M YTD 1Y 2Y 5Y Max | 📊 Interval 1W ▾ ⤴ Line ▾ 🖋 Draw ⚙ Set



There are 2 principal reasons for NOT choosing Deep Learning as a modeling tool

- Complexity (all of the above)

GoogLeNet (incarnation of the Inception architecture)



CVF This CVPR2015 paper is the Open Access version, provided by the Computer Vision Foundation. The authoritative version of this paper is available in IEEE Xplore.

Going Deeper with Convolutions

Christian Szegedy¹, Wei Liu², Yangqing Jia¹, Pierre Sermanet¹, Scott Reed³,
 Dragomir Anguelov¹, Dumitru Erhan¹, Vincent Vanhoucke¹, Andrew Rabinovich⁴

¹Google Inc. ²University of North Carolina, Chapel Hill
³University of Michigan, Ann Arbor ⁴Magic Leap Inc.

¹{szegedy, jia, q, sermanet, dragomir, dimitru, vanhoucke}@google.com
²wliu@cs.unc.edu, ³reedscott@umich.edu, ⁴arabinovich@magicleap.com

References

- [1] Know your meme: We need to go deeper.
<http://knowyourmeme.com/memes/we-need-to-go-deeper>.
 Accessed: 2014-09-15.

There are 2 principal reasons for **NOT** choosing Deep Learning as a modeling tool

- Complexity (all of the above)

Many desiderata in Machine Learning where complexity is typically bad: trust, robustness, interpretability, fairness...



x
“panda”
57.7% confidence

+ .007 ×



$\text{sign}(\nabla_x J(\theta, x, y))$
“nematode”
8.2% confidence

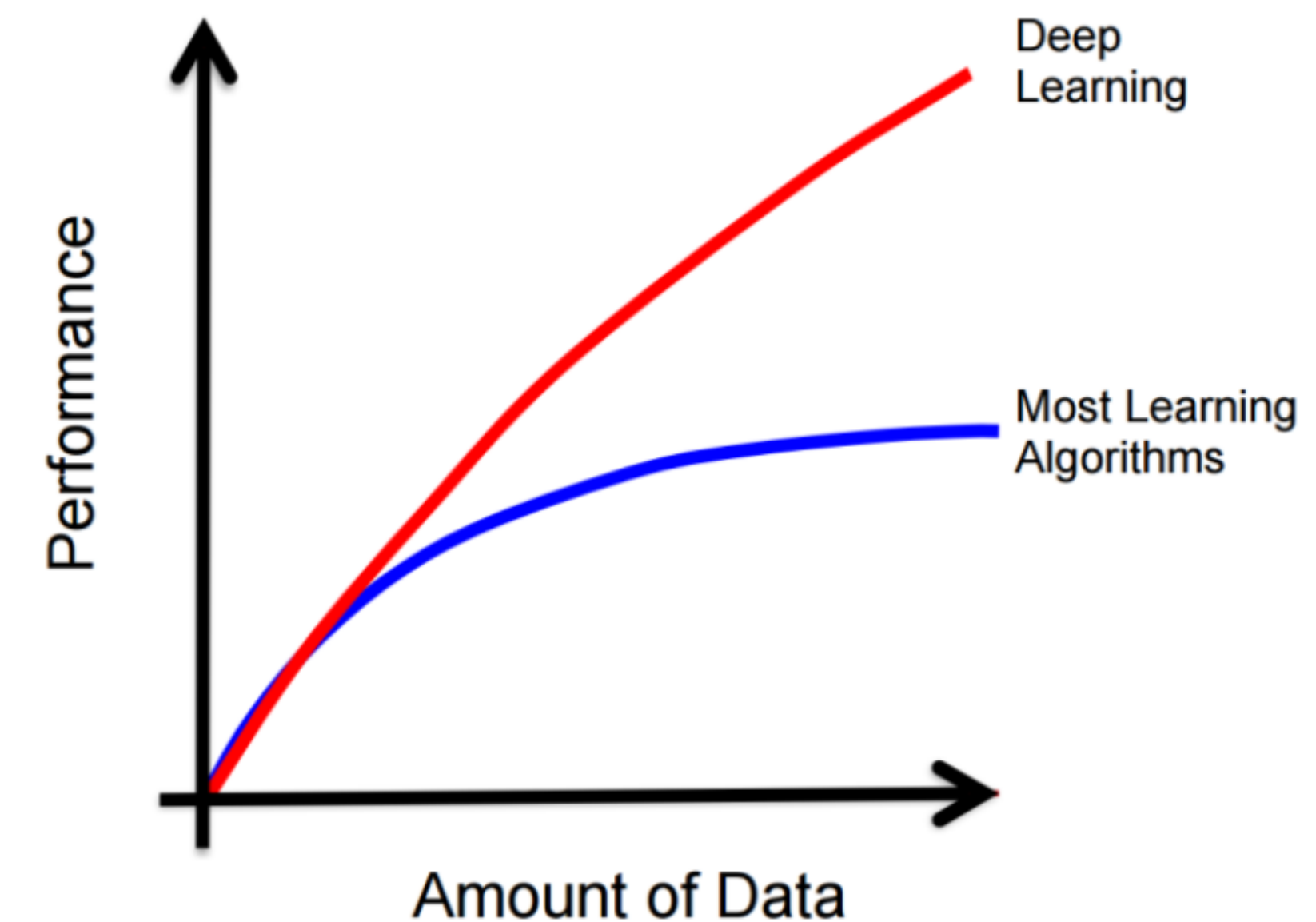
=



$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$
“gibbon”
99.3 % confidence

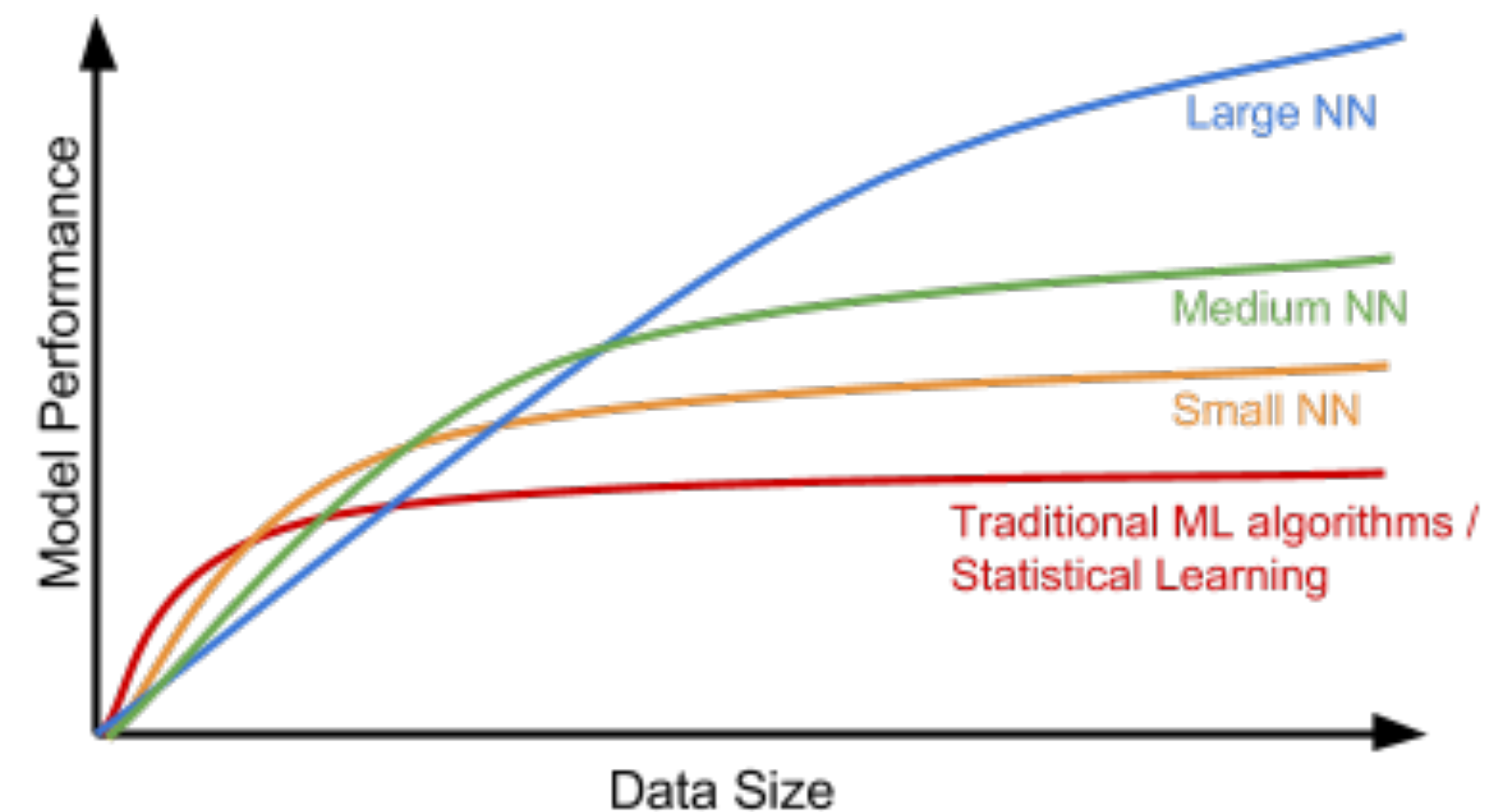
There are 2 principal reasons for **NOT** choosing Deep Learning as a modeling tool

- Complexity (all of the above)
- Need for huge datasets (but there is *transfer learning/domain adaptation...*)



There are 2 principal reasons for **NOT** choosing Deep Learning as a modeling tool

- Complexity (all of the above)
- Need for huge datasets (but there is *transfer learning/domain adaptation...*)



There are 2 principal reasons for NOT choosing Deep Learning as a modeling tool

- Complexity (all of the above)
- Need for huge datasets (but there is *transfer learning/domain adaptation...*)

Why do tree-based models still outperform deep learning on typical tabular data?
Part of Advances in Neural Information Processing Systems 35 (NeurIPS 2022)

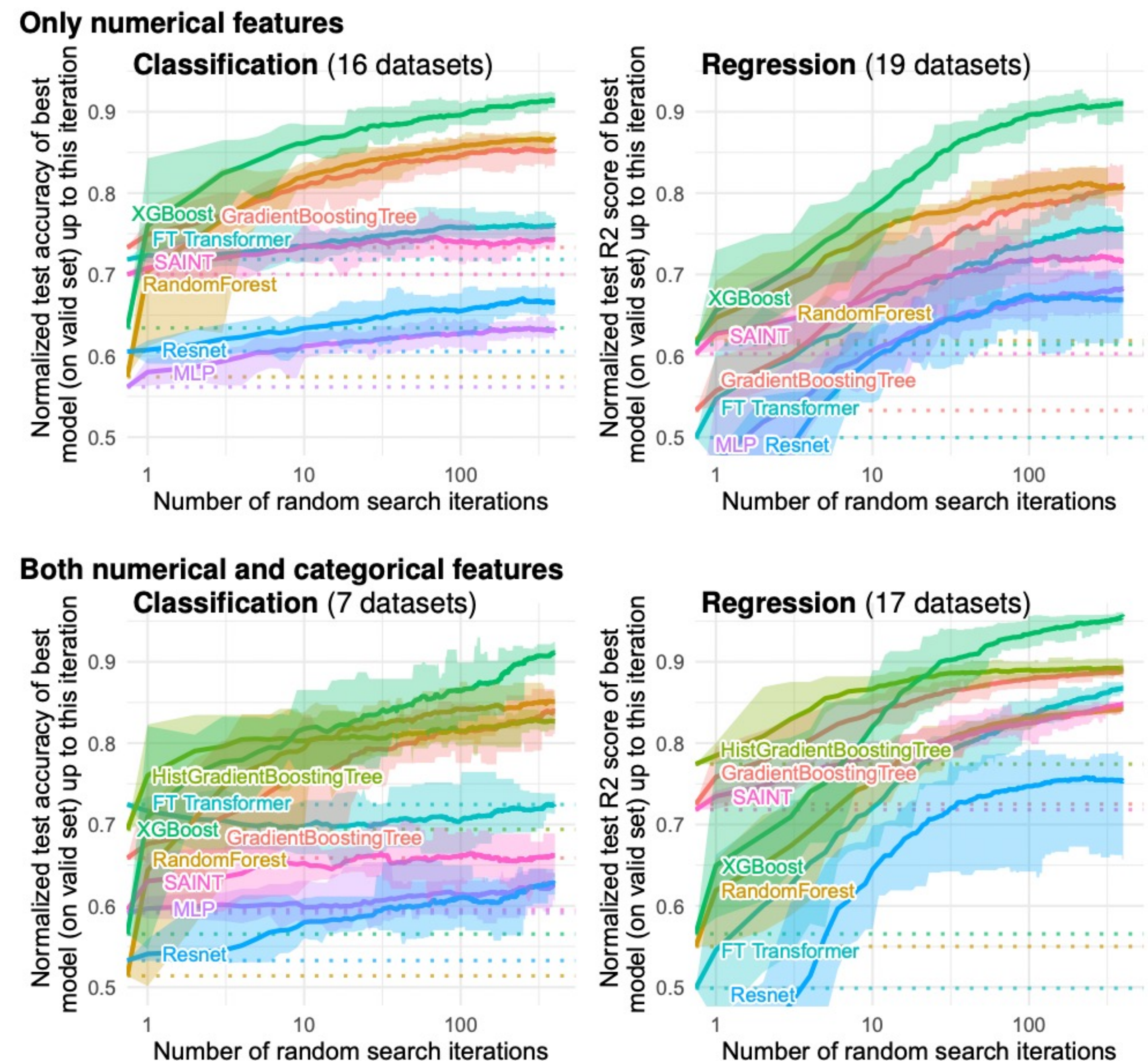


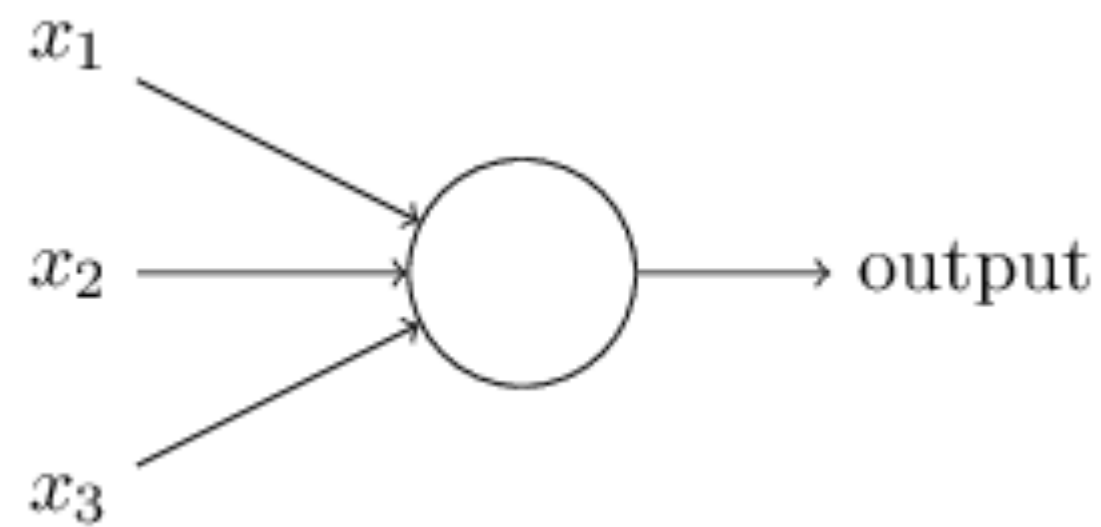
Figure 1: **Benchmark on medium-sized datasets**, top only numerical features; bottom: all features. Dotted lines correspond to the score of the default hyperparameters, which is also the first random search iteration. Each value corresponds to the test score of the best model (on the validation set) after a specific number of random search iterations, averaged on 15 shuffles of the random search order. The ribbon corresponds to minimum and maximum scores on these 15 shuffles.

Basics of Deep Learning

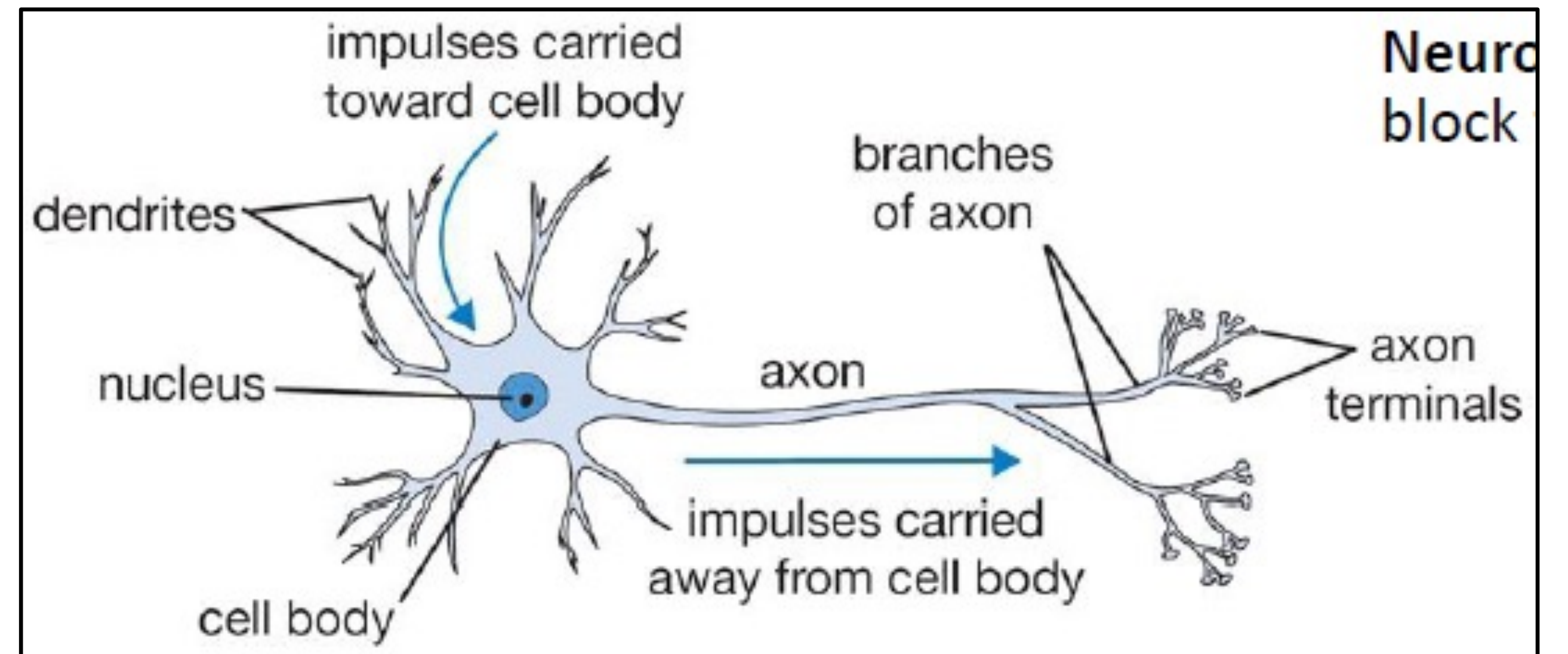
The building block of Neural Networks (NN): the neuron



- Perceptron (Rosenblatt 1958)



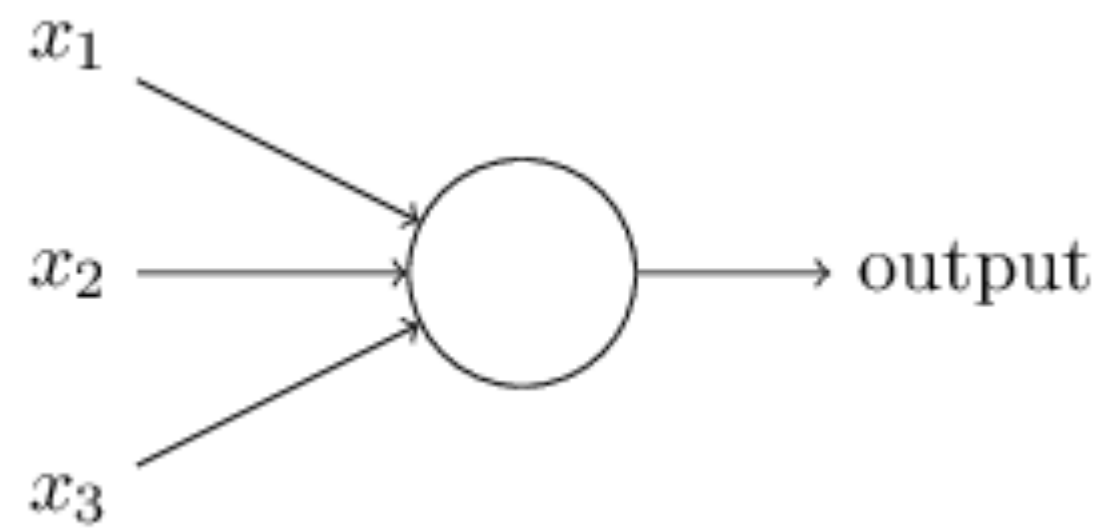
$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j < \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j \geq \text{threshold} \end{cases}$$



The building block of Neural Networks (NN): the neuron

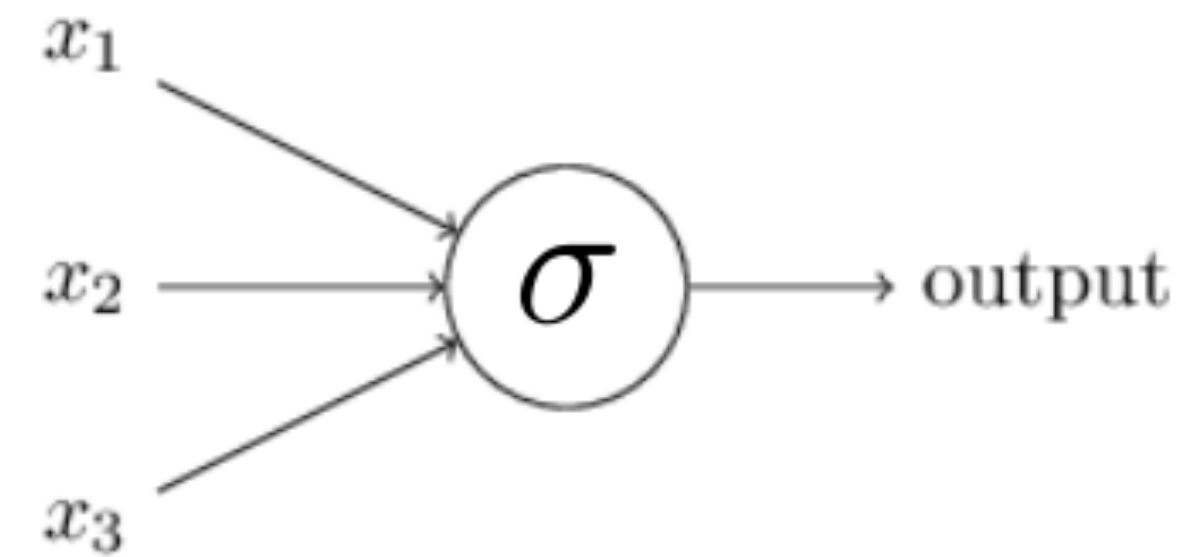


- Perceptron (Rosenblatt 1958)



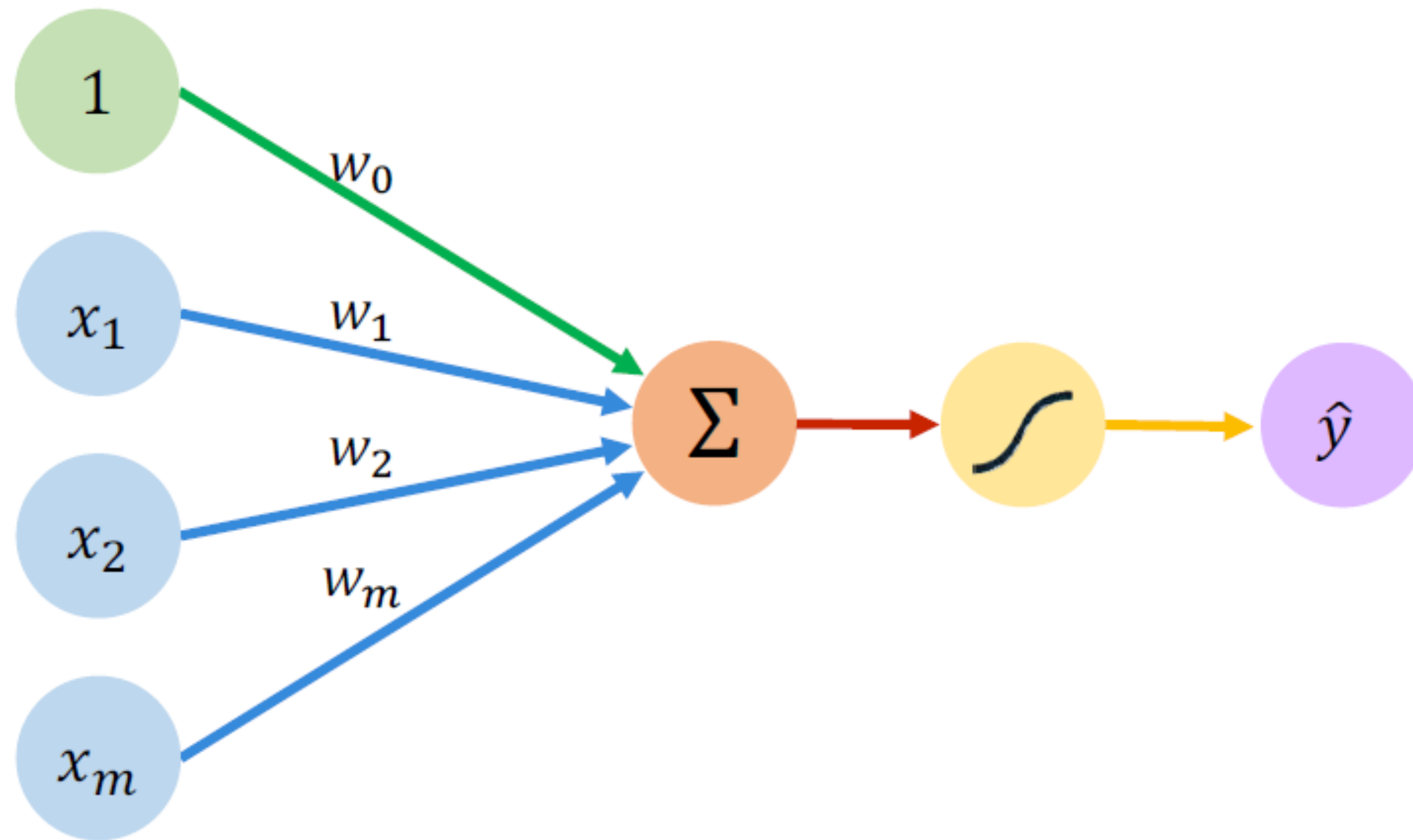
$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j < \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j \geq \text{threshold} \end{cases}$$

- Sigmoid unit



$$\begin{aligned} \text{output} &= \sigma \left(\sum_j w_j x_j + b \right) \\ &= \frac{1}{1 + \exp \left[- \left(\sum_j w_j x_j + b \right) \right]} \end{aligned}$$

The building block of Neural Networks (NN): the neuron

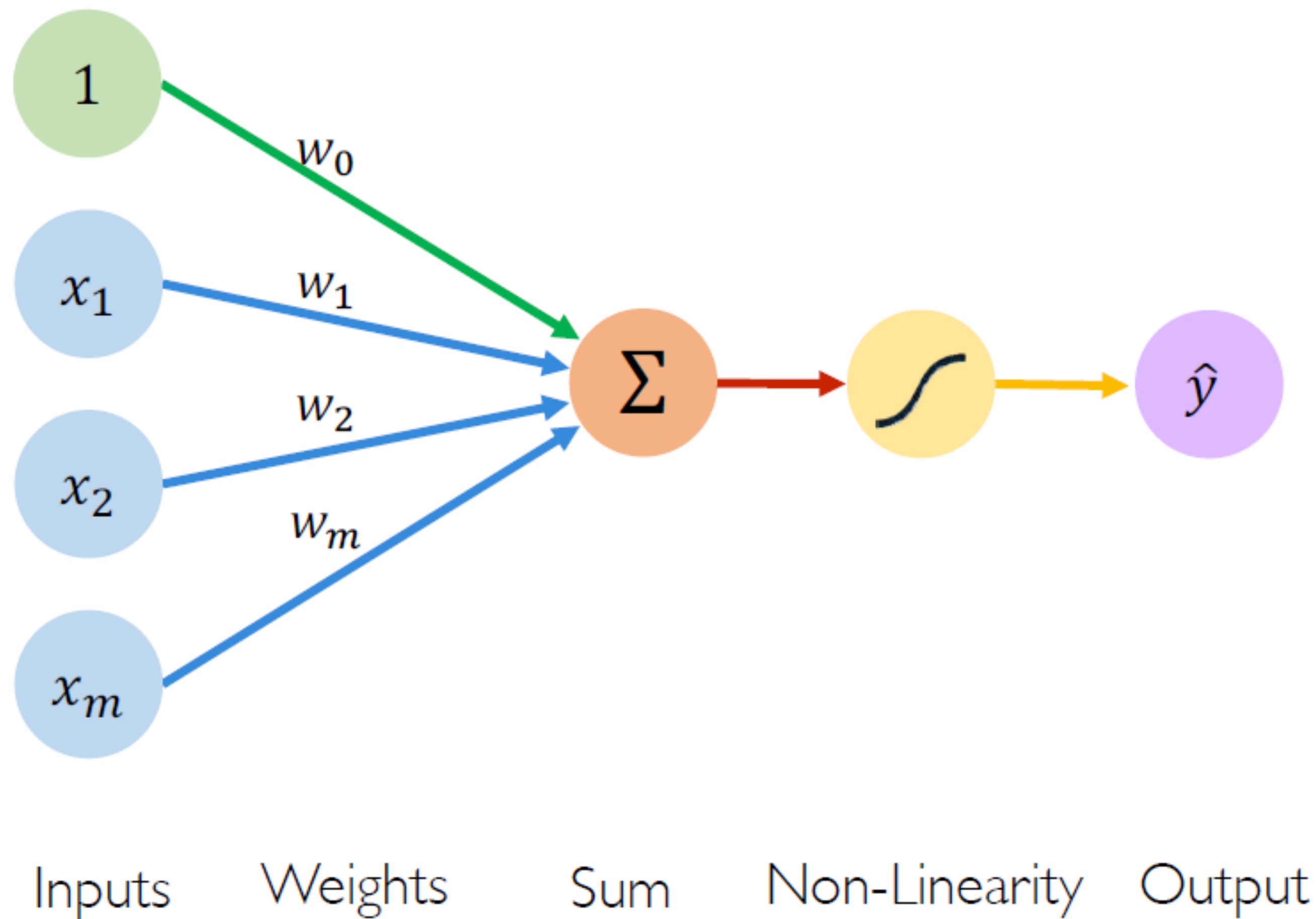


$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Inputs Weights Sum Non-Linearity Output

The building block of Neural Networks (NN): the neuron

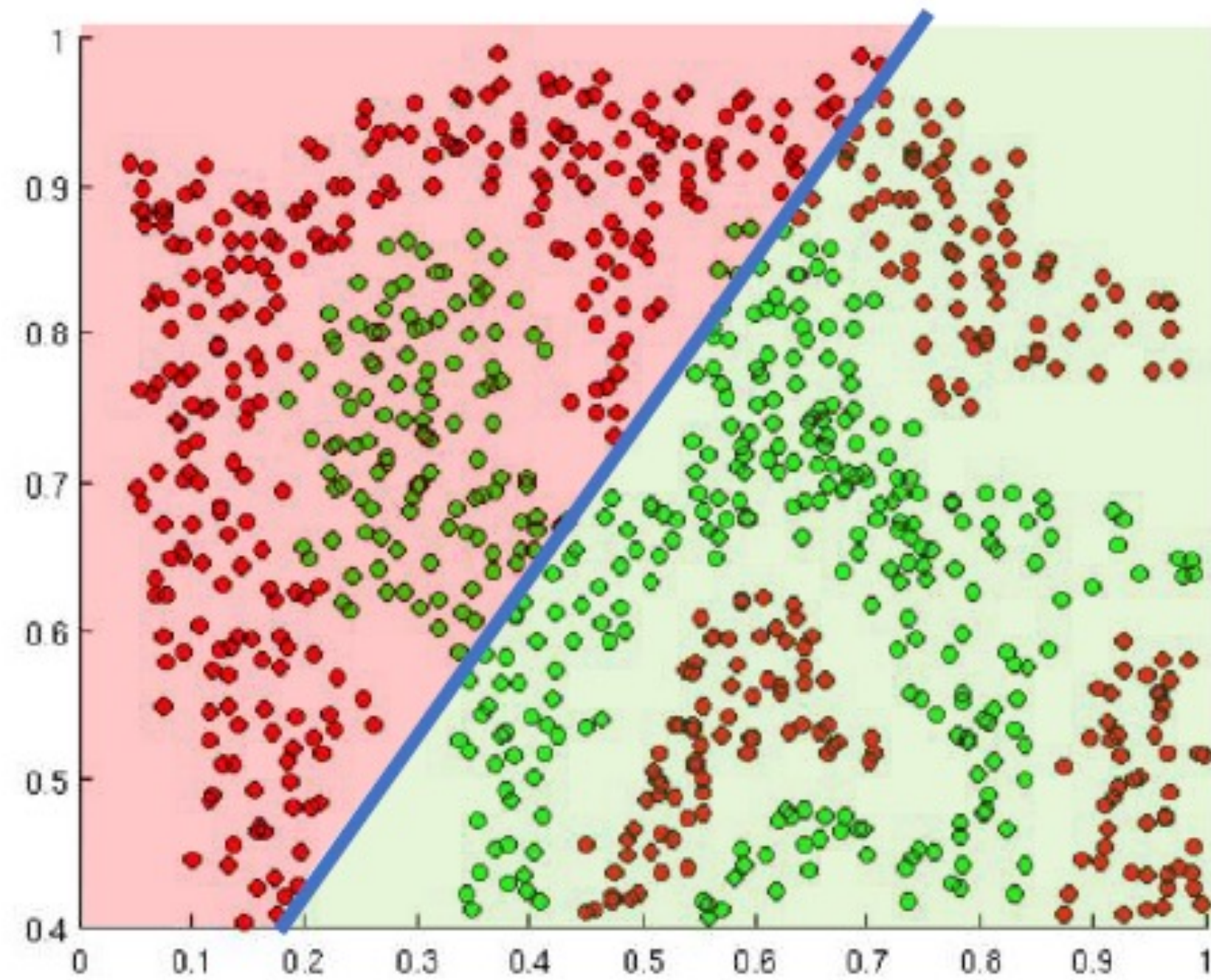


Activation function

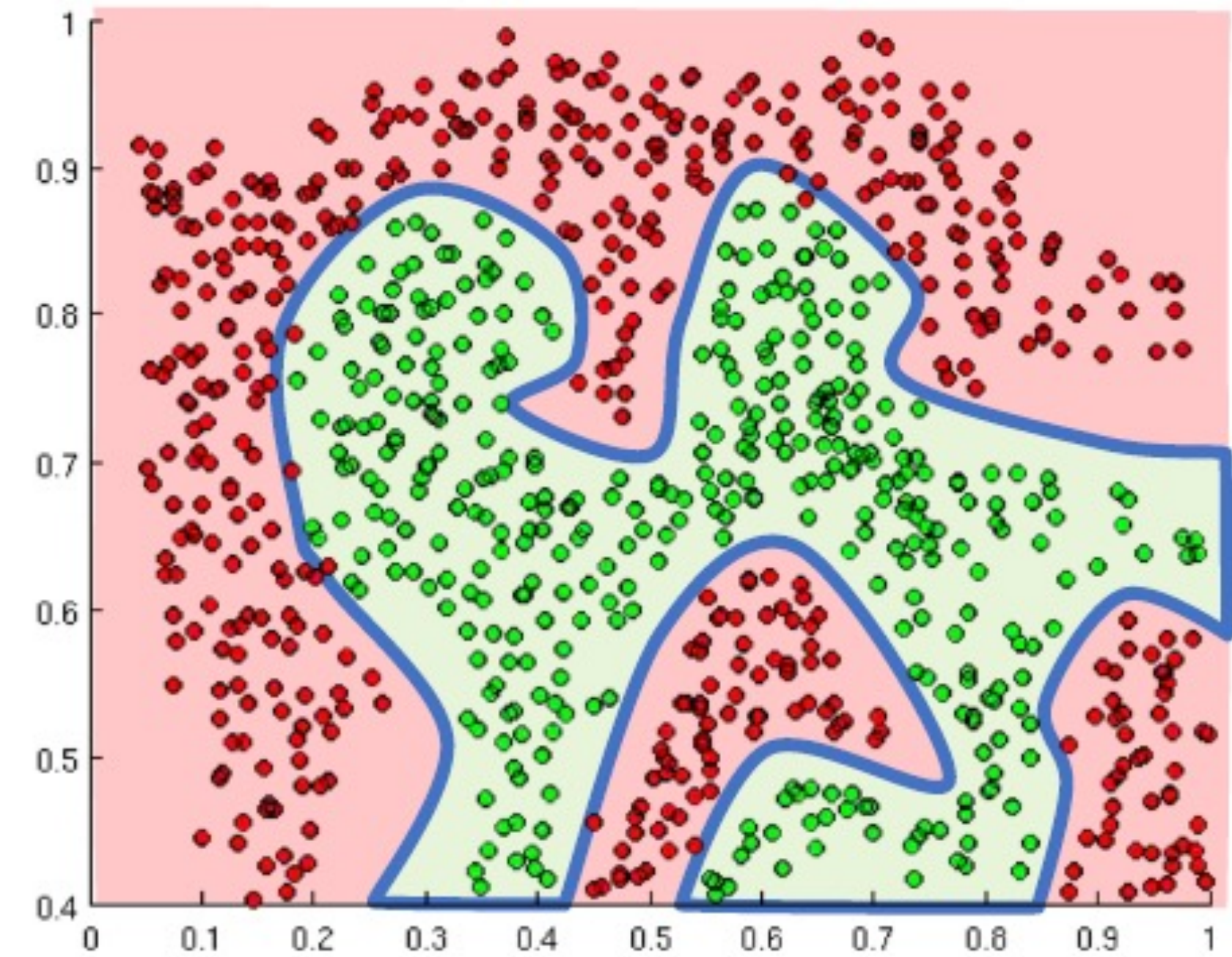
$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Activation functions



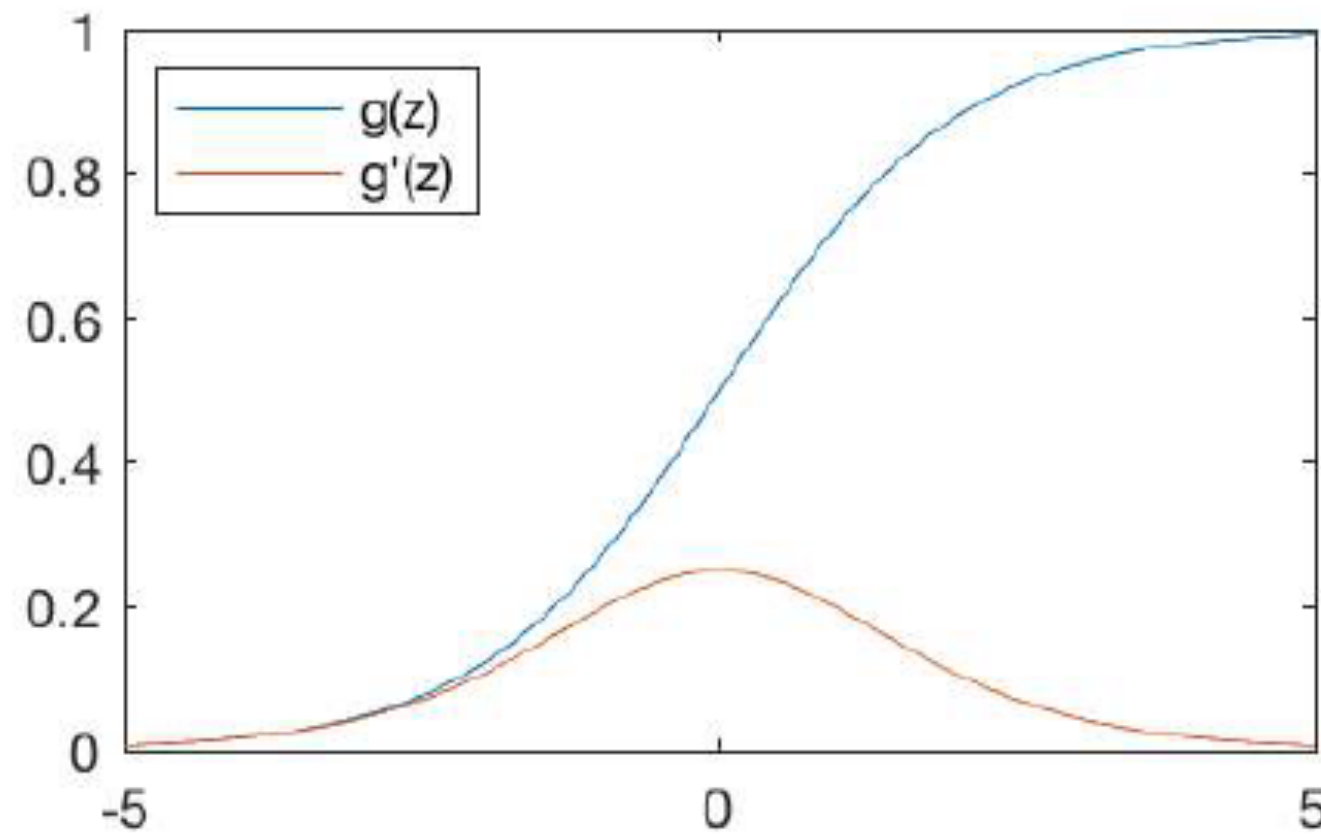
Linear Activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Activation functions

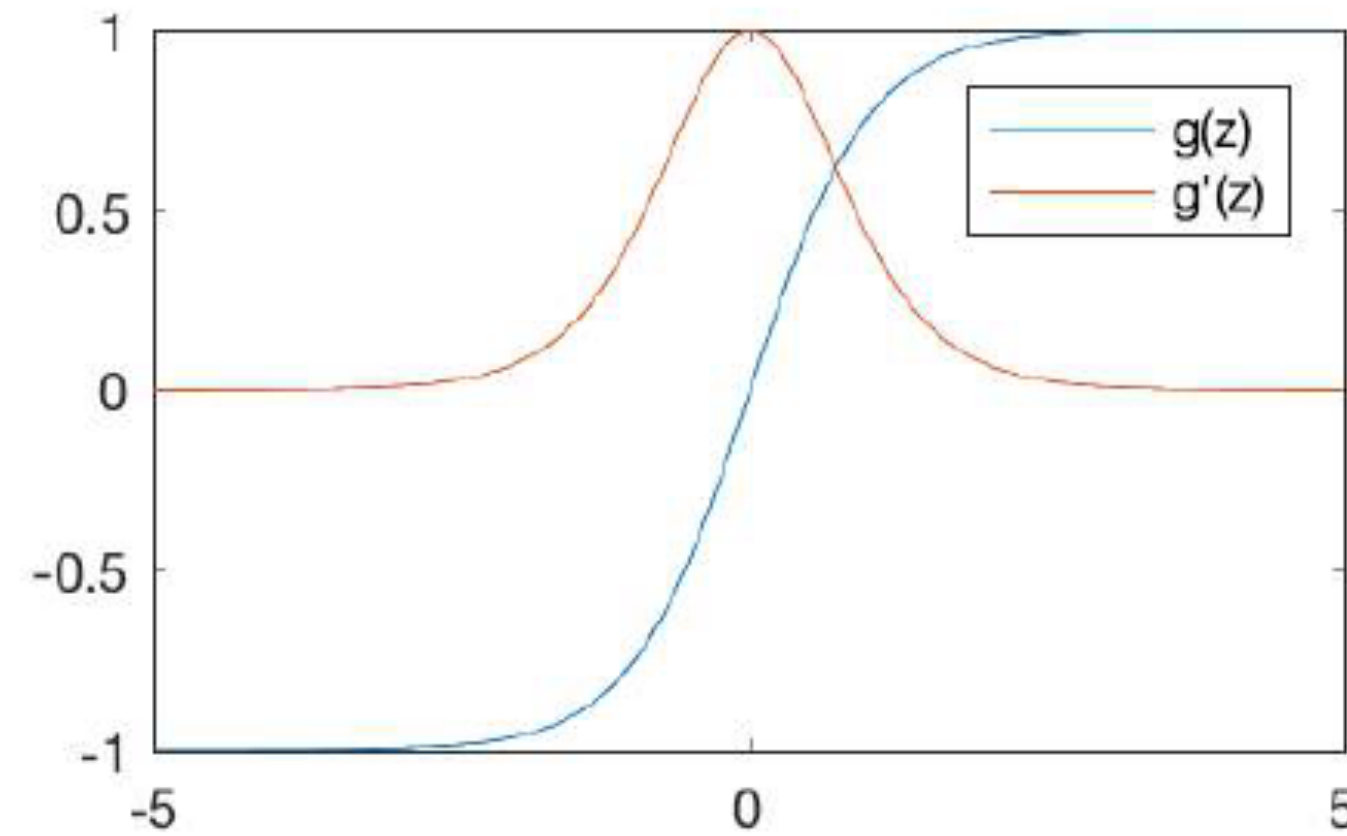
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

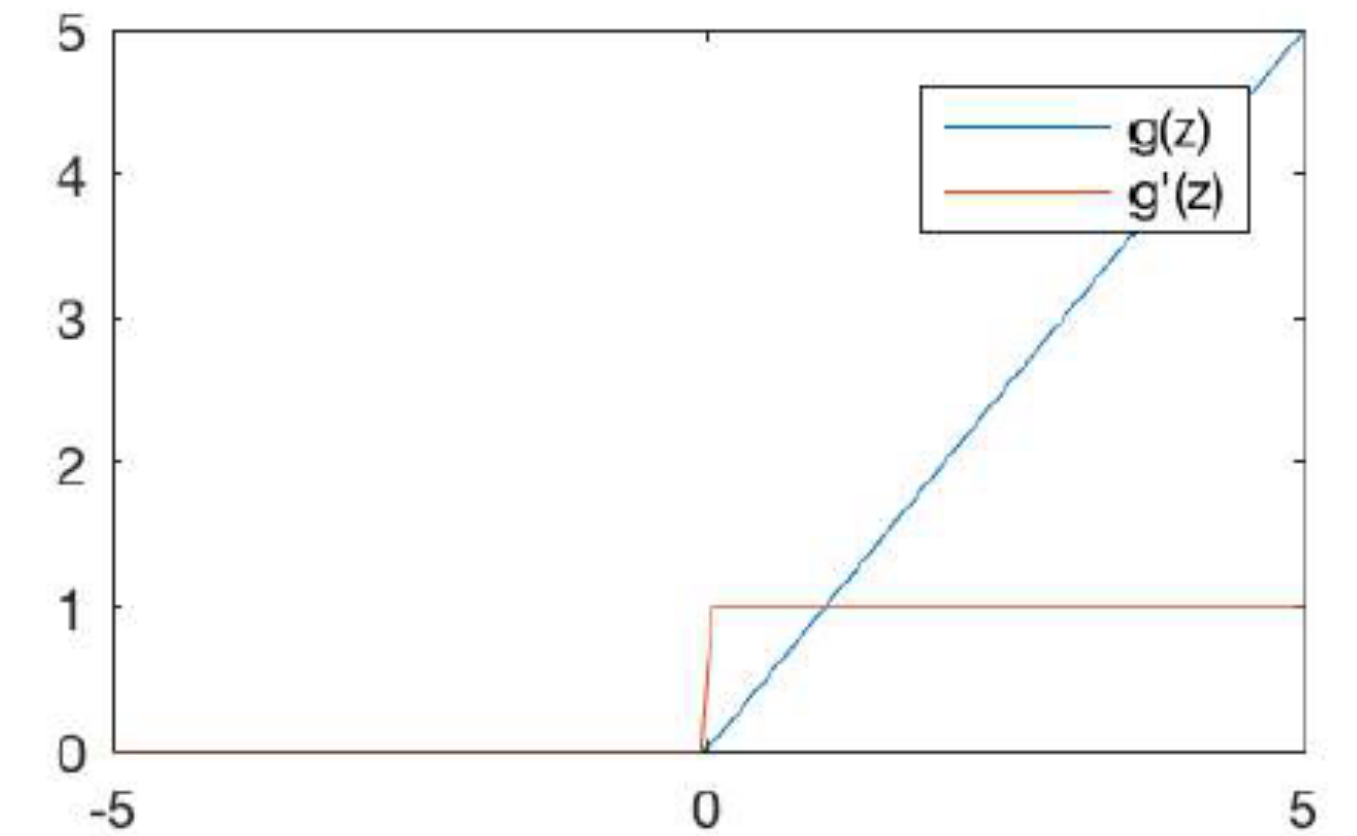
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

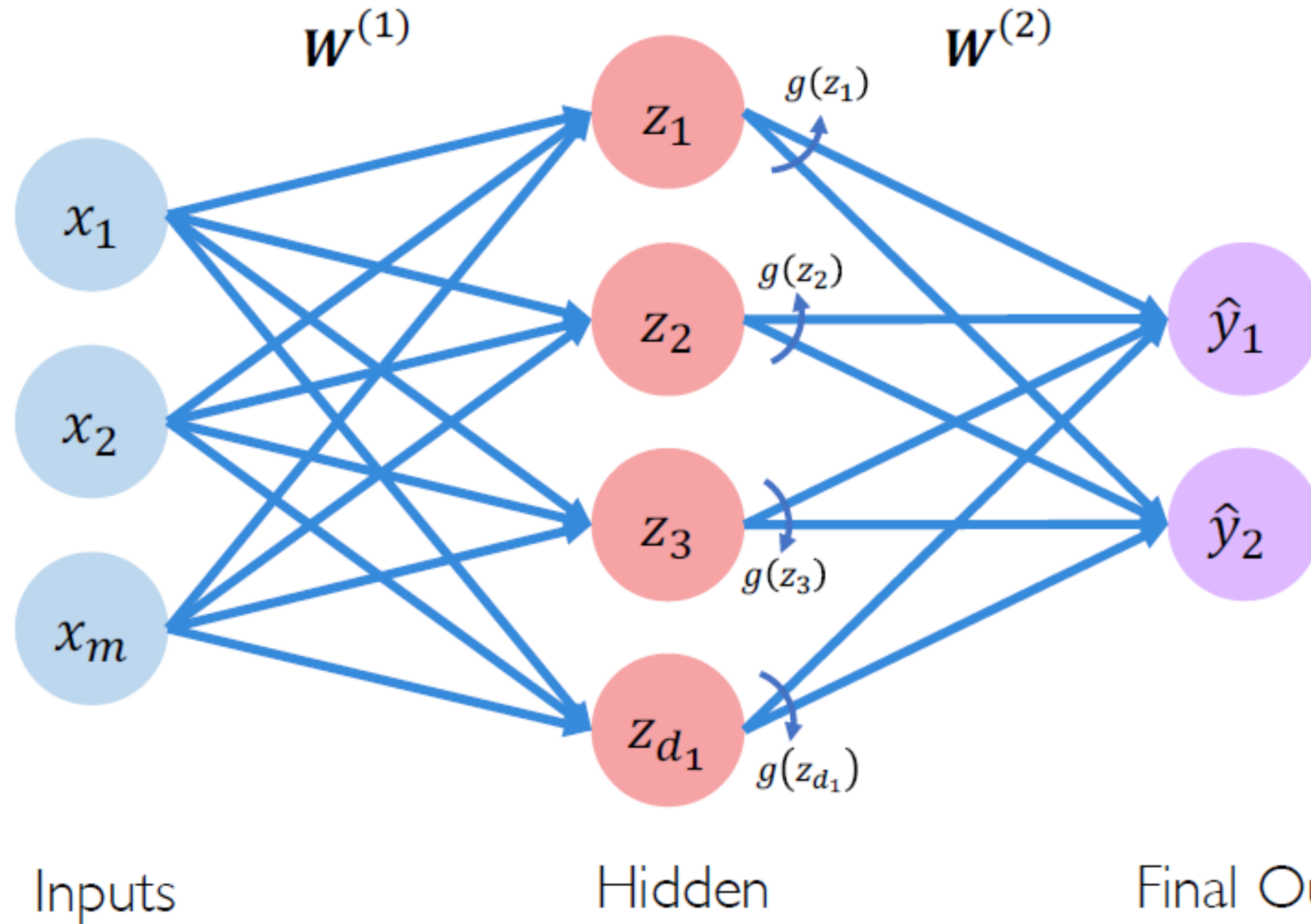
Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

Feedforward (Single Layer) Neural Network

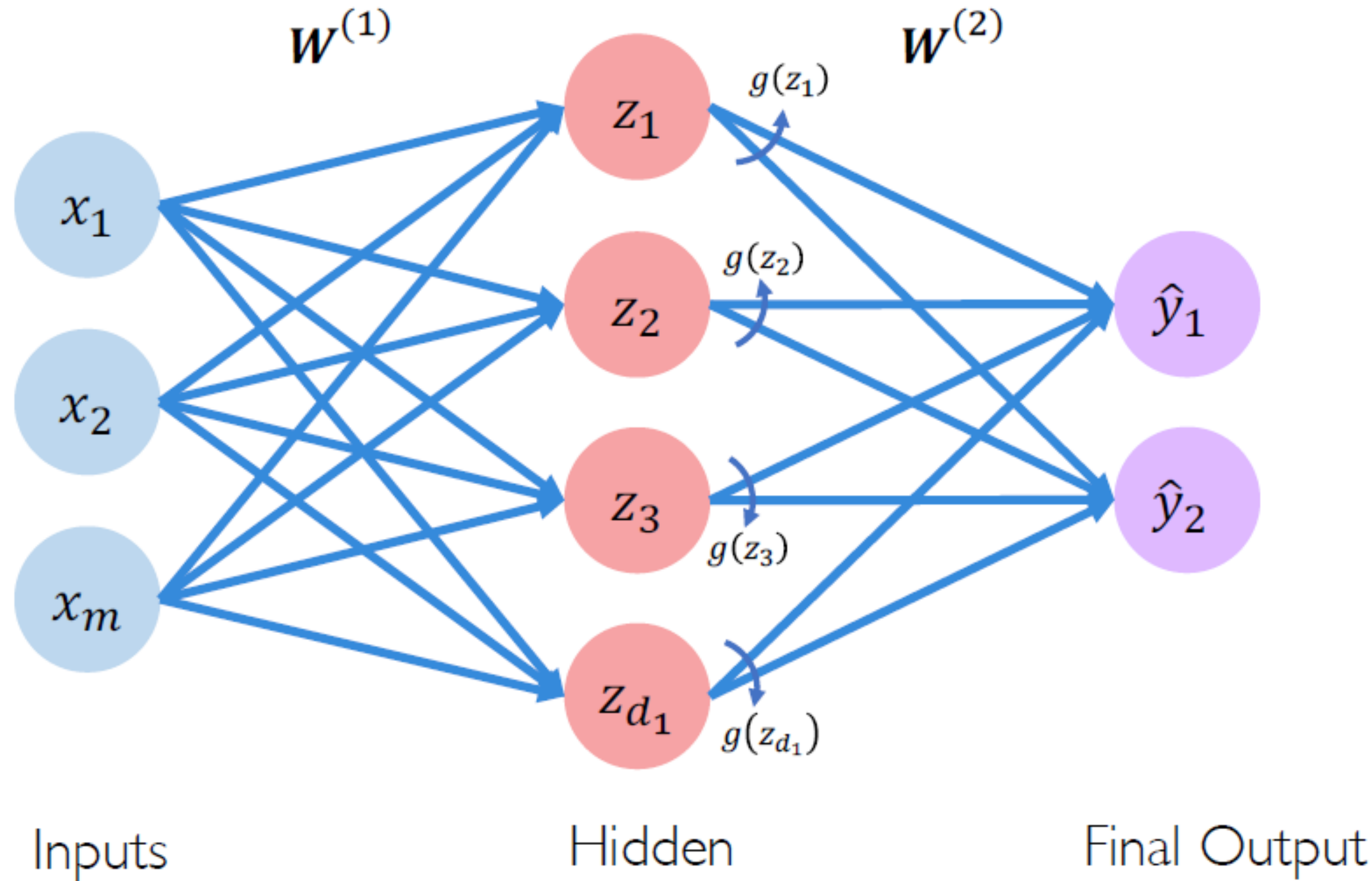


So-called *Vanilla Neural Network*

T. Hastie, R. Tibshirani, J. Friedman *The Elements of Statistical Learning*

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)} \right)$$

Feedforward (Single Layer) Neural Network

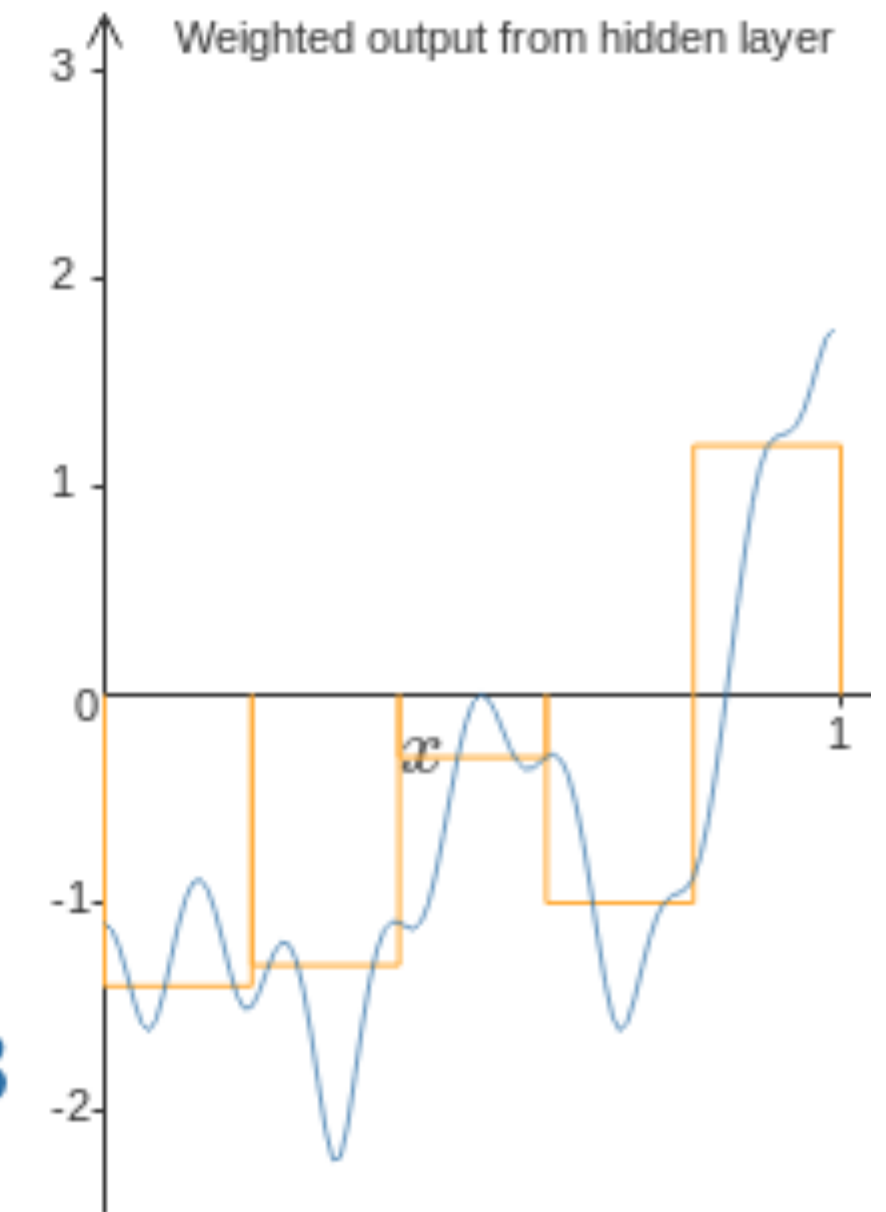
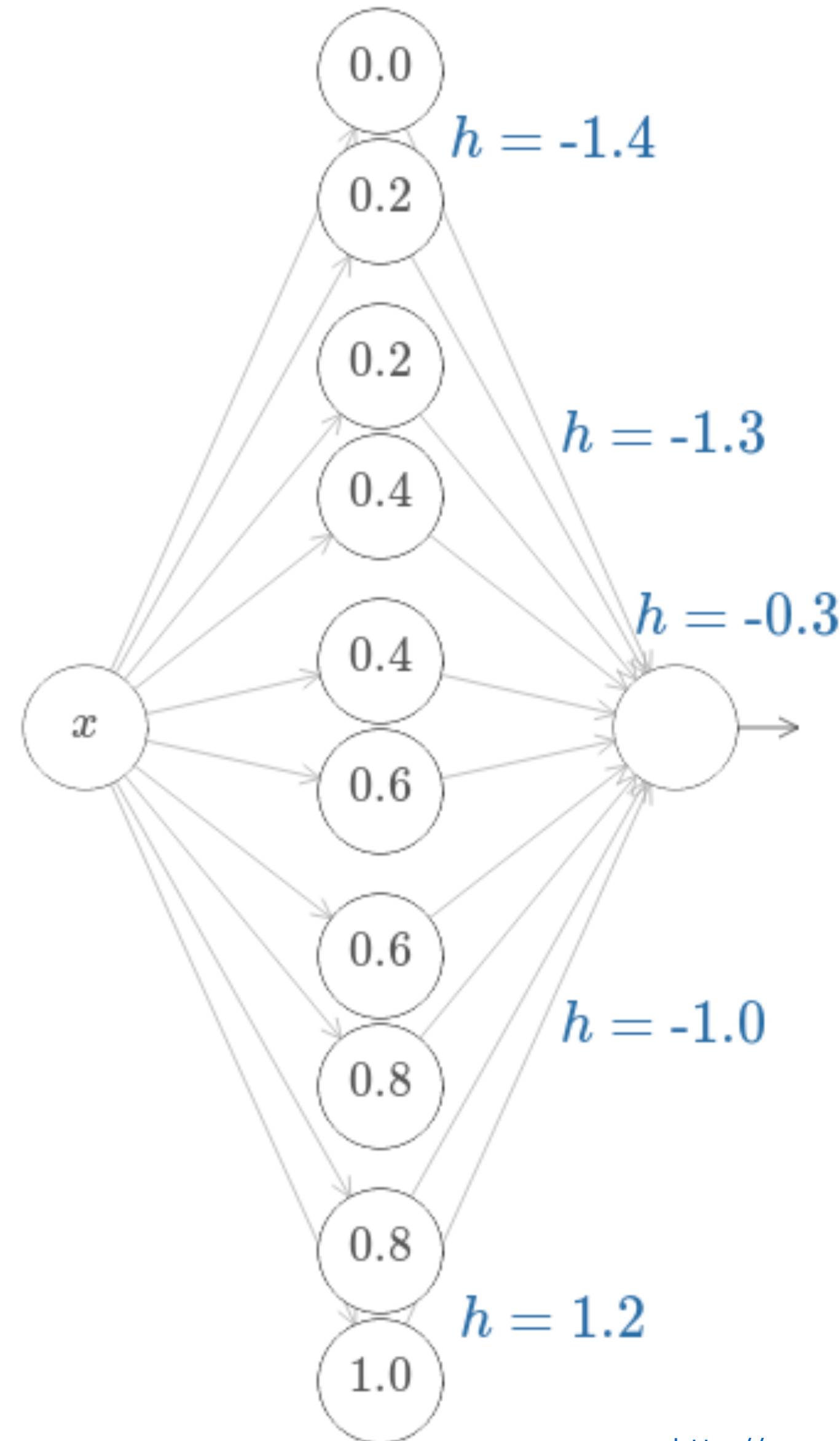
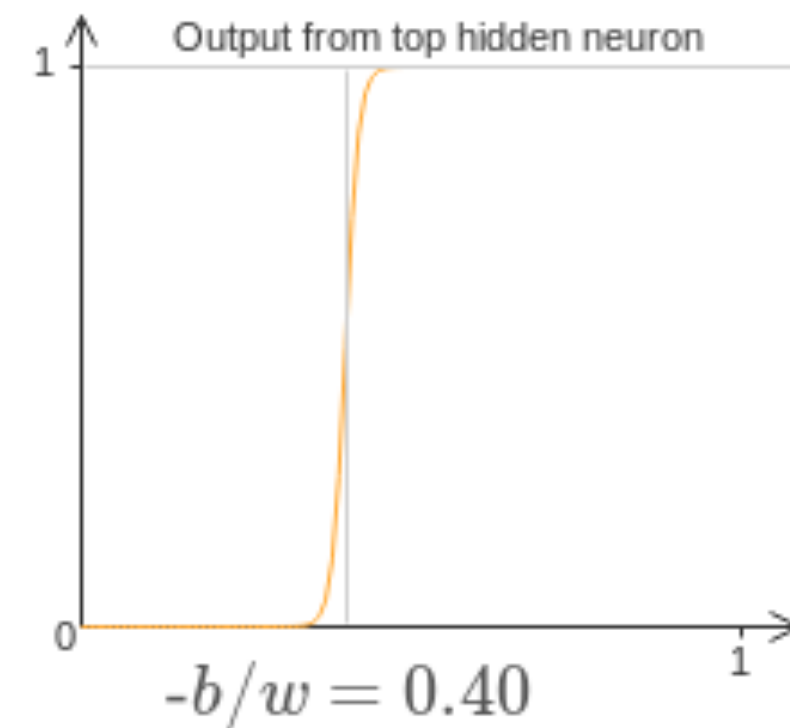
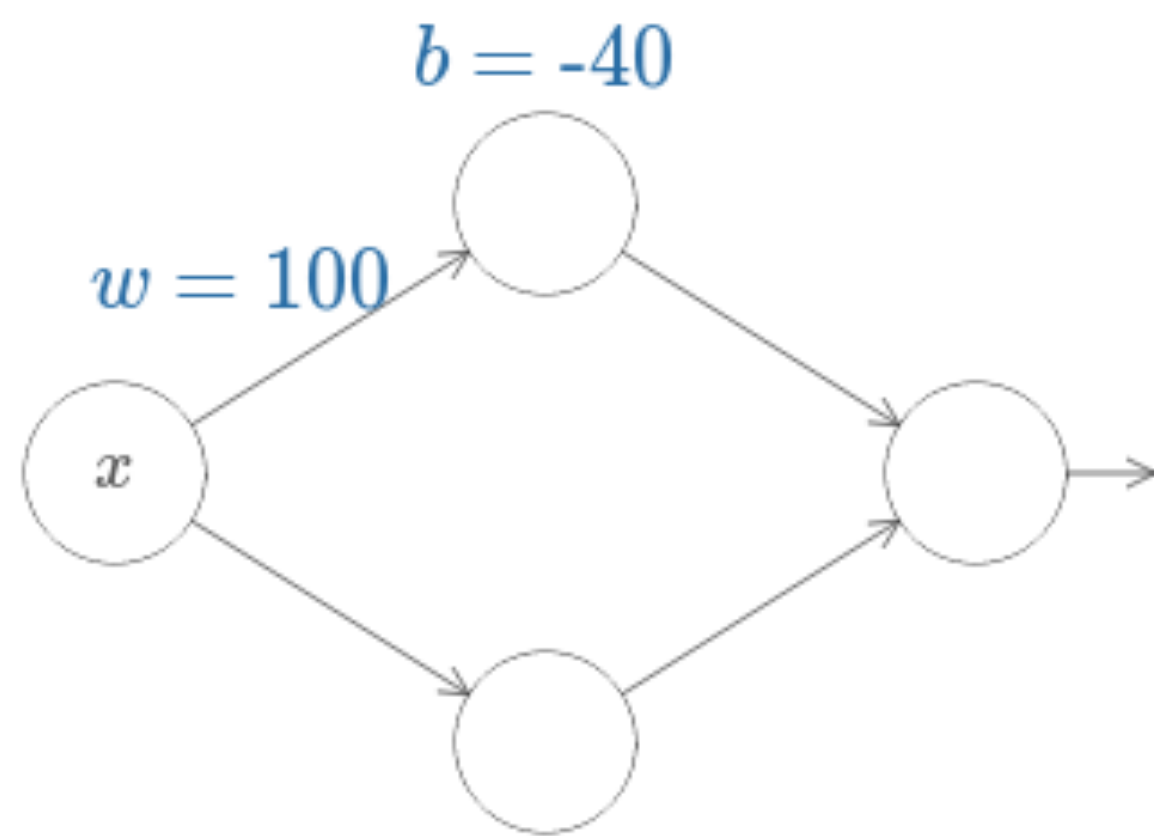


Concepts:

- Fully connected network
- Architecture design:
 - Size of the hidden layer
 - Weights
 - Initialization

Universality

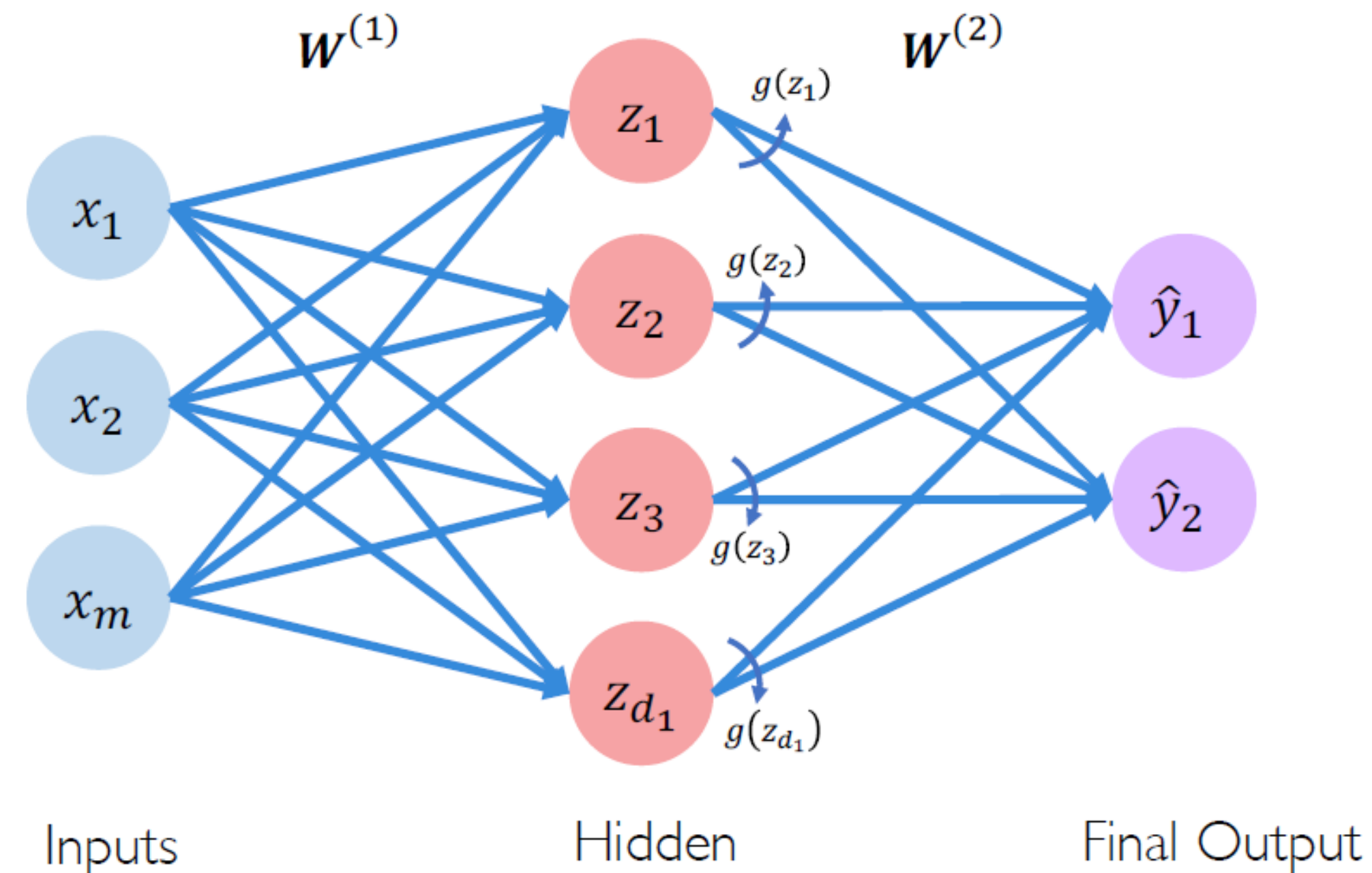
Neural nets with a single hidden layer can be used to approximate any continuous function to any desired precision.



Average deviation: 0.38
Success!

Reset

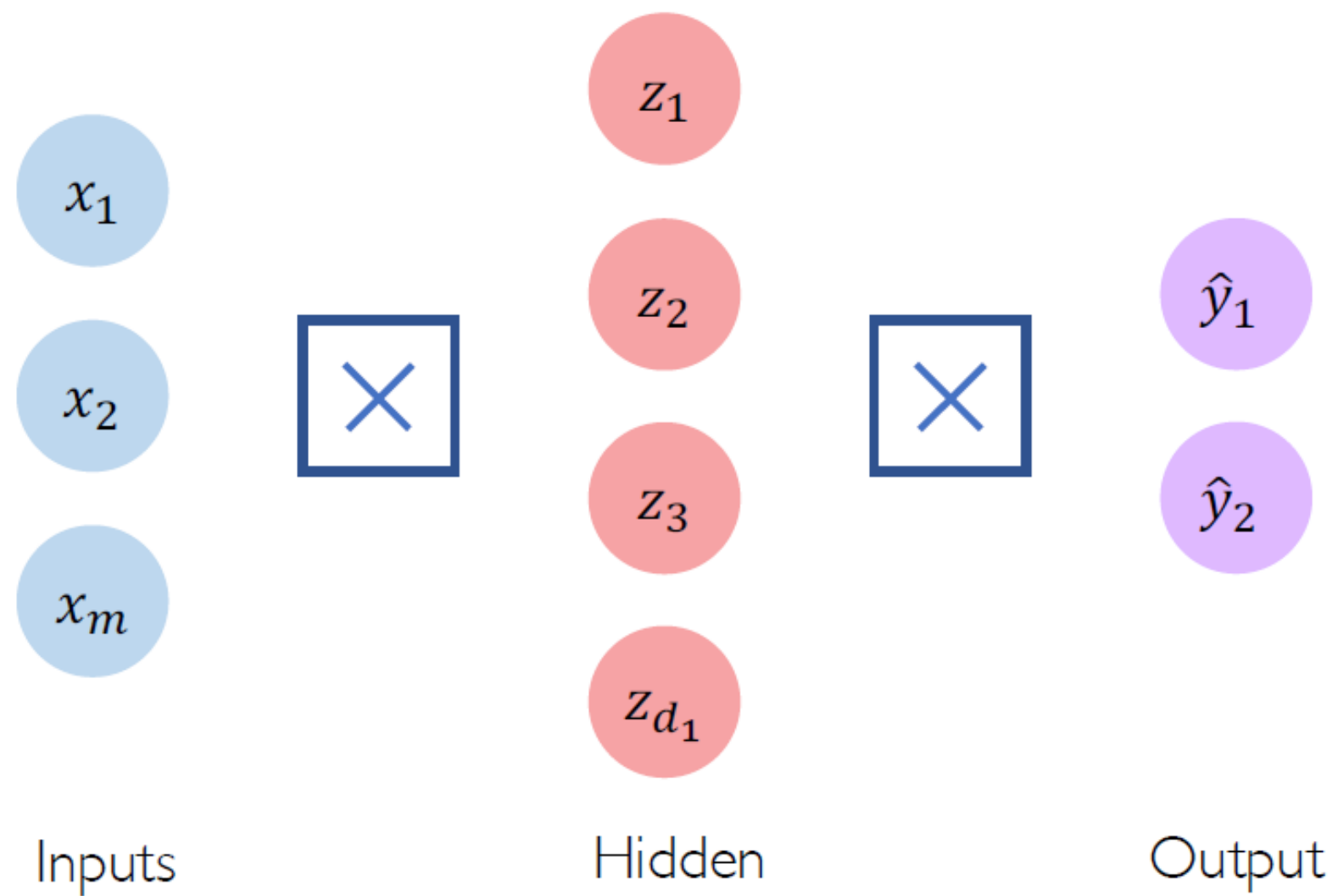
From Neural Network to Deep Learning architecture



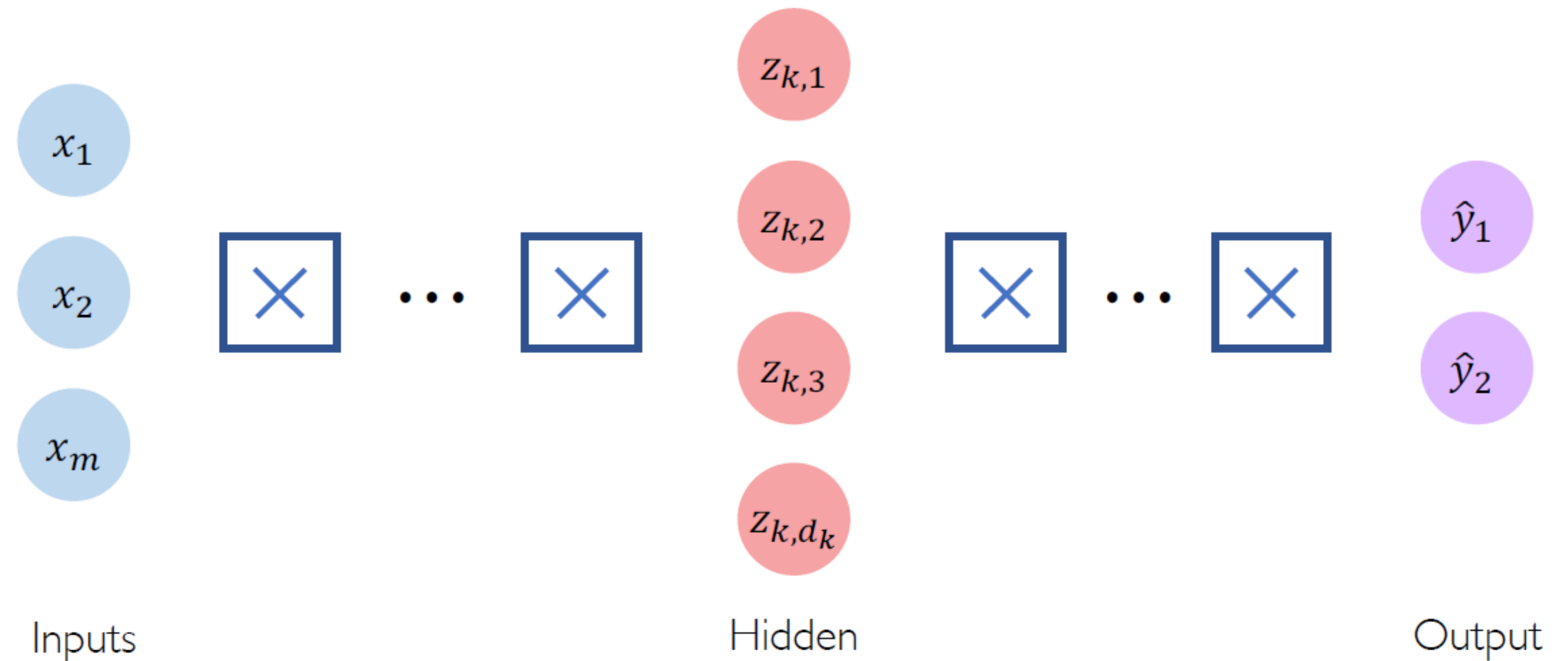
MIT Introduction to Deep Learning <http://introtodeeplearning.com>

- A Vanilla Neural Network is a Universal Approximator*: any functions can be described by it
- However, some functions can be compactly represented with k layers, while they may need an exponential number of neurons with 1 layer

From Neural Network to Deep Learning architecture



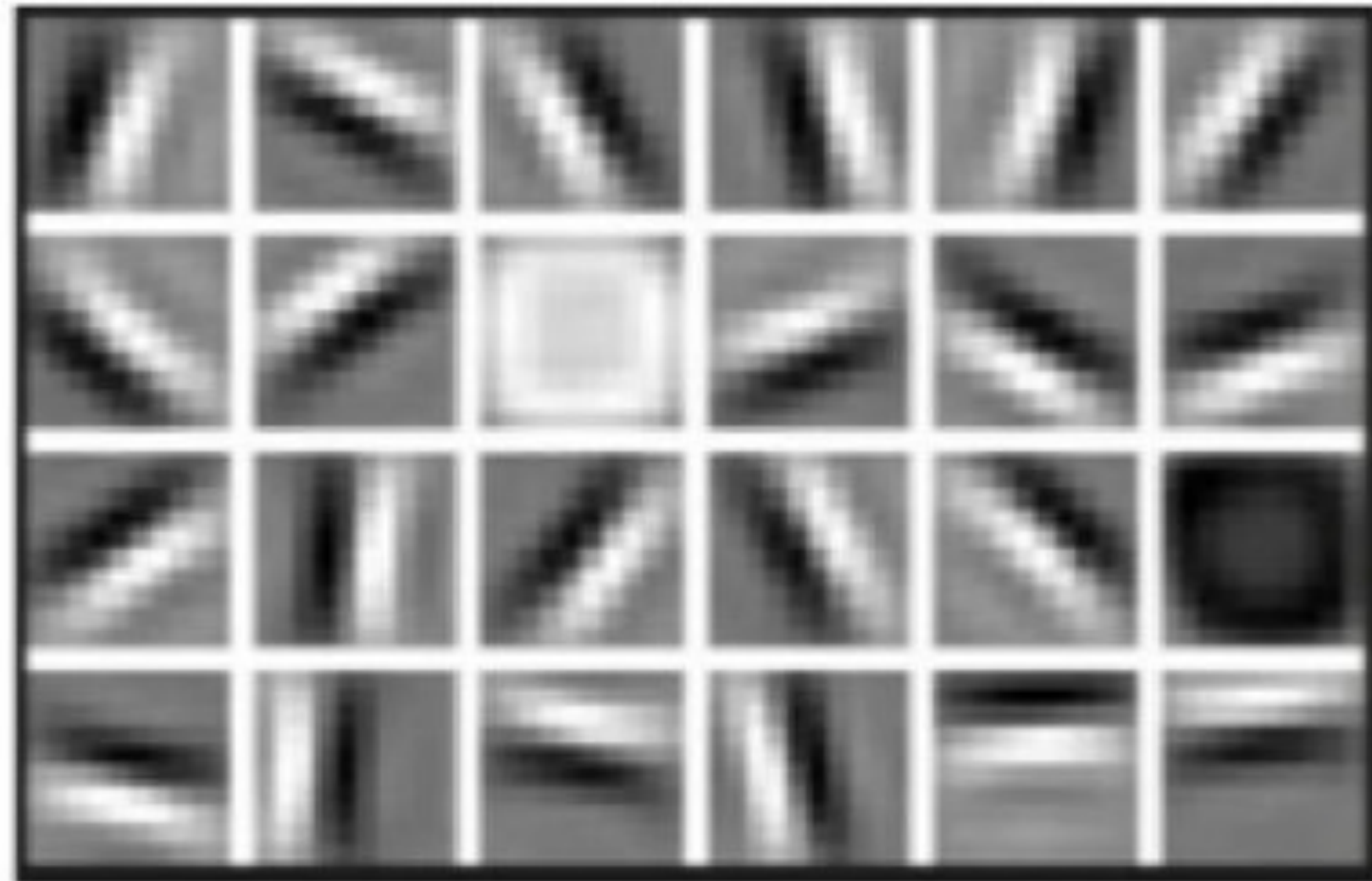
From 1 Layer to 'many' layers



$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{d_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

From Neural Network to Deep Learning architecture

Low Level Features



Lines & Edges

Mid Level Features



Eyes & Nose & Ears

High Level Features



Facial Structure

Training a Neural Network: Loss Function

$$\mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

A *Loss* expresses a cost associated with the NN predictions

The *empirical loss* (also known as *objective function*, *cost function*, *empirical risk*) measures the total loss over the (training/test) dataset

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Training a Neural Network: Loss Function

Depending on the task we can have different losses:

- *Cross entropy loss*: for classification tasks, NNs that have in output a probability

$$J(W) = \frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log \left(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log \left(1 - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right)$$

$f(x)$	y
0.1	1
0.8	0
0.6	1
$\vdots$	$\vdots$

- *Mean squared error loss* for regression tasks

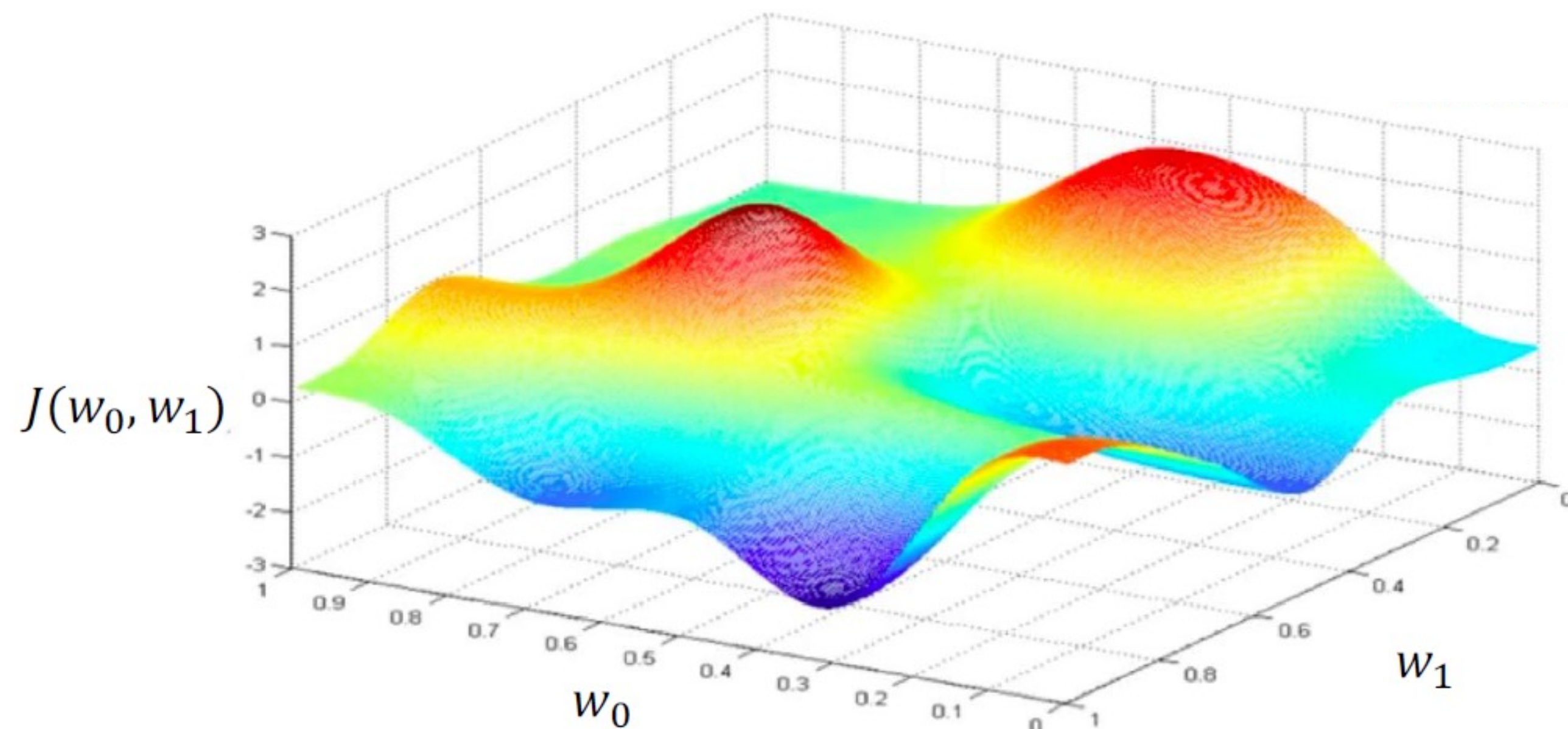
$$J(W) = \frac{1}{n} \sum_{i=1}^n \left(\underbrace{y^{(i)}}_{\text{Actual}} - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right)^2$$

Training a Neural Network = Minimizing a Loss

We seek for a set of weights that achieve minimal loss:

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$

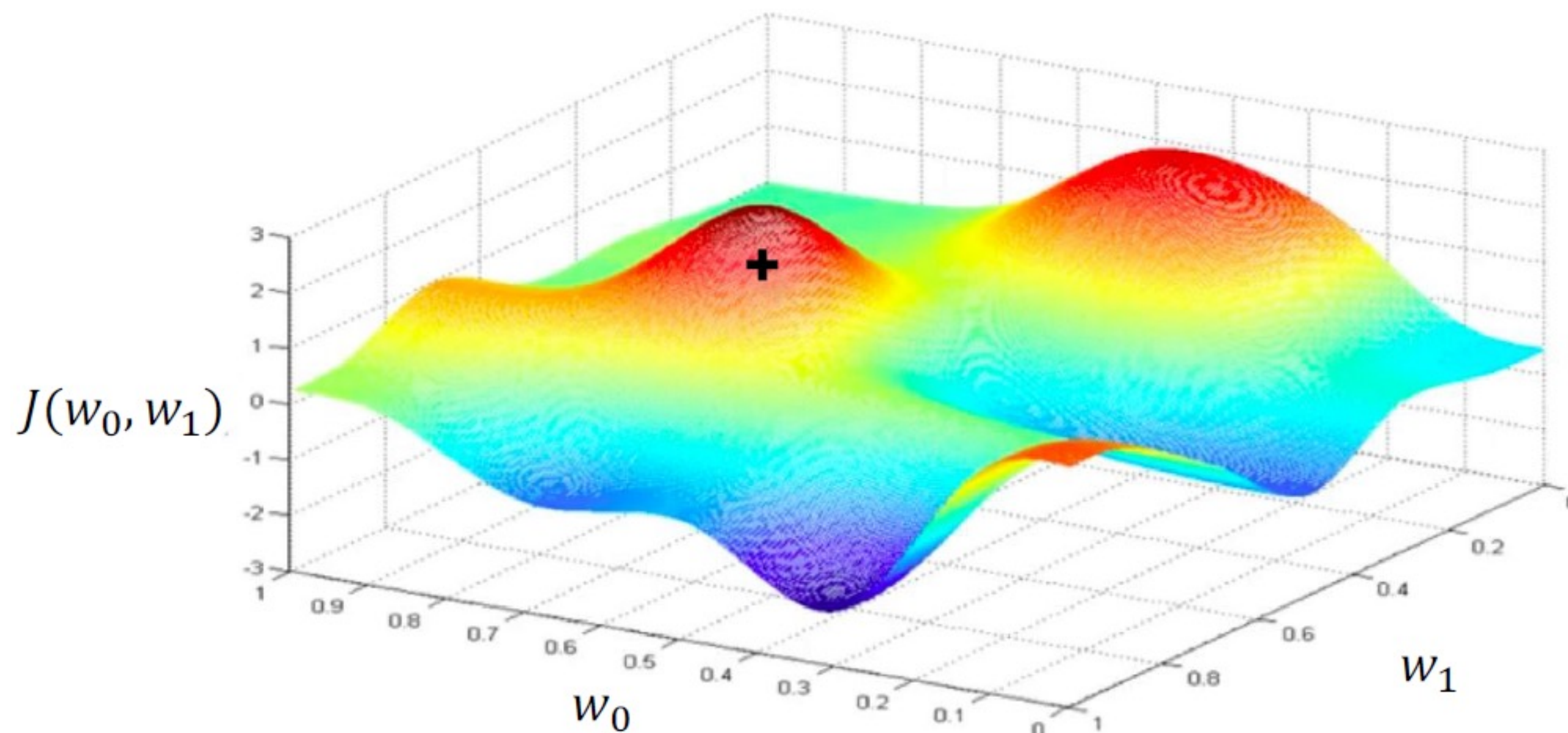


Training a Neural Network = Minimizing a Loss

We seek for a set of weights that achieve minimal loss:

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$



Algorithm

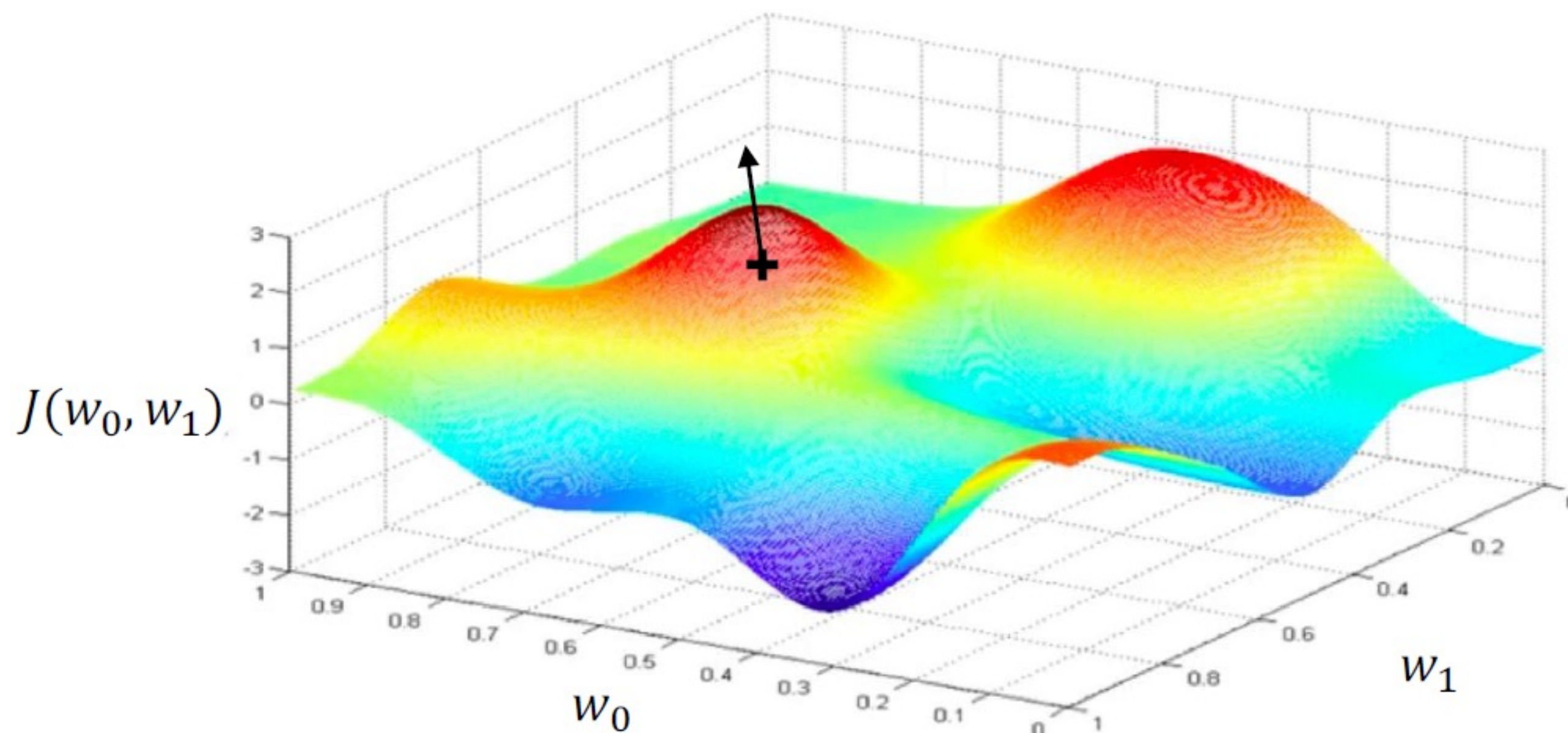
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$

Training a Neural Network = Minimizing a Loss

We seek for a set of weights that achieve minimal loss:

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$



Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$

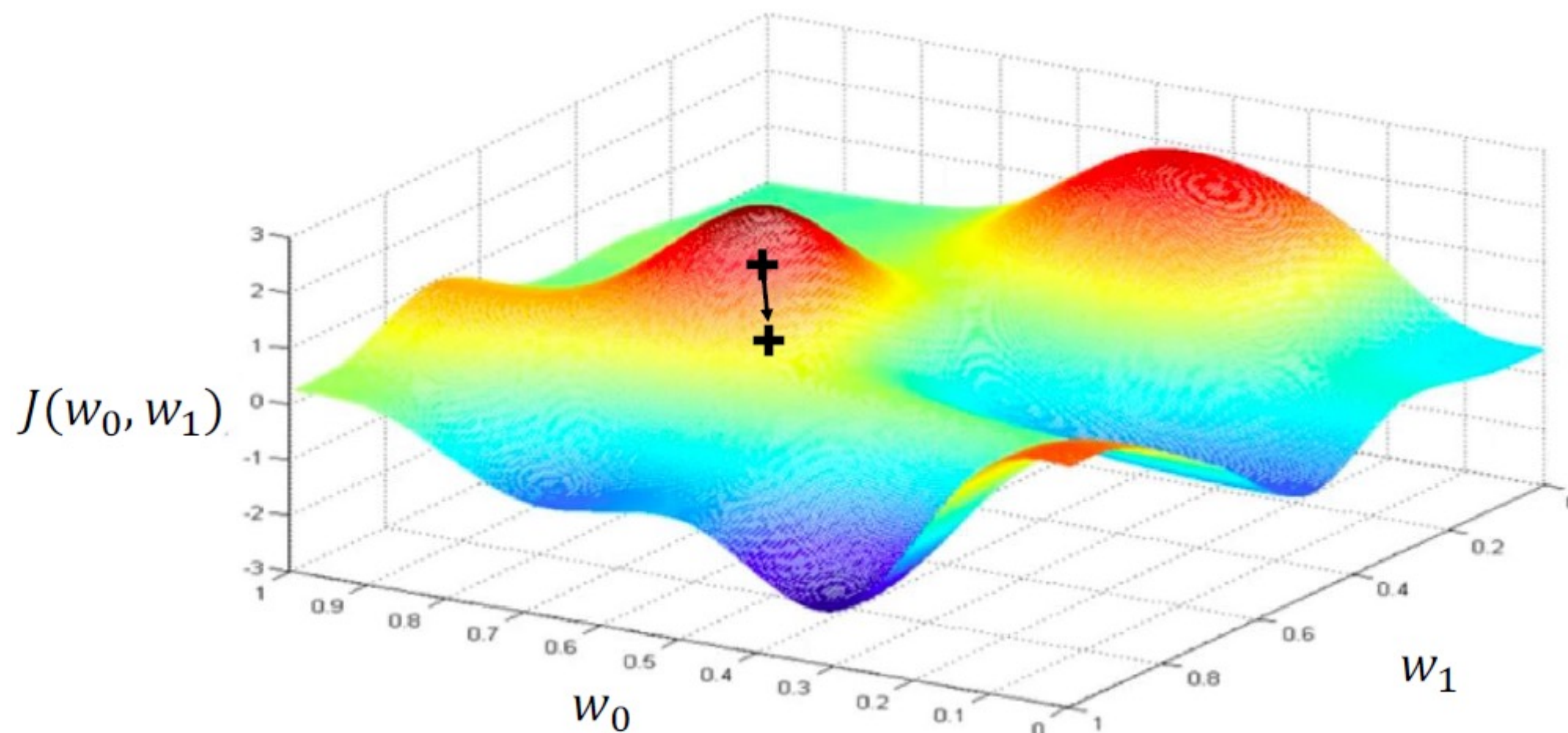
Compute gradient, $\frac{\partial J(W)}{\partial W}$

Training a Neural Network = Minimizing a Loss

We seek for a set of weights that achieve minimal loss:

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$



Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$

Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$

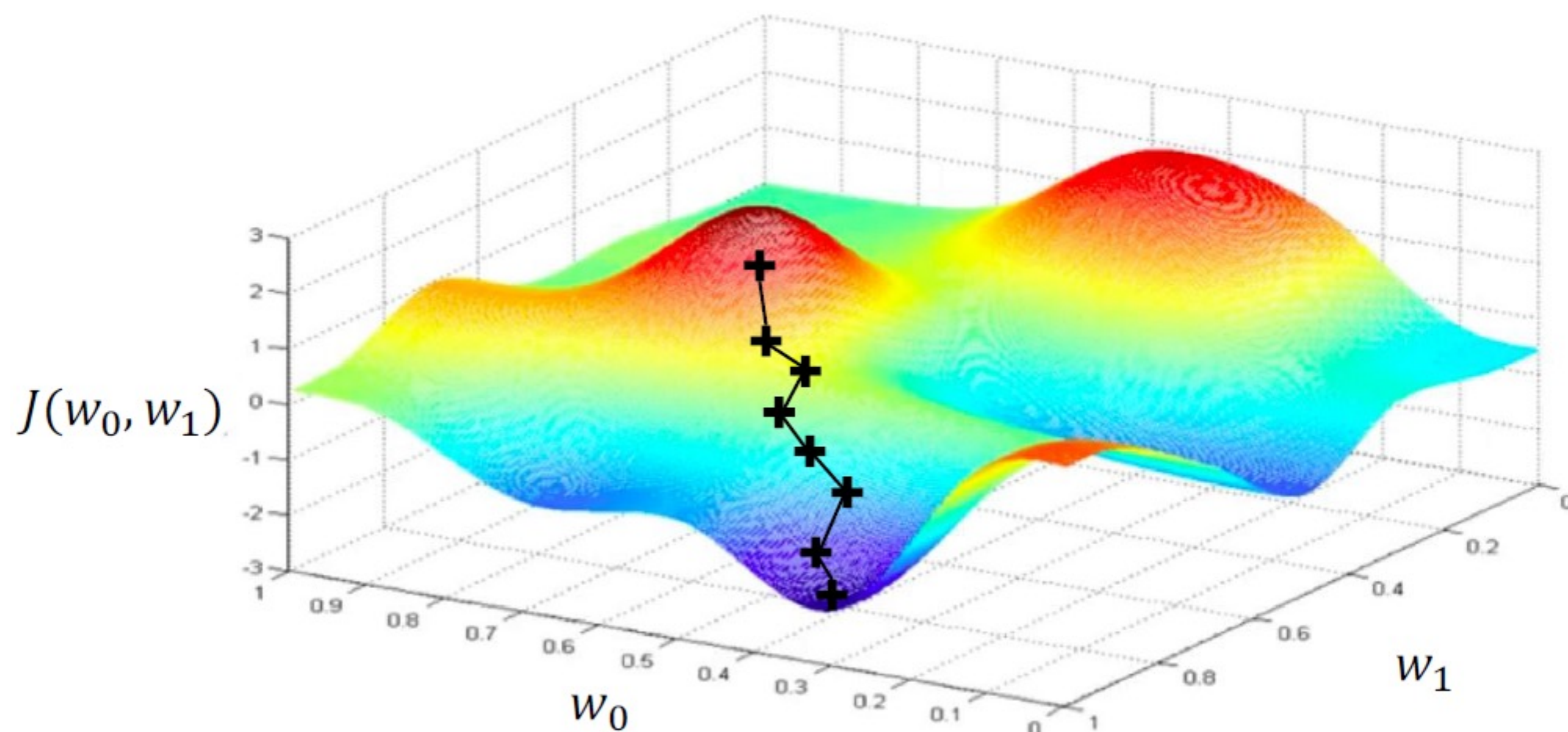
Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$

Training a Neural Network = Minimizing a Loss

We seek for a set of weights that achieve minimal loss:

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$



Algorithm

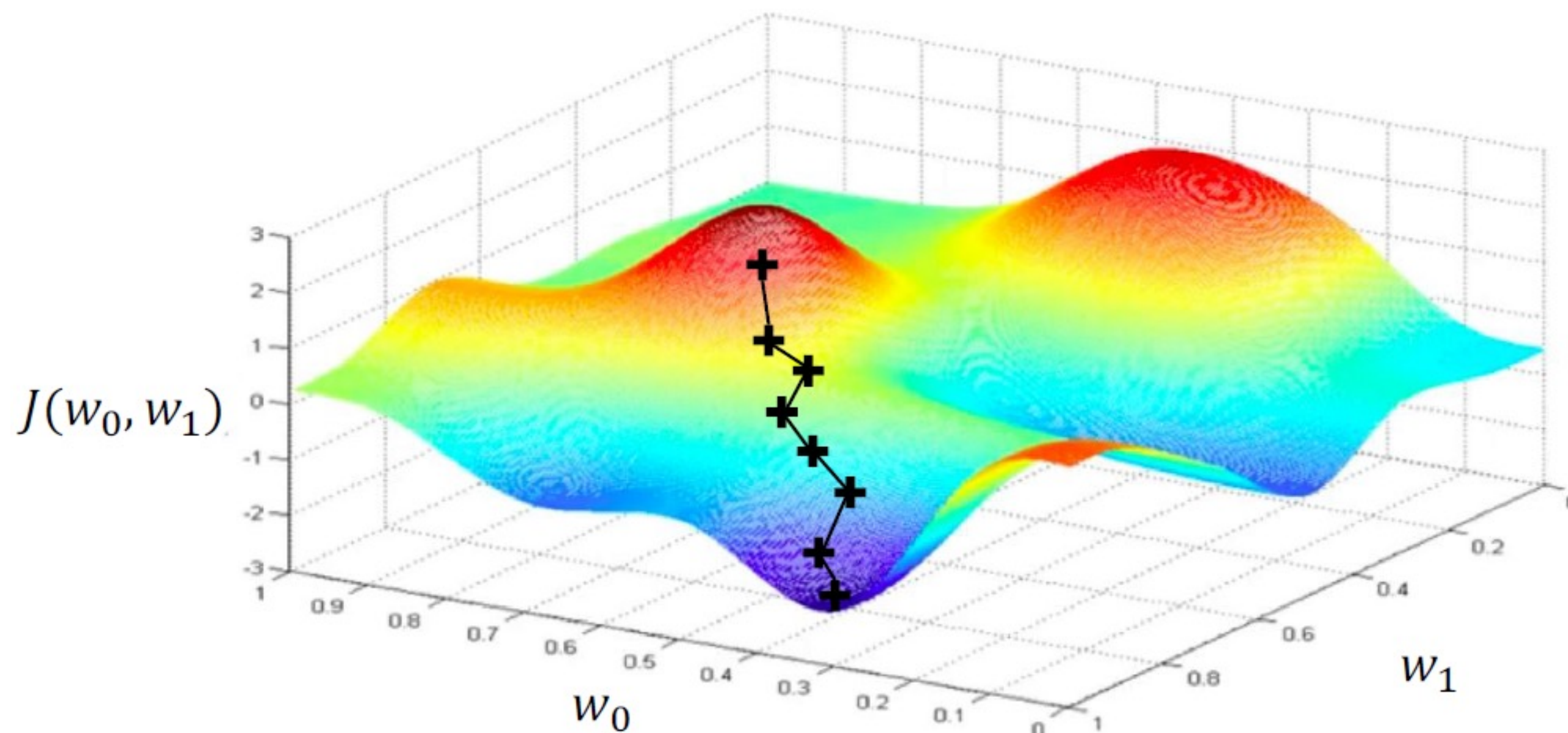
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

Training a Neural Network = Minimizing a Loss

We seek for a set of weights that achieve minimal loss:

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

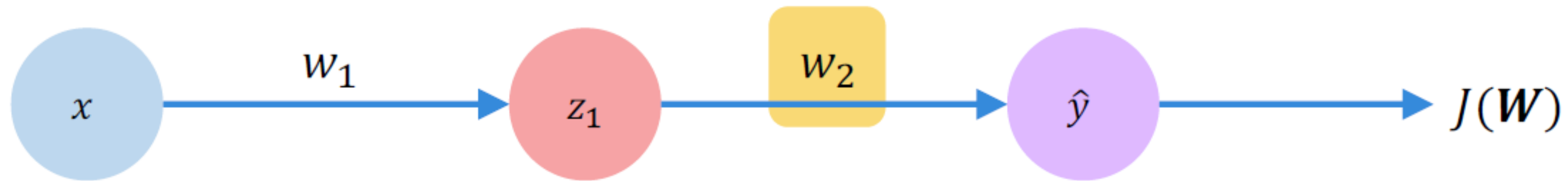


Algorithm (Gradient Descent)

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
Learning Rate
5. Return weights

How to compute the gradient: backpropagation

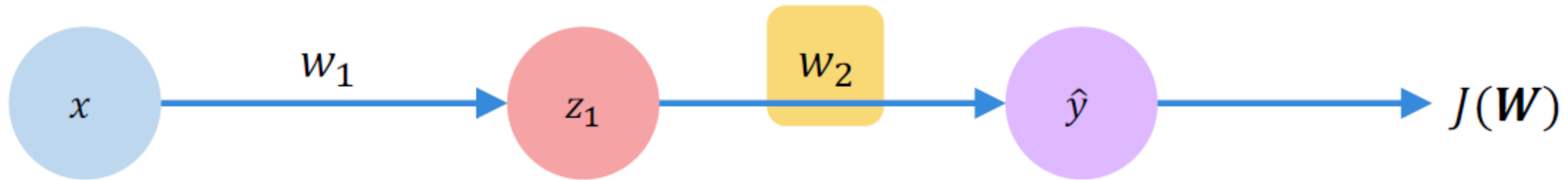
Backpropagation is about understanding how changing the weights and biases in a network changes the cost function.



Let's consider a simple NN with one node: how the final loss is affected by changes in w_2 ?

How to compute the gradient: backpropagation

Let's consider a simple NN with one node: how the final loss is affected by changes in w_2

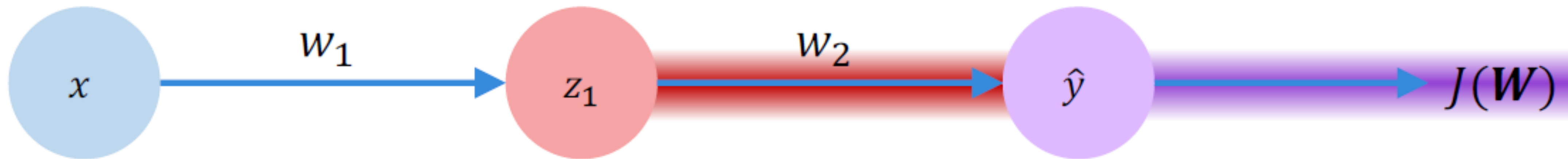


We apply the chain rule!

$$\frac{\partial J(W)}{\partial w_2} =$$

How to compute the gradient: backpropagation

Let's consider a simple NN with one node: how the final loss is affected by changes in w_2

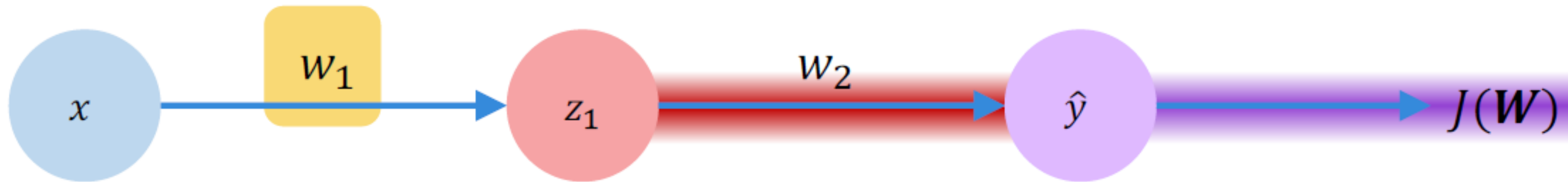


We apply the chain rule!

$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

How to compute the gradient: backpropagation

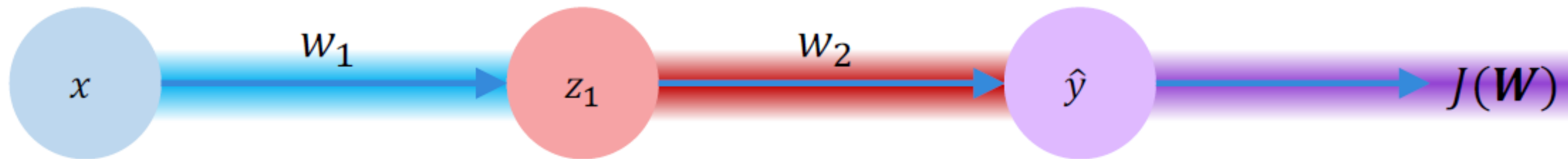
This simple network is characterized also by the weight w_1



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{Apply chain rule!}} * \underbrace{\frac{\partial \hat{y}}{\partial w_1}}_{\text{Apply chain rule!}}$$

How to compute the gradient: backpropagation

This simple network is characterized also by the weight w_1

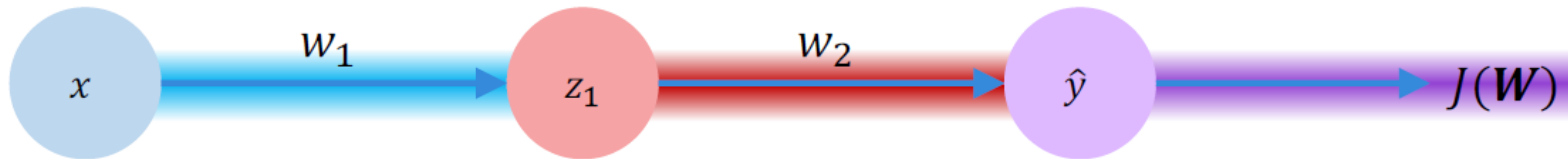


$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

This procedure has to be done for each parameter of the newtwork!

How to compute the gradient: backpropagation

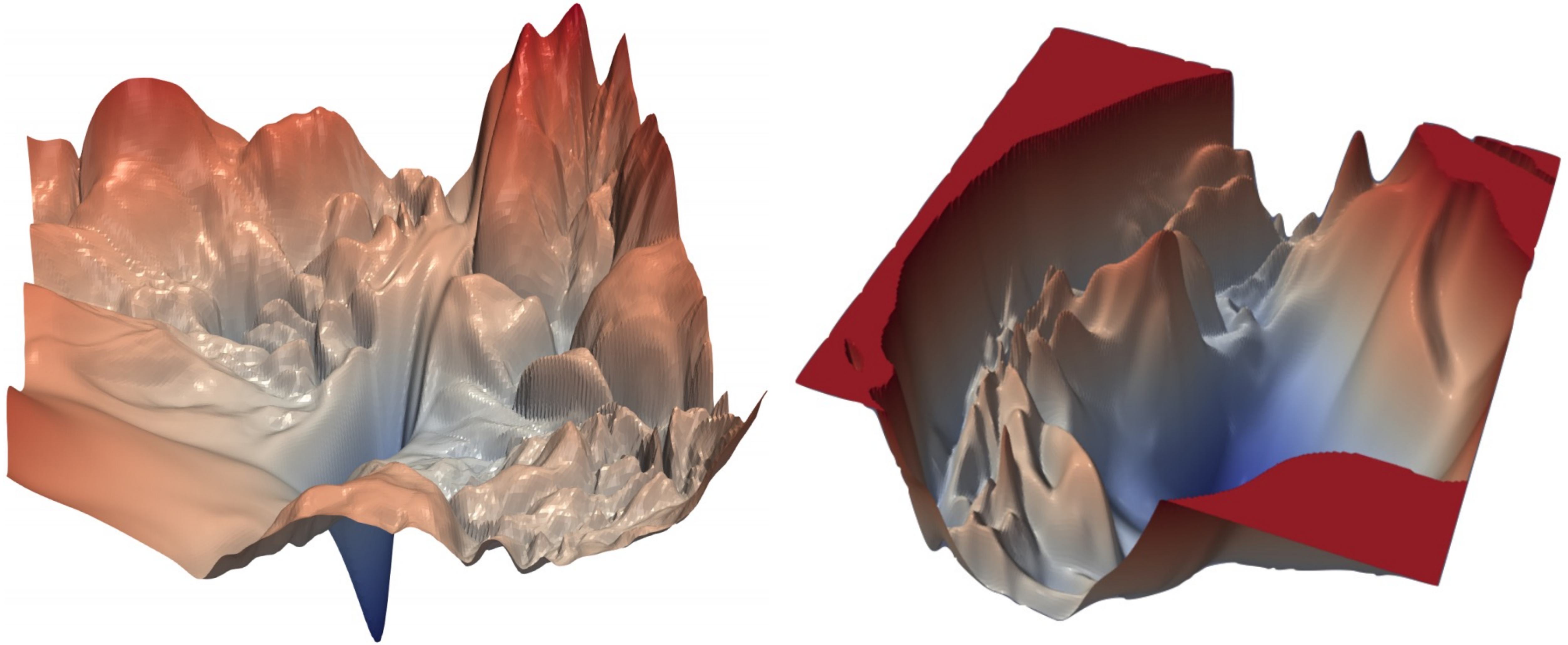
This simple network is characterized also by the weight w_1



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

You can have a look here for the math:
<http://neuralnetworksanddeeplearning.com/chap2.html>

Loss functions may be quite complex...



To cope with complex loss functions and to optimize training the following algorithm is generally sophisticated

Algorithm (Gradient Descent)

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

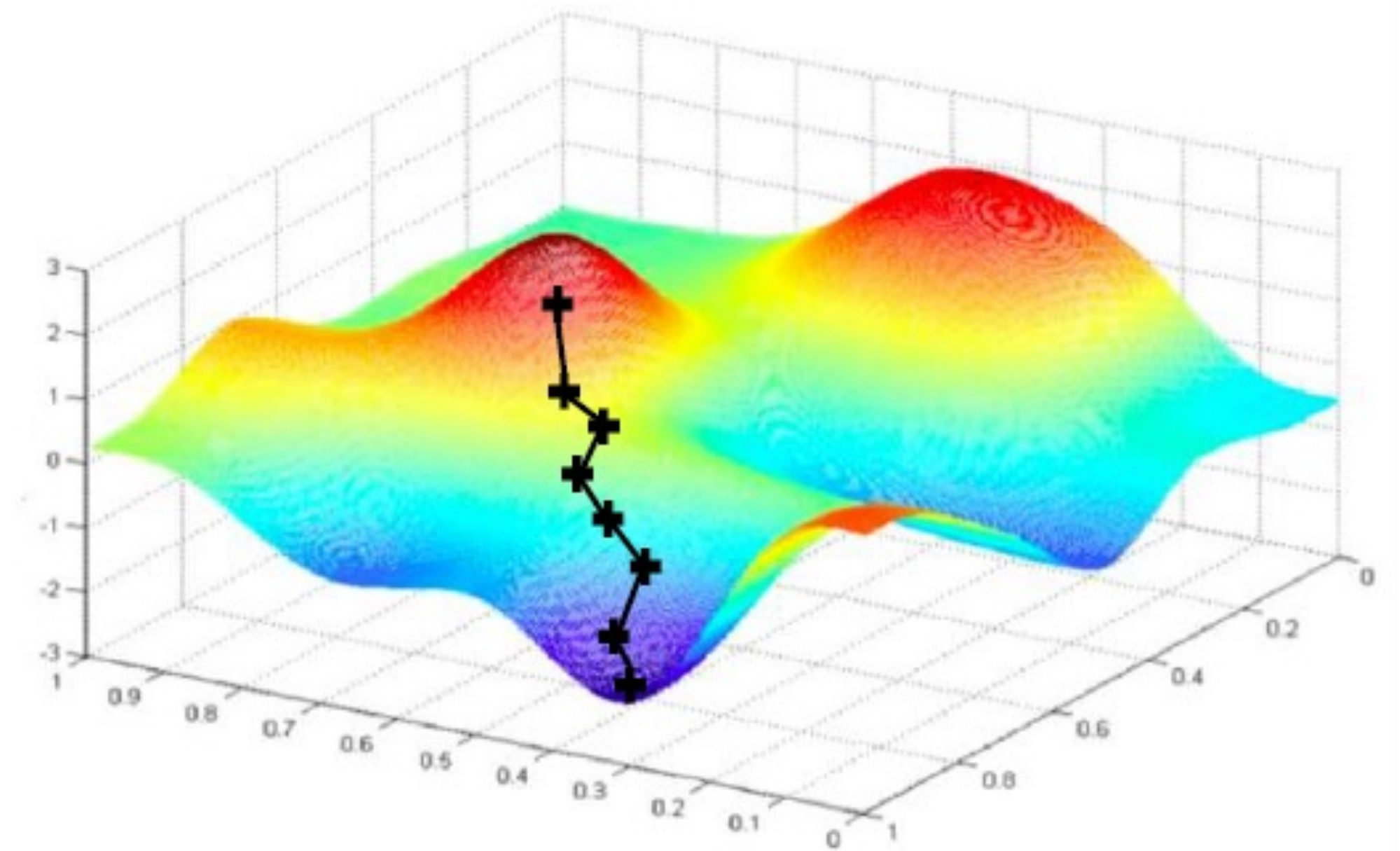
Learning Rate

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

Can be very burdensome
to compute...
Average over all samples
in the dataset!

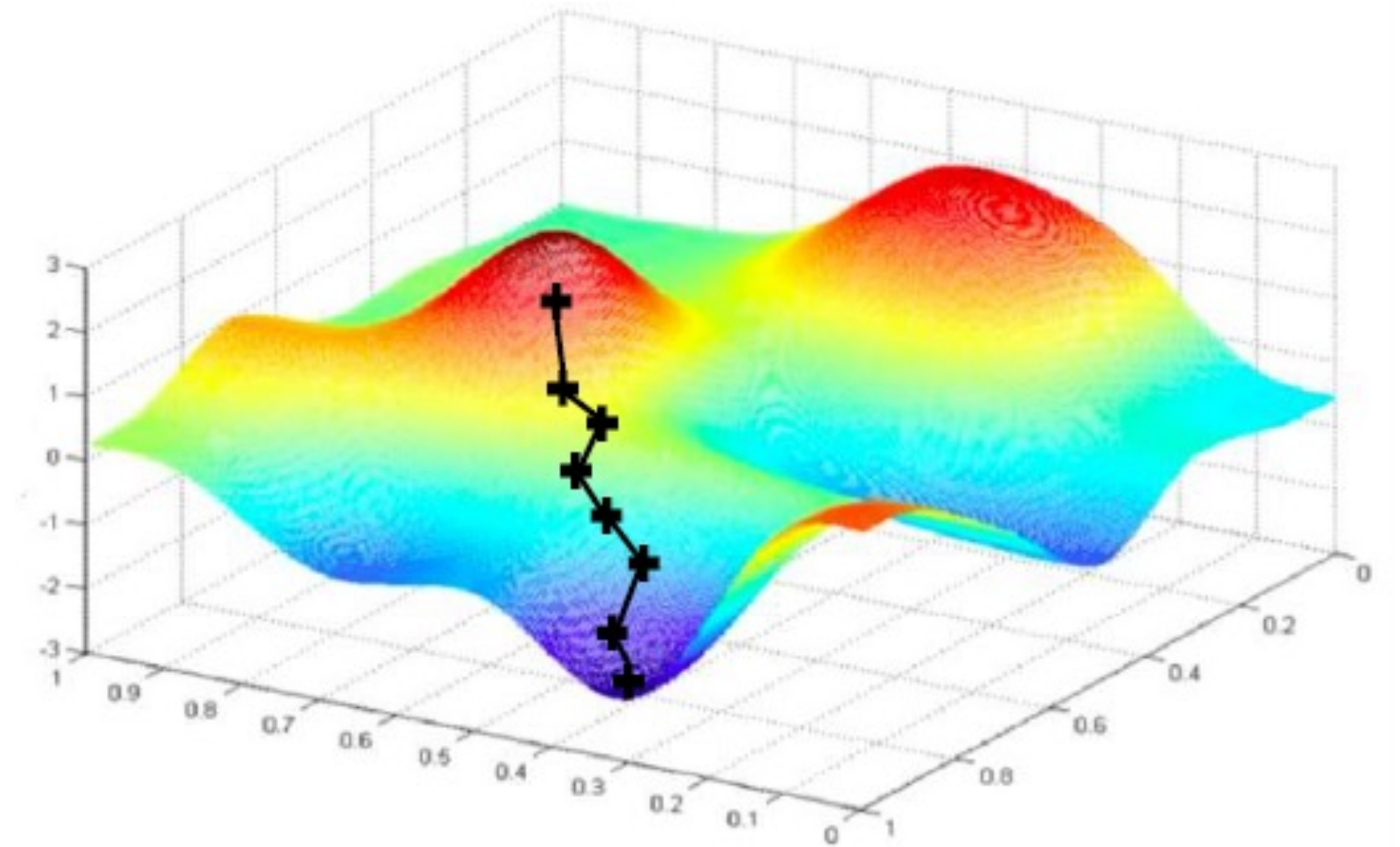


Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(W)}{\partial W}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
6. Return weights

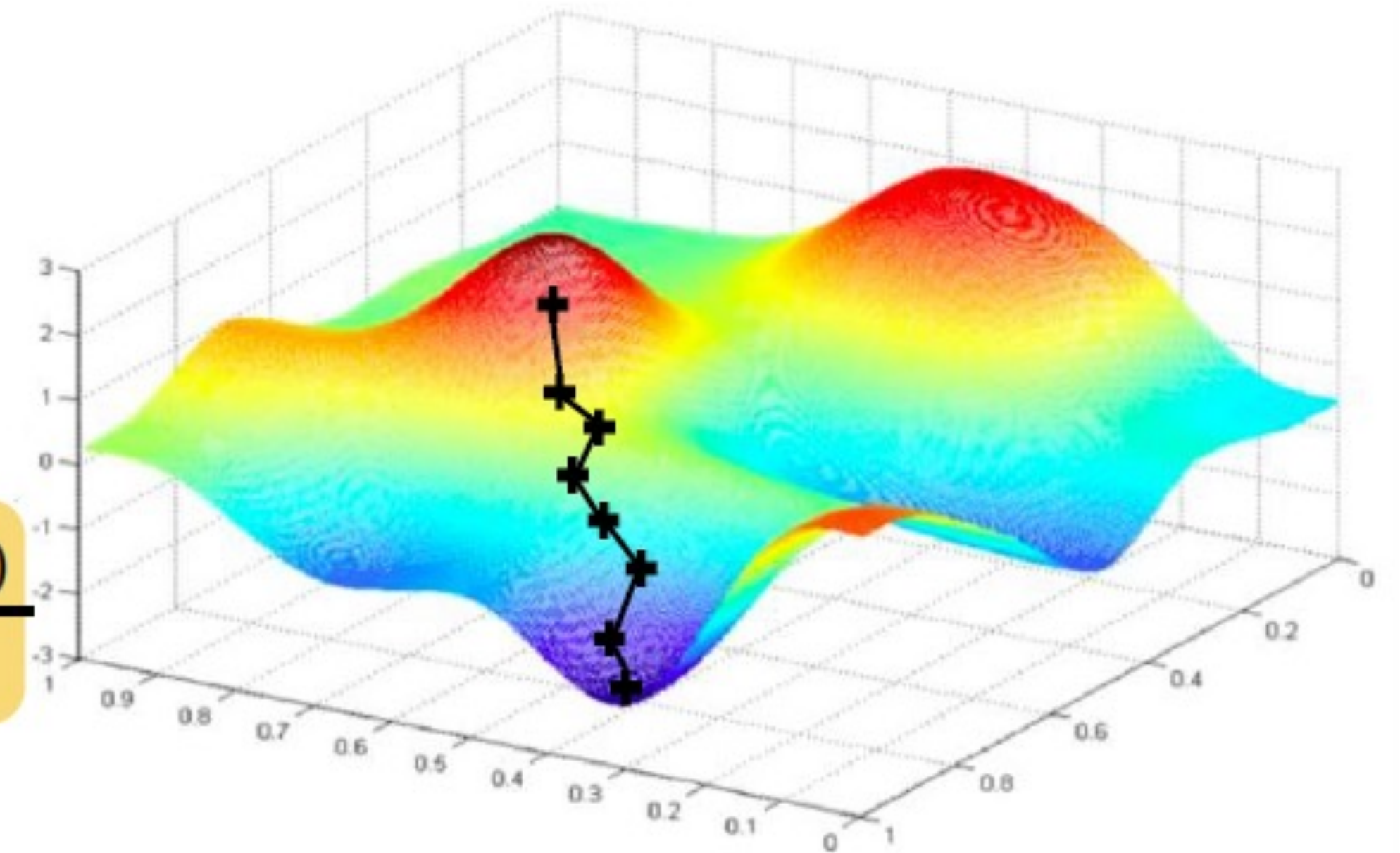
Easy to compute but
very noisy
(stochastic)!



Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
6. Return weights



Fast to compute and a much better estimate of the true gradient!

Stochastic Gradient Descent: Mini-batches

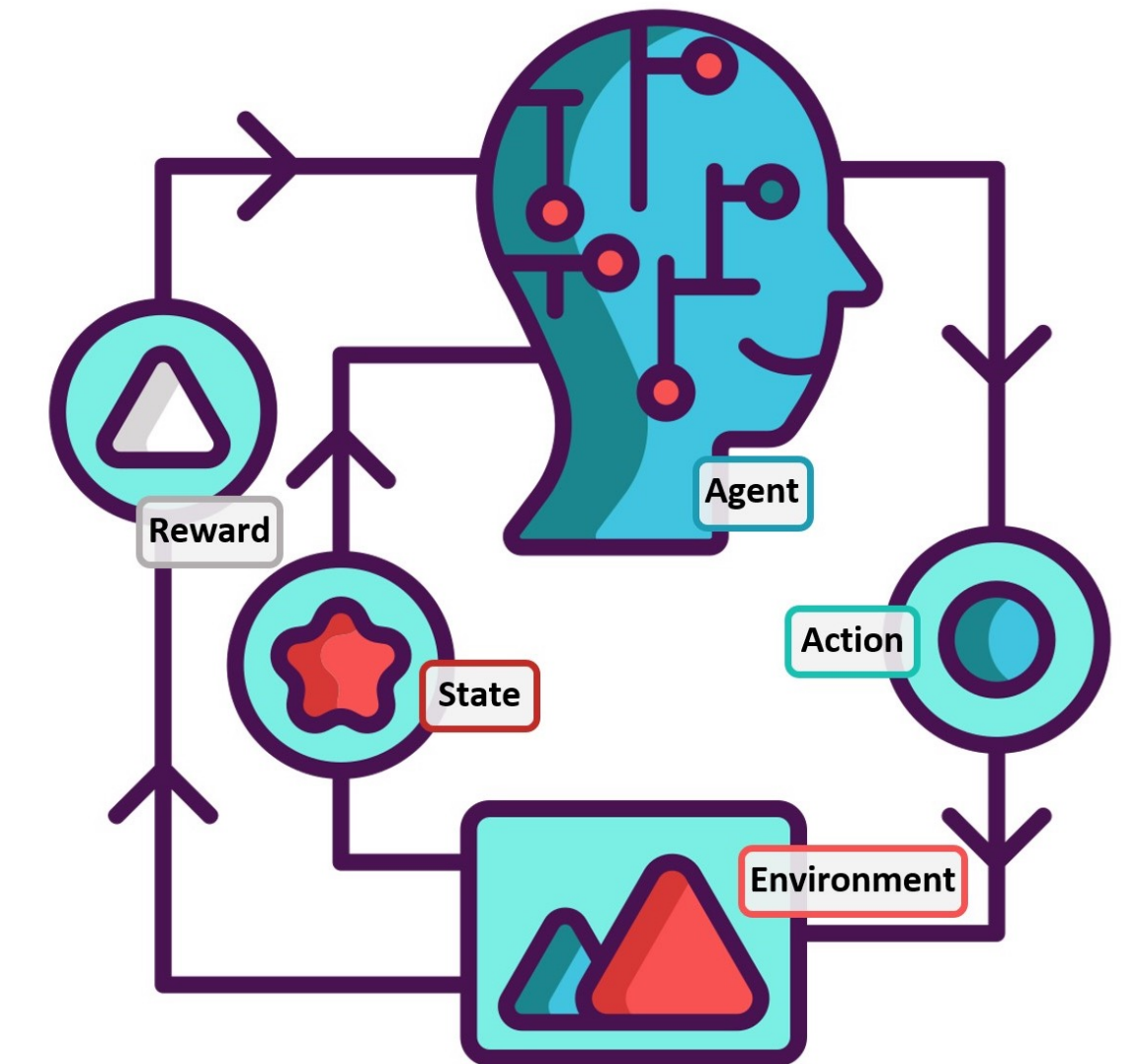
The set of B data points is called *mini-batch*

Mini-batches allow to accurate estimation of gradient, smoother convergence, larger learning rates

Mini-batches lead to fast training: computation can be parallelized and significant speed increases can be obtained on GPUs



Reinforcement Learning



**Thank you for your
attention!**

Gian Antonio Susto
Ruggero Carli

